

Server Manage 운영 위키

DECS 서버 관리 코드의 기능, 경계와 설계 의도

문서 기준: 2026-08-10 저장소 상태

소스: /home/hyrn/admin_wiki

목차

1. 전체 구조
2. container-images
3. kerberos-nfs
4. monitoring
5. remote-operations
6. server-state
7. infra
8. kdc-setup

System

원본: md/system/index.md

개요

System 영역은 **DECS GPU 서버와 사용자 container를 일관된 상태로 운영하기 위한 시스템 관리 코드**를 담당한다. 새로운 서버를 기존 서버와 같은 설정으로 구축하고, 사용자가 사용할 container와 image를 관리하며, 서버 전원부터 monitoring과 공유 storage 접근까지 운영에 필요한 전체 흐름을 유지하는 것이 목적이다.

주요 담당 범위는 다음과 같다.

- 모든 GPU 서버의 Docker, NVIDIA, Kubernetes와 network 공통 설정을 점검하고 신규 서버에도 같은 기준을 적용한다.
- 사용자 container의 생성·수정·삭제와 공통 container image를 관리한다.
- 서버 원격 부팅과 부팅 순서를 관리하여 사용자 관리 DB와 GPU 서버가 올바른 순서로 준비되도록 한다.
- 서버, GPU, container와 사용자 상태를 수집하여 장애와 이상 상태를 관측한다.
- AD/Kerberos 인증을 사용해 사용자가 NAS/NFS 공유 storage에 안전하게 접근하도록 한다.

즉, Backend나 Frontend 애플리케이션 기능이 아니라 **GPU 서버에서 사용자 작업 환경이 실제로 생성되고, 실행되고, 관측되고, storage에 연결되는 운영 기반**을 관리하는 영역이다.

목차

구성요소	상세 문서	PDF
전체 통합 매뉴얼	-	PDF 열기
전체 구조	현재 페이지	PDF 열기
container-images/	문서 열기	PDF 열기
server-state/	문서 열기	PDF 열기
remote-operations/	문서 열기	PDF 열기
monitoring/	문서 열기	PDF 열기
kerberos-nfs/	문서 열기	PDF 열기

container-images

원본: md/system/container-images/index.md, md/system/container-images/design.md, md/system/container-images/operations.md

역할: DECS GPU 컨테이너 이미지의 빌드 입력, 시작 시 런타임 구성, 검증을 소유한다.

이 문서는 container-images 문서의 시작점이다. **설계**는 이미지 variant manifest, Dockerfile, entrypoint 구성과 **user-lifecycle** 과의 계약을 설명하고, **운영**은 빌드·배포에 사용하는 명령, 테스트 전략과 변경 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	디렉터리 지도, manifest 기반 variant, Dockerfile/entrypoint 구성, user-lifecycle 계약
운영	빌드/배포 흐름, 테스트 전략, 변경 가이드, 운영 안전 수칙

처음 보는 사람은 **설계 문서**에서 이미지가 무엇을 책임지고 무엇을 책임지지 않는지 먼저 확인하고, 실제로 빌드하거나 새 variant를 추가해야 할 때 **운영 문서**로 이동한다.

container-images 설계

개요 · 운영

역할: DECS GPU 컨테이너 이미지의 빌드 입력, 시작 시 런타임 구성, 검증을 소유한다.

1. 책임 범위

container-images 는 CUDA/TensorFlow 조합별로 이미지를 만들되, 그 안의 구성(계정, SSH, Jupyter 등)은 모든 이 이미지에서 동일하게 유지한다. 이미지가 책임지는 것은 OS·CUDA·Python/Jupyter 패키지와 컨테이너 안의 계정, SSH, VNC(선택), Kerberos ccache 사용 환경이다.

이미지는 UID/GID를 결정하지 않는다. 사용자 identity, 포트, 홈 디렉터리와 Kerberos keytab은 **user-lifecycle** 와 호스트가 관리한다. 이 경계를 지켜야 이미지가 사용자 DB나 특정 서버에 종속되지 않는다.

2. 디렉터리 지도

경로	핵심 기능
<code>Dockerfile</code>	모든 variant가 공유하는 단일 이미지 정의
<code>image-variants.json</code>	base image, CUDA/TensorFlow, 최소 driver, alias의 권위 있는 목록
<code>entrypoint.sh</code>	실행 시 사용자·그룹·SSH·Jupyter·VNC·Kerberos 환경 구성
<code>scripts/variant_matrix.py</code>	manifest를 Docker/GitHub Actions build matrix로 변환
<code>scripts/build_variants.py</code>	로컬 build/push 명령 생성과 실행
<code>scripts/test_image_variants.py</code>	이미지 메타데이터, TensorFlow와 선택적 GPU smoke test
<code>scripts/test_uid_create_container.py</code>	<code>user-lifecycle</code> CLI와의 dry-run 계약 검증
<code>tests/</code>	root_squash, Kerberos, sudo와 build 설정 회귀 테스트
<code>.github/workflows/docker-publish.yml</code>	PR merge 또는 수동 실행 시 matrix build/push

3. 핵심 기능

3.1 manifest 기반 이미지 variant

`image-variants.json` 한 곳에 다음 항목을 둔다.

- variant ID와 base NVIDIA CUDA image
- CUDA, TensorFlow, Python, Ubuntu 버전
- TensorFlow 설치 패키지와 conda 패키지 제약
- 필요한 최소 NVIDIA driver
- `stable` 또는 `experimental` 지원 상태
- 날짜가 없는 사용자용 alias

현재 안정 계열은 CUDA 11.8, 12.2, 12.3, 12.5이며 CUDA 12.8/H200 계열은 실장비 검증 전까지 experimental이다. 빌드 도구와 GitHub Actions가 같은 manifest를 읽으므로 로컬과 CI의 variant가 달라지지 않는다.

3.2 단일 Dockerfile

`Dockerfile` 은 `BASE_IMAGE`, `CUDA_VERSION`, `TENSORFLOW_VERSION`, `MIN_NVIDIA_DRIVER`, `CONDA_PACKAGES` 등을 build argument로 받는다. 모든 variant는 공통으로 SSH, Kerberos client, 한국어 글꼴/입력기, Chrome, Xfce, TigerVNC/noVNC, Miniforge, Jupyter를 포함한다.

variant별 Dockerfile을 복제하지 않은 이유는 보안 패치나 공통 패키지 변경을 한 번만 적용하고, 차이는 manifest의 데이터로 검토하기 위해서다. 버전별 예외가 필요하면 먼저 manifest 값으로 표현하고, 정말 다른 설치 흐름일 때만 Dockerfile에 조건을 추가한다.

3.3 실행 시 identity 구성

`entrypoint.sh`의 주요 흐름은 다음과 같다.

1. 이미지 variant와 실제 host driver 정보를 출력한다.
2. `USER_ID`, `UID`, `GID`, `USER_GROUP`을 검증하고 기존 홈의 owner와 맞춘다.
3. supplemental group과 sudo 정책을 구성한다.
4. SSH 로그인과 Jupyter 설정을 만든다.
5. Kerberos 모드이면 전달받은 ccache를 사용자 환경에 연결한다.
6. Jupyter를 시작하고, 요청된 경우에만 VNC/noVNC를 시작한다.

호스트 mount가 `root_squash`인 환경에서는 root가 사용자 홈을 임의로 `chown` 할 수 없다. 그래서 이미 결정된 UID/GID로 사용자를 실행하고, 홈에 쓸 수 없는 root 작업을 최소화한다. 사용자 홈의 기존 비밀번호와 VNC password 파일도 재시작 때 보존한다.

3.4 제한적 sudo와 공유 helper

기본 `DECS_USER_SUDO_MODE=restricted`는 패키지 설치에 필요한 제한된 sudo는 허용하지만 UID 전환, mount, 권한 변경, root shell과 우회 가능한 interpreter 실행을 막는다.

사용자 그룹 공유는 관리자 sudo 대신 `group-dir-share` helper로 제공한다. 사용자 자신의 권한으로 디렉토리를 만들고 `2770`과 default ACL을 설정하므로 공유 기능과 identity 격리를 함께 유지한다.

Kerberos keytab과 ccache를 분리한 이유와 container credential 경계는 [Kerberos/NFS의 keytab과 ccache 모델](#)을 따른다.

3.5 GPU 호환성 표시와 강제 모드

이미지는 시작할 때 build에 기록된 최소 NVIDIA driver와 `nvidia-smi` 결과를 출력한다. 기본값은 경고 중심이며 `STRICT_CUDA_COMPAT=true`일 때만 최소 driver보다 낮은 host에서 시작을 실패시킨다.

기본을 강제 실패로 두지 않은 이유는 GPU가 없는 검증 환경이나 긴급 진단 container까지 막지 않기 위해서다. 운영 배포에서 호환성을 반드시 보장해야 하면 strict mode를 명시한다.

4. 사용자 생명주기와의 계약

`uidctl create-container`는 image 이름과 version tag, 최종 UID/GID, 포트와 home mount를 Docker 환경변수/옵션으로 전달한다. 이미지가 기대하는 주요 값은 다음과 같다.

값	의미
<code>USER_ID</code>	컨테이너 사용자 이름
<code>UID</code> , <code>GID</code>	DB·AD·NFS와 이미 일치해 검증된 숫자 identity
<code>USER_GROUP</code>	primary group 이름
<code>ENABLE_VNC</code>	VNC/noVNC opt-in

값	의미
KRB5CCNAME	컨테이너에 보이는 ccache 경로
DECS_KERBEROS_HOST_KEYTAB	호스트 관리 ticket을 기다리는 모드
DECS_USER_SUDO_MODE	disabled, restricted, allowed 중 하나

TARGET_UID 나 TARGET_GID 같은 두 번째 identity 체계를 만들지 않는다. 하나의 UID / GID 만 사용해야 container, NFS와 DB가 어긋나는 오류를 조기에 발견할 수 있다.

container-images 운영

개요 · 설계

1. 빌드와 배포 흐름

```
image-variants.json
|
+--> variant_matrix.py --> GitHub Actions matrix --> build/push
|
+--> build_variants.py -----> local build/push
|
+--> test_image_variants.py -----> smoke test
```

전체 명령 확인:

```
cd /home/jy/server_manage/container-images
python3 scripts/build_variants.py --dry-run
```

특정 variant build:

```
python3 scripts/build_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

registry push는 build 결과와 tag 목록을 확인한 뒤 --push 를 추가한다. 날짜 tag는 재현 가능한 배포 지점을 만들고, stable / latest alias는 현재 권장 variant를 가리킨다. 운영 기록에는 alias보다 날짜가 포함된 tag를 남기는 것이 좋다.

2. 테스트 전략

빠른 정적/회귀 테스트:

```
cd /home/jy/server_manage/container-images
bash tests/test_image_build_config.sh
bash tests/test_entrpoint_root_squash.sh
```

이미지 smoke test:

```
python3 scripts/test_image_variants.py \
  --variant cuda12.5-tf2.20-ubuntu22.04
```

실제 GPU까지 확인할 때만 `--gpu` 를 사용한다. 사용자 생성 계약은 실제 DB를 바꾸지 않는 `dry-run/print` 경로로 검증한다.

```
python3 scripts/test_uid_create_container.py \
  --variant cuda12.5-tf2.20-ubuntu22.04 \
  --print-only
```

3. 변경 가이드

새 variant 추가

1. `image-variants.json` 에 버전, base image와 최소 driver를 추가한다.
2. `python3 scripts/variant_matrix.py` 결과의 tag를 확인한다.
3. build `dry-run`과 shell 회귀 테스트를 실행한다.
4. 실제 image smoke test와 가능하면 대상 GPU host test를 수행한다.
5. 검증 전에는 `support: experimental` 과 명시적인 alias를 사용한다.

공통 런타임 변경

`Dockerfile` 변경과 `entrypoint.sh` 변경을 구분한다. package/파일은 build 시점에 넣고, 사용자 UID나 mount처럼 컨테이너마다 달라지는 항목만 `entrypoint` 에서 처리한다. `entrypoint` 변경은 재시작 시 idempotent한지, 기존 홈의 파일을 덮어쓰지 않는지, `root_squash` 환경에서 동작하는지를 함께 확인한다.

4. 운영 안전 수칙

- 실제 사용자 비밀번호를 image layer나 manifest에 넣지 않는다.
- keytab을 image 또는 container mount로 제공하지 않는다.

- `latest` 만 기록하지 말고 장애 분석이 가능한 날짜 tag를 남긴다.
- `experimental variant`를 `stable alias`로 바꾸기 전에 실장비 GPU test를 한다.
- `sudo` 완화는 `ccache`와 `host bind mount`에 대한 우회 가능성을 먼저 검토한다.
- 로컬 source 변경과 이미 registry에 push된 image를 구분해서 진단한다.

Kerberos/NFS

원본: md/system/kerberos-nfs/index.md, md/system/kerberos-nfs/design.md, md/system/kerberos-nfs/operations.md, md/system/kerberos-nfs/debugging/index.md, md/system/kerberos-nfs/debugging/lab9-why-sec-krb5.md, md/system/kerberos-nfs/debugging/nfs-v41-session-slot-stuck.md

역할: FARM/LAB에서 AD Kerberos로 사용자를 인증하고, 같은 UID/GID로 NFS 공유 스토리지에 접근하게 하는 구조와 운영 기준을 설명한다.

이 문서는 Kerberos/NFS 문서의 시작점이다. 처음 읽는 사람은 **설계**에서 전체 인증 흐름을 먼저 이해한 뒤 **운영**에서 생성·점검·복구 명령을 확인한다. 과거 장애의 증거와 재현 실험은 **디버깅 로그**에서 확인한다. 환경별 실제 값과 최종 실행 절차는 인프라 저장소의 FARM/LAB canonical runbook에서 확인한다.

문서 구성

문서	읽어야 하는 경우	핵심 내용
현재 페이지	전체 범위와 문서 위치를 빠르게 확인할 때	목적, 원칙, 관리 범위
설계	인증이 어느 서버와 프로세스를 거치는지 이해할 때	FARM/LAB NFS version·principal 차이, keytab 발급, RPCSEC_GSS, 권한 판정
운영	계정/컨테이너 생성, timer 확인, mount·KVNO 장애를 처리할 때	Synology/Linux storage 차이, 명령, 점검표, 수동 repair/rotation
디버깅 로그	재현된 장애의 원인과 실험 조건·명령·증거를 확인할 때	장애별 증상, 가설, 재현, 판정, 복구·예방

한눈에 보는 구조

관리 서버(uidctl)

- └─ AD DC: 사용자·RFC2307·keytab 생성
- └─ 계산 호스트: root-only keytab + ccache refresh timer
- └─ NAS/storage: NFS service keytab + 사용자 home
- └─ container: keytab이 아닌 ccache만 사용

사용자 I/O

container process -> host kernel NFS client -> rpc.gssd
-> AD KDC ticket -> NAS svcgssd/NFS -> UID/GID 권한 판정

현재 구성에서 지켜야 할 원칙은 다음과 같다.

- 사용자 keytab은 장기 자격증명이므로 계산 호스트의 `/etc/decs-krb/keytabs/<username>.keytab`에 `root:root0400`으로만 둔다.
- 컨테이너에는 keytab을 넣지 않는다. 호스트가 만든 `/run/user/<uid>/krb5cc`와 읽기 전용 `/etc/krb5.conf`만 bind mount한다.
- 컨테이너가 시작되기 전에 사용자별 `decs-krb-refresh@<username>.timer`를 활성화하고 유효한 ccache를 한 번 발급한다.
- NFS mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 쓴다. FARM의 source는 `nas.farm.decs.internal:/volume1/share`다.
- `addr=100.100.100.120`은 FQDN이 해석된 실제 전송 주소로 사용할 수 있다. source 자체를 `100.100.100.120:/...`로 바꾸면 안 된다.
- 기본 security flavor는 `sec=krb5`다. `krb5i`와 `krb5p`는 무결성·암호화가 명시적으로 필요한 경로에서 성능 비용과 함께 선택한다.

Identity 불변 조건

한 사용자에게 대해 다음 숫자가 같아야 한다.

```
UID DB ubuntu_uid
= AD RFC2307 uidNumber
= NFS client에서 관측한 home owner UID
= container UID
```

primary GID도 같은 원칙 적용

이 조건을 확인하지 않고 NAS에서 `chown`만 반복하면 AD, DB, NAS의 identity가 더 어긋날 수 있다. 계정 생성 흐름은 Docker와 DB를 확정하기 전에 AD/NAS/host identity와 실제 Kerberos NFS 쓰기를 검증한다.

모듈별 관리 범위

작업	담당 모듈
AD 사용자·그룹, 사용자 keytab, host ccache timer, container 생성	<code>user-lifecycle</code>
공통 host Kerberos/NFS desired state와 fstab	<code>server-state</code>
NAS NFS service account·SPN·service keytab의 수동 변경	<code>kerberos-nfs</code>
GSS readiness, keytab drift 관측, mount 상태, canary, forensics	<code>monitoring</code>
ccache 소비, 시작 전 credential 확인, restricted sudo	<code>container-images</code>

관측 실패가 곧바로 AD password 변경이나 NAS service 재시작으로 이어지지 않게 읽기 전용 monitoring과 상태 변경 작업을 분리한다.

기준 문서와 코드

- [FARM canonical runbook](#)
- [LAB canonical runbook](#)
- [user-lifecycle Kerberos 구현](#)
- [container 생성 구현](#)
- [host Kerberos/NFS desired state](#)
- [Kerberos/NFS keytab health check](#)
- [NFS GSS monitoring](#)

실제 endpoint, principal, mount option이 바뀌면 이 위키보다 canonical runbook과 코드 설정을 먼저 갱신하고, 검증 시각과 대상 host를 함께 기록한다.

Kerberos/NFS 설계

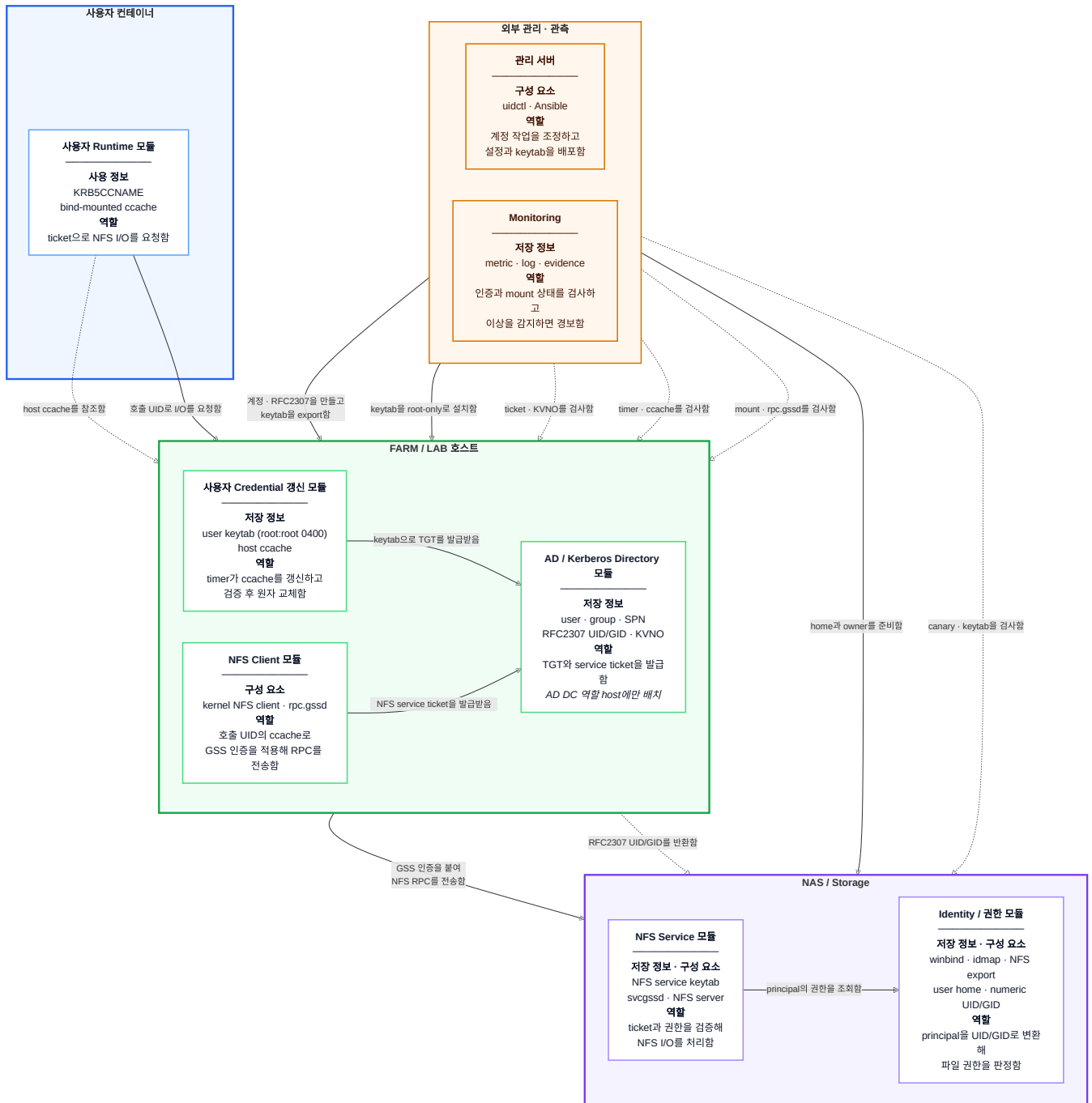
이 문서는 keytab이 만들어지는 시점부터 사용자가 NFS 파일을 읽고 쓰는 시점까지, 어느 서버의 어떤 객체와 프로세스가 관여하는지를 설명한다.

1. 설계 목표

- 비밀번호를 컨테이너에 저장하지 않고 사용자별 Kerberos 인증을 제공한다.
- DB, AD, 계산 호스트, NAS와 컨테이너가 같은 numeric UID/GID를 사용한다.
- NFS 서버는 FQDN 기반 service principal로 인증하고 client는 `sec=krb5` 로 RPCSEC_GSS context를 만든다.
- keytab 같은 장기 자격증명의 노출 범위를 host root로 제한한다.
- ticket 갱신과 컨테이너 lifecycle을 분리하여 컨테이너 재시작 중에도 credential을 안정적으로 유지한다.
- monitoring은 drift를 관측하되 공유 NAS와 AD를 자동으로 변경하지 않는다.

2. 전체 구성

다이어그램의 **굵은 바깥 제목은 배치 영역**이고, 영역 안 각 카드의 **굵은 첫 줄은 모듈 이름**이다. 카드 안의 **저장 정보 / 구성 요소**에는 명사만 적고, **역할**에는 **주체가 무엇을 하는지 동사로** 적었다. 화살표에는 출발 모듈이 요청하거나 제공하는 동작을 표시했으며, 점선은 credential 참조나 읽기 전용 조회·관측을 나타낸다. AD/KDC는 모든 계산 host에 있는 것이 아니라 FARM/LAB 중 **AD DC 역할을 맡은 host에만** 있다. 다이어그램에는 핵심 값과 동작만 표시하고 상세 책임은 아래 표에서 설명한다.



구성요소별 책임

영역	모듈	내부 구성 요소	역할
외부 관리	계정 생성 조정	uidctl, Ansible runner	계정 생성 절차를 조정하고 AD, host, storage에 필요한 상태를 준비
FARM/LAB 호스트	AD/Kerberos Directory	Samba AD DC, KDC, 사용자·그룹·SPN·RFC2307 객체	principal과 Unix identity 저장, keytab export, TGT/service ticket 발급; AD DC 역할 host에만 배치
FARM/LAB 호스트	사용자 Credential 갱신	user keytab, decs-krb-refresh@.service/.timer, host ccache	root-only keytab으로 ccache를 만들고 검증 후 원자 교체

영역	모듈	내부 구성 요소	역할
FARM/LAB 호스트	NFS Client	kernel NFS client, <code>rpc.gssd</code>	호출 UID의 ccache로 RPCSEC_GSS context를 만들고 NFS RPC 전송
사용자 컨 테이너	사용자 Runtime	사용자 process, bind-mounted ccache, <code>KRB5CCNAME</code>	keytab 없이 host가 갱신한 ticket을 사용하여 NFS 파일 I/O 수행
NAS/Stora ge	NFS Service	service keytab, <code>svcgssd</code> , NFS server	<code>nfs/<fqdn>@<realm></code> service ticket을 수락하고 NFS 요청 처리
NAS/Stora ge	Identity/권한	Samba, winbind, idmap, NFS export와 filesystem	Kerberos principal을 RFC2307 UID/GID로 해석하여 최 종 파일 권한 판정
외부 관측	Monitoring	readiness, KVNO checker, mount probe, canary, forensics	AD/host/storage 상태를 읽기 전용으로 수집하고 drift와 장애를 경보

3. FARM과 LAB의 현재 설정 차이

FARM과 LAB은 “사용자 keytab은 host root만 보관하고 container에는 ccache만 전달한다”는 credential 모델은 같다. 그러나 realm, NFS server 구현, mount source, service principal과 NFS protocol version은 서로 다르다. 한 환경의 값을 다른 환경에 그대로 복사하면 KDC가 잘못된 service ticket을 발급하거나 NFS server가 ticket을 복호화하지 못한다.

3.1 Identity와 endpoint

항목	FARM	LAB
Kerberos realm	<code>FARM.DECS.INTERNAL</code>	<code>LAB.DECS.INTERNAL</code>
DNS domain / NetBIOS	<code>farm.decs.internal</code> / <code>FARM</code>	<code>lab.decs.internal</code> / <code>LAB</code>
AD/KDC	<code>dc1.farm.decs.internal</code> ; farm2가 primary이고 farm6·farm7을 replica 경로로 사용	<code>dc1.lab.decs.internal</code> ; lab2
NFS server 구현	Synology NAS가 FARM AD domain member로 동작	일반 Linux storage host가 LAB AD domain member와 NFS server로 동작
관리 SSH	<code>192.168.2.30:6954</code>	<code>192.168.1.20:6953</code>
NFS data 주소	<code>100.100.100.120</code>	<code>100.100.100.100</code>
운영 mount source	<code>nas.farm.decs.internal:/volume1/share</code>	<code>lab- storage.lab.decs.internal:/294t/dcloud/share</code>
계산 host mount target	<code>/home/tako<번호>/share</code>	<code>/home/tako<번호>/share</code>

항목	FARM	LAB
사용자 home 원본	/volume1/share/user-share/<username>	/294t/dcloud/share/user-share/<username>

관리 주소는 SSH와 장비 관리에만 사용한다. NFS source에는 반드시 service principal의 hostname을 사용한다. LAB의 /294t/dcloud/share/test_krb 와 /294t/health/nfs-gss-canary 는 각각 격리 PoC와 canary용 export이며 운영 사용자 mount source인 /294t/dcloud/share 와는 용도가 다르다.

3.2 NFS와 Kerberos service identity

항목	FARM	LAB
현재 NFS version	vers=4.0	vers=4.1
기본 security flavor	sec=krb5	sec=krb5
server export flavor	production storage IP에는 sec=krb5:krb5i:krb5p ; client가 krb5 를 선택	현재 LAB baseline은 sec=krb5
NFS service principal	nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL	nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
추가 principal	nfs/NAS@FARM.DECS.INTERNAL , nfs/nas.ai lab.dgu@AILAB.DGU 를 NAS keytab에서 함께 보존	LAB profile에는 추가 acceptor 설정 없음
NFS SPN 등록 계정	전용 AD user svc-nfs-farm ; Synology machine account NAS\$ 와 분리	Linux storage computer account LAB-STORAGE\$
NFS service keytab	/etc/nfs/krb5.keytab 이 ticket 검증 기준이며 /etc/krb5.keytab 도 checker가 비교	/etc/krb5.keytab
server GSS process	Synology의 /usr/sbin/svcgssd ; 여러 realm의 nfs/* 를 받아야 하므로 -p 로 하나를 고정하지 않음	Linux rpc-svcgssd 또는 NFS server가 관리하는 rpc.svcgssd
client machine principal 예	FARM8\$@FARM.DECS.INTERNAL	LAB9\$@LAB.DECS.INTERNAL
GSS canary	전용 export가 없어 현재 비활성	lab-storage.lab.decs.internal:/294t/health/nfs-gss-canary , vers=4.1,sec=krb5 로 활성화

사용자 principal도 realm에 따라 <username>@FARM.DECS.INTERNAL 또는 <username>@LAB.DECS.INTERNAL 이 된다. 사용자 keytab, host machine keytab과 NFS service keytab은 서로 다른 principal과 lifecycle을 가지므로 한 파일로 합쳐서는 안 된다.

3.3 NFS version이 다른 이유

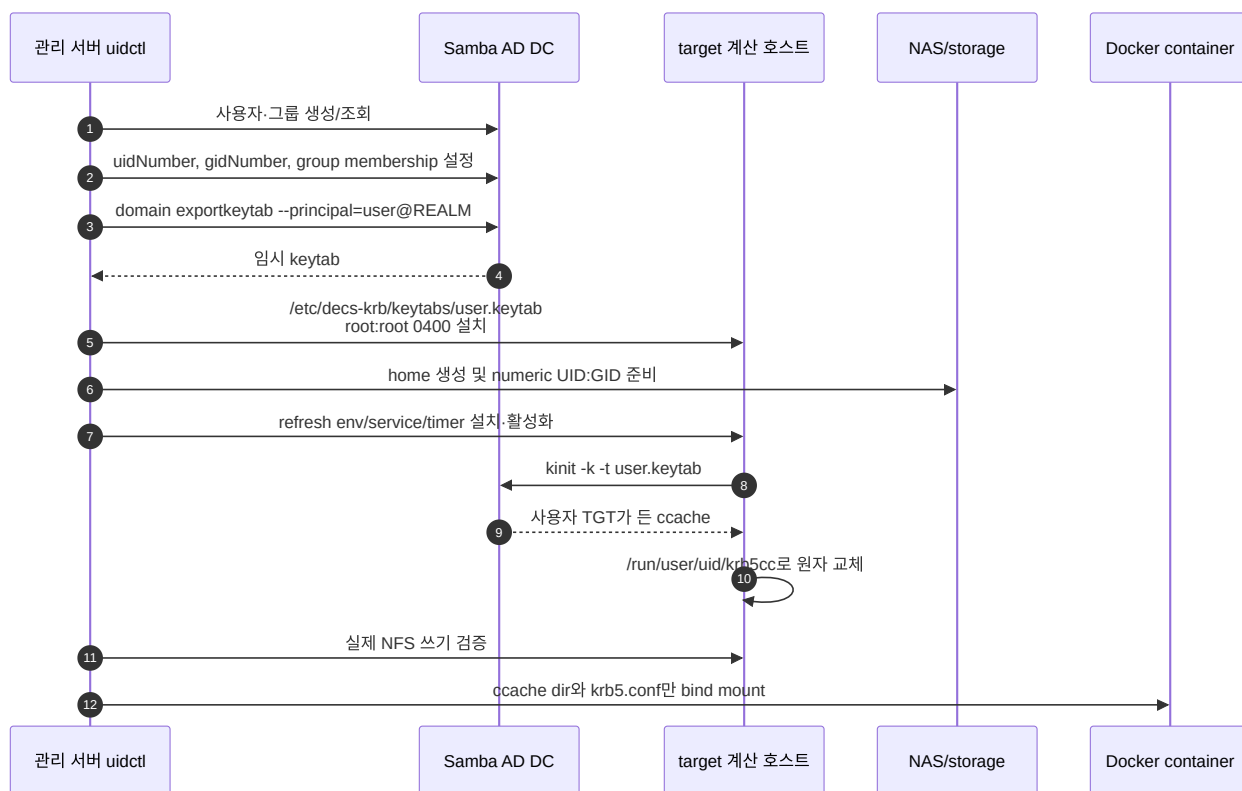
FARM의 Synology NAS에는 NFSv4.1 기능이 활성화되어 있지만, rollout에서 `vers=4.1` 과 `RPCSEC_GSS`를 함께 사용했을 때 timeout 또는 access denied가 발생했다. 따라서 현재 운영 baseline은 `vers=4.0,sec=krb5,hard,timeo=600,retrans=2` 다. `fstab`에서 1 MiB `rsz/size` 를 요청해도 실제 연결에서는 Synology와 128 KiB로 협상될 수 있다.

LAB은 Linux storage의 NFSv4.1을 사용한다. NFSv4.1 session/slot 동작은 server kernel 구현에 따라 달라지므로, 구형 kernel의 idmap deferral과 slot 고착 문제는 [NFSv4.1 session slot 고착](#)에서 별도로 관리한다. 두 환경의 version은 각각 검증된 값이므로 LAB을 임의로 v4.0으로 낮추거나 Synology NAS를 검증 없이 v4.1로 올리지는 않는다.

현재 설정값은 [server-state Kerberos/NFS playbook](#)에 정의되어 있다. Monitoring에서 기대하는 값은 `exporter` 변수에서 확인한다.

4. 사용자 keytab 발급 흐름

keytab은 컨테이너 안에서 만들지 않는다. AD DC에서 export한 뒤 target 계산 호스트의 root-only 경로에 설치한다.



실제 생성 구현은 다음 순서로 동작한다.

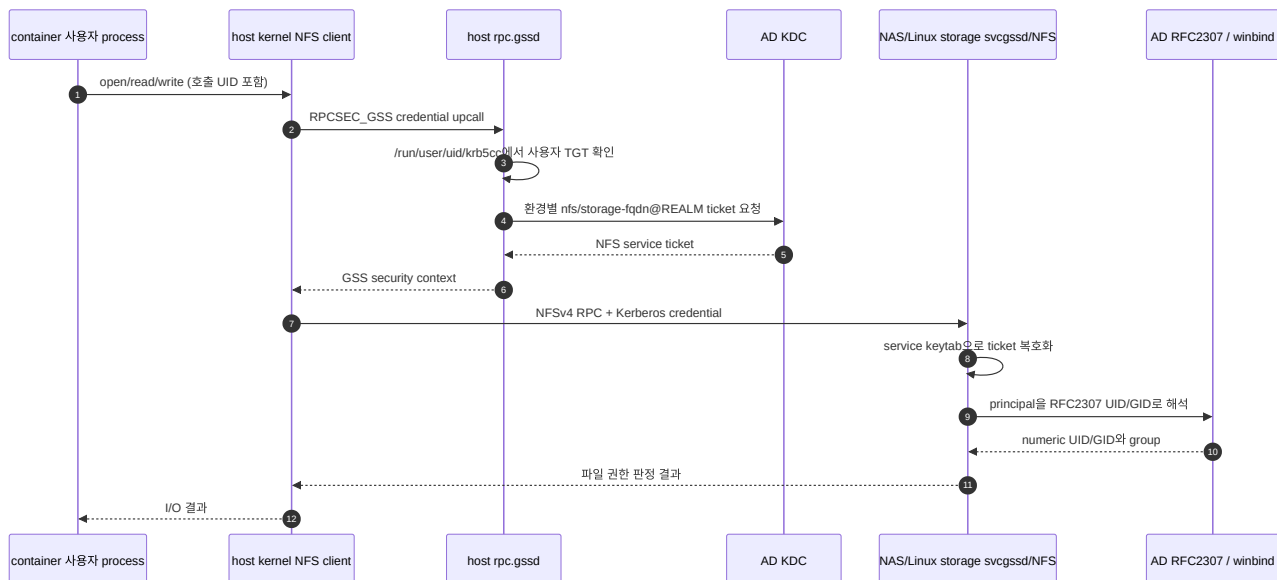
1. DB/기존 AD/storage 값을 사용해 UID/GID를 선택한다.
2. AD user/group과 RFC2307 `uidNumber`, `gidNumber` 를 보장한다.
3. AD DC에서 사용자 keytab을 임시 파일로 export하고 `0400` 으로 만든다.
4. target이 다른 host이면 Ansible `fetch` 와 `copy` 로 root-only keytab을 옮긴다.
5. NAS/storage home을 동일 UID/GID로 준비한다.
6. target host에 ccache directory, refresh helper, service/timer를 만든다.
7. host에서 사용자 ccache를 발급하고 NFS owner와 실제 write/delete를 검증한다.

8. 검증 후에만 컨테이너와 DB record를 확정한다.

관련 코드는 [AD identity](#)와 [keytab 생성](#), [container 생성 transaction](#), [credential](#) 경로 모델에서 확인한다.

5. 실제 NFS 인증 흐름

keytab 발급과 매 파일 접근의 인증은 서로 다른 흐름이다. 평상시 파일 I/O에서 컨테이너가 keytab을 직접 읽는 일은 없다.



FARM에서는 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL`, LAB에서는 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL` ticket을 요청한다. 이 값은 mount source의 hostname에서 결정되며 단순 표시용 설정이 아니다.

인증이 실패한 단계에 따라 증상이 달라진다.

실패 위치	대표 증상
사용자 keytab/ccache	<code>kinit</code> 또는 <code>klist</code> 실패, 사용자 접근만 <code>Permission denied</code>
DNS/FQDN/SPN	<code>kvno nfs/<fqdn></code> 실패, IP source mount에서 principal 불일치
host machine keytab/ <code>rpc.gssd</code>	부팅 시 mount 실패, readiness의 keytab/kinit/rpc-gssd stage 실패
NAS service keytab/KVNO	여러 client가 동시에 GSS 실패, <code>kvno -k</code> 복호화 실패
RFC2307/idmap	인증은 되지만 owner가 <code>nobody</code> 이거나 UID/GID가 달라 쓰기 거부
NFS transport/kernel	<code>not responding</code> , D-state, mount probe timeout

6. Keytab과 ccache를 분리한 이유

구분	keytab	ccache
의미	principal의 장기 비밀키	이미 발급된 TGT/service ticket 묶음
새 ticket 발급	가능	제한된 renew 기간 안에서만 가능
유출 영향	rotation할 때까지 반복 악용 가능	ticket lifetime/renew lifetime에 제한
저장 위치	host root-only	/run/user/<uid>/krb5cc, 사용자 0600
container 전달	금지	bind mount하여 전달

컨테이너 삭제만으로 유출된 keytab을 무효화할 수 없기 때문에 image, Docker volume, Kubernetes Secret에 사용자 keytab을 그대로 넣지 않는다. 호스트 refresh service는 임시 ccache에서 klist 검증을 끝낸 뒤 소유권 uid:gid, mode 0600을 적용하고 최종 경로로 원자 교체한다.

현재 코드가 구현하는 대상은 Docker container다. Kubernetes Pod도 같은 원칙을 적용해야 하지만, Pod가 다른 node로 이동할 수 있으므로 단순 hostPath와 특정 node의 systemd timer에 의존해서는 안 된다. Pod 지원을 추가할 때는 node 배치, credential 발급 controller/CSI, rotation과 Pod 종료 시 정리를 별도 설계해야 한다.

7. Ticket 갱신 설계

decs-krb-refresh@<username>.timer는 기본적으로 boot 2분 뒤 시작하고 사용자별 설정 주기(현재 일반적으로 1시간)마다 oneshot service를 실행한다.

```
유효 ccache 있음
├─ renew deadline이 margin보다 멀 -> 임시 복사본에서 kinit -R
└─ deadline 임박/renew 실패      -> root-only keytab으로 새 ticket 발급

발급 결과 -> klist 검증 -> uid:gid 0600 -> 최종 ccache로 atomic move
```

기본 재발급 여유는 DECS_KRB_REISSUE_BEFORE_SECONDS=86400 (24시간)이다. 기존 ticket의 PAC/group 정보는 renew 시 유지될 수 있으므로 AD group 변경 뒤에는 keytab을 이용한 fresh login을 한 번 강제해야 한다.

8. NFS service identity와 KVNO

FARM NFS SPN은 Synology domain member의 machine account NAS\$가 아니라 전용 AD service account svc-nfs-farm에 등록되어 있다.

```
nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
nfs/NAS@FARM.DECS.INTERNAL
```

과거에는 `NAS$` machine password가 주기적으로 변경될 때 KVNO도 바뀌어 NAS의 정적 NFS keytab과 어긋날 수 있었다. 이를 service identity와 domain membership의 lifecycle 결합 문제로 보고 NFS SPN을 전용 계정으로 분리했다.

현재 `svc-nfs-farm`에는 자동 password expiration/rotation timer가 없다. 따라서 “NAS KVNO가 지금도 주기적으로 바뀐다”가 아니라 **과거 자동 변경 문제를 전용 계정으로 제거했고, 현재 KVNO는 관리자가 명시적으로 rotation할 때만 바뀐다**가 정확한 운영 모델이다. 별도 5분 checker는 변경을 만들지 않고 AD KVNO와 두 NAS keytab의 일치만 확인한다.

NAS의 `svcgssd`는 FARM과 AILAB principal을 한 keytab에서 받아들이므로 `-p`로 FARM principal 하나만 강제하지 않는다. keytab repair 시에도 AILAB acceptor를 보존한다.

LAB은 Synology NAS용 전용 service account와 keytab merge 절차를 사용하지 않는다. 현재 LAB keytab checker profile은 `LAB-STORAGE$` computer account의 KVNO와 Linux storage의 `/etc/krb5.keytab` 안 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL`을 비교한다. FARM repair나 rotation script를 LAB storage에 실행하면 안 된다.

9. FQDN mount와 service principal

Kerberos의 NFS service identity는 host-based principal이다. FARM mount source는 다음처럼 principal의 hostname과 같아야 한다.

```
mount source: nas.farm.decs.internal:/volume1/share
service SPN:  nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL
transport:    addr=100.100.100.120
```

`100.100.100.120:/volume1/share`를 source로 쓰면 client가 IP 기반 service identity를 찾으려 하거나 기대한 FQDN SPN과 연결하지 못할 수 있다. 반면 source를 FQDN으로 유지한 상태에서 runtime option에 `addr=100.100.100.120`이 보이는 것은 정상이다. 관리 SSH 주소 `192.168.2.30`은 NFS transport로 사용하지 않는다.

LAB도 같은 규칙을 적용한다.

```
mount source: lab-storage.lab.decs.internal:/294t/dcloud/share
service SPN:  nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL
transport:    100.100.100.100
```

LAB 관리 SSH 주소 `192.168.1.20`이나 data IP `100.100.100.100:/294t/dcloud/share`를 mount source로 직접 쓰지 않는다.

10. 보안 flavor와 권한 모델

- `sec=krb5`: 사용자/서비스 인증. RPC payload 무결성·암호화 wrapping 없음.
- `sec=krb5i`: 인증 + RPC payload 무결성.

- `sec=krb5p`: 인증 + 무결성 + privacy encryption.

현재 내부 FARM/LAB 기본은 `sec=krb5` 다. packet capture에 payload가 포함될 수 있으므로 NFS forensics 산출물은 root-only로 보관한다.

Kerberos 인증 뒤 최종 파일 권한은 numeric UID/GID와 mode/ACL로 결정된다. 컨테이너 root가 host credential 경계를 우회하지 못하도록 Kerberos mode에서는 `DECS_USER_SUDO_MODE=restricted` 를 함께 사용한다. 다만 restricted sudo는 완전한 sandbox가 아니므로 강한 격리가 필요하면 sudo와 `SYS_ADMIN` , host bind mount 범위도 별도로 줄여야 한다.

11. 설정과 구현 위치

관심사	기준 코드/문서
FARM endpoint, SPN, mount option, NAS keytab	FARM runbook
LAB topology와 rollout gate	LAB runbook
사용자 AD/keytab/ccache 명령 생성	commands.py
keytab·ccache·home 경로	paths.py
create-container transaction과 Docker bind	create_container.py
container credential 대기와 restricted sudo	entrypoint.sh
host machine keytab/readiness/fstab	kerberos_nfs_client_recovery.yml
NFS GSS readiness와 recovery	cluster-monitor-exporter scripts
NAS service keytab drift checker	check-nfs-keytab.sh

Kerberos/NFS 운영

이 문서는 현재 Docker 기반 FARM/LAB 환경의 일상 점검과 장애 대응 절차를 설명한다. 환경별 endpoint와 적용 상태는 반드시 canonical runbook에서 다시 확인한다.

1. 운영 원칙

1. 관측과 변경을 분리한다. 5분 keytab checker는 자동 repair/rotation을 하지 않는다.
2. 사용자 keytab은 host root-only로 두고 container에는 ccache만 전달한다.
3. 컨테이너를 시작하기 전에 refresh timer와 최초 ccache 발급을 완료한다.
4. mount source는 FQDN을 사용하고 IP는 `addr=` transport 값으로만 사용한다.

5. `findmnt`, `klist`, `kvno`, readiness 순서로 원인을 좁힌 뒤 service restart나 remount를 마지막 수단으로 사용한다.
6. UID/GID 불일치는 AD·DB·NFS 숫자를 비교한 뒤 고친다. 먼저 `chown` 하지 않는다.

2. Kerberos container 생성 (추후 자동화 시스템으로 대체될 예정)

대표 CLI 형태는 다음과 같다. 실제 image/version, 사용자 정보와 대상 server는 요청에 맞게 지정한다.

```
cd /home/jy/server_manage/user-lifecycle/script

python3 -B -m uid_manager.cli create-container \
  --server-id FARM8 \
  --name "사용자 이름" \
  --username <username> \
  --group <groupname> \
  --image <image> \
  --version <version> \
  --created-by <operator> \
  --email <email> \
  --phone <phone> \
  --enable-kerberos
```

먼저 `--dry-run`으로 UID/GID, target host, mount source, container command를 확인하는 것을 권장한다. 기존 사용자 password/key를 의도적으로 바꿀 때만 `--rotate-kerberos-keytab`을 추가한다. 이 옵션은 AD user password와 KVNO를 바꾸므로 단순 재배포 옵션으로 사용하지 않는다.

생성 transaction은 다음 조건을 모두 통과한 후 Docker/DB를 확정해야 한다.

- AD `uidNumber/gidNumber`가 선택한 DB UID/GID와 일치한다.
- target host keytab이 `root:root 0400`이다.
- ccache timer가 enabled/active이고 최초 ccache가 유효하다.
- NAS/storage home의 numeric owner가 기대한 UID/GID다.
- 새로 만든 사용자 credential로 host NFS write/delete가 성공한다.
- Docker에는 ccache directory와 읽기 전용 `krb5.conf`만 전달된다.

구현은 `create_container.py`와 Kerberos command builder에서 확인한다.

3. Host keytab과 ccache 확인

파일 권한

```
sudo stat -c '%U:%G %a %n' \  
    /etc/decs-krb/keytabs/<username>.keytab \  
    /etc/decs-krb/refresh.d/<username>.env  
  
sudo stat -c '%U:%G %a %n' /run/user/<uid> /run/user/<uid>/krb5cc
```

기대값은 다음과 같다.

```
keytab:      root:root 400  
refresh env: root:root 600  
ccache dir: <uid>:<gid> 700  
ccache:      <uid>:<gid> 600
```

timer와 ticket

```
systemctl list-timers 'decs-krb-refresh@*.timer'  
systemctl status 'decs-krb-refresh@<username>.timer' --no-pager  
sudo systemctl start 'decs-krb-refresh@<username>.service'  
sudo journalctl -u 'decs-krb-refresh@<username>.service' -n 100 --no-pager  
  
sudo -u '#<uid>' env KRB5CCNAME=FILE:/run/user/<uid>/krb5cc \  
    klist -c FILE:/run/user/<uid>/krb5cc
```

컨테이너 생성 때 timer unit을 설치·enable/start하고, 기존 ccache가 유효하지 않으면 oneshot service를 즉시 실행하도록 구현되어 있다. timer만 존재하고 최초 ccache가 없는 상태에서 컨테이너를 먼저 시작하면 Kerberized home 접근이 실패할 수 있다.

AD group을 변경했다면 renew만 기다리지 말고 fresh ticket을 발급한다.

```
sudo rm -f /run/user/<uid>/krb5cc  
sudo systemctl start 'decs-krb-refresh@<username>.service'  
sudo -u '#<uid>' klist -c FILE:/run/user/<uid>/krb5cc
```

4. 컨테이너 credential 확인

```
docker inspect <container> --format '{{range .Config.Env}}{{println .}}{{end}}' |  
grep -E '^(KRB5CCNAME|DECS_KRB5_PRINCIPAL|DECS_KERBEROS_ENABLED)='  
  
docker exec --user <username> <container> \  
sh -lc 'klist -c "$KRB5CCNAME" && touch ~/.__krb_test && rm ~/.__krb_test'
```

다음은 잘못된 상태다.

- container mount나 image 안에 *.keytab 이 있음
- KRB5CCNAME 이 host와 공유되지 않은 임시 경로를 가리킴
- container UID가 DB/AD UID와 다름
- Kerberos mode인데 unrestricted sudo와 광범위한 host mount를 함께 허용함

현재 저장소에는 Kubernetes Pod용 credential controller가 구현되어 있지 않다. Pod 운영을 추가한다면 Pod 생성 시 “어느 node에서 누가 keytab을 보관하고 ccache를 갱신할지”부터 구현해야 하며, Docker용 systemd timer 명령을 그대로 복사해 완료로 간주하면 안 된다.

5. NFS mount source 확인

환경마다 source, NFS version과 service principal이 다르다.

FARM의 올바른 형태는 다음과 같다.

```
nas.farm.decs.internal:/volume1/share /home/tako8/share nfs4  
defaults,vers=4.0,rsize=1048576,wsize=1048576,sec=krb5,proto=tcp,hard,timeo=600,retrans=2,addr=100.100.100.120,  
_netdev,exec,nouser 0 0
```

LAB의 올바른 형태는 다음과 같다. 실제 fstab에는 rpc-gssd 와 decs-kerberos-nfs-ready.service 에 대한 x-systemd.requires/after option도 playbook이 함께 추가한다.

```
lab-storage.lab.decs.internal:/294t/dcloud/share /home/tako9/share nfs4  
defaults,vers=4.1,sec=krb5,proto=tcp,hard,_netdev,exec,nouser 0 0
```

핵심은 source가 각 realm의 NFS service principal과 일치하는 FQDN이라는 점이다.

FARM 올바른: `nas.farm.decs.internal:/volume1/share`
FARM 잘못된: `100.100.100.120:/volume1/share` 또는 `192.168.2.30:/volume1/share`
FARM 정상: runtime option의 `addr=100.100.100.120`

LAB 올바른: `lab-storage.lab.decs.internal:/294t/dcloud/share`
LAB 잘못된: `100.100.100.100:/294t/dcloud/share` 또는 `192.168.1.20:/294t/dcloud/share`
LAB 정상: FQDN을 해석한 실제 transport 주소 `100.100.100.100`

IP source를 쓰면 `nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL` service principal 또는 `nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL` 과 이름이 맞지 않는다. `192.168.2.30` 과 `192.168.1.20` 은 관리 SSH 주소이며 운영 NFS data path가 아니다.

점검 명령:

```
MOUNT_TARGET=/home/tako9/share
STORAGE_FQDN=lab-storage.lab.decs.internal

findmnt -T "$MOUNT_TARGET" -o TARGET,SOURCE,FSTYPE,OPTIONS
nfsstat -m | sed -n "\\|${MOUNT_TARGET}|,+4p"
getent hosts "$STORAGE_FQDN"
```

FARM host에서는 값을 `/home/tako<번호>/share` 와 `nas.farm.decs.internal` 로 바꾼다. `findmnt` 의 `vers=4.0` 은 FARM, `vers=4.1` 은 LAB이어야 하고 두 환경 모두 `sec=krb5` 여야 한다.

`fstab` 설정은 [server-state playbook](#)에서 관리한다. 이미 mount된 legacy superblock은 유지보수 창에 `clean unmount/remount`한다. NFS caller가 D-state이면 강제 unmount를 반복하지 말고 포렌식 보존과 host reboot 판단으로 전환한다.

6. Machine credential과 NFS service ticket 확인

계산 호스트의 root mount에는 host machine keytab과 `rpc.gssd` 가 필요하다. realm과 NFS principal을 host 이름에 맞춰 선택한다.


```

short=$(hostname -s | tr '[:lower:]' '[:upper:]')

case "$short" in
    FARM*) KRB_REALM=FARM.DECS.INTERNAL
           KRB_NFS_PRINCIPAL=nfs/nas.farm.decs.internal@FARM.DECS.INTERNAL ;;
    LAB*) KRB_REALM=LAB.DECS.INTERNAL
          KRB_NFS_PRINCIPAL=nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL ;;
    *)    printf '지원하지 않는 host: %s\n' "$short" >&2; exit 1 ;;
esac
KRB_MACHINE_PRINCIPAL="${short}\${KRB_REALM}"

sudo klist -kte /etc/krb5.keytab
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kinit -k -t /etc/krb5.keytab "$KRB_MACHINE_PRINCIPAL"
sudo KRB5CCNAME=FILE:/run/decs-host-check.ccache \
    kvno "$KRB_NFS_PRINCIPAL"
sudo rm -f /run/decs-host-check.ccache

systemctl status rpc-gssd --no-pager
systemctl cat rpc-gssd
pgrep -a rpc.gssd

```

`rpc.gssd` 를 `-n` 으로 시작하지 않는다. root mount는 host machine credential을 사용해야 하며, `-n` 은 UID 0 사용자 credential을 찾게 하여 ticket이 없을 때 mount를 잃게 만들 수 있다.

7. Synology NAS와 Linux storage 운영 차이

FARM의 Synology NAS와 LAB의 Linux storage는 같은 NFS/Kerberos 개념을 사용하지만 설정을 관리하는 방식, service 이름과 keytab lifecycle이 다르다. client에서 확인하는 `findmnt`, `klist`, `kvno` 절차는 비슷해도 server 변경 명령은 서로 바뀌 실행하면 안 된다.

7.1 운영 경계 비교

항목	FARM Synology NAS	LAB Linux storage
관리 접속	<code>jy@192.168.2.30:6954</code>	<code>jy@192.168.1.20:6953</code>
AD join 도구	DSM UI 또는 <code>/usr/syno/sbin/synowin</code>	표준 Samba <code>net ads join</code>
Samba/winbind	Synology SMB package 경로와 <code>pkg-synosamba-*</code> unit	<code>/etc/samba/smb.conf</code> , 표준 <code>winbind.service</code>

항목	FARM Synology NAS	LAB Linux storage
RFC2307 NSS	<code>files winbind syno</code> 순서와 Synology idmap 설정을 함께 확인	<code>files systemd winbind</code> 와 Samba <code>idmap config</code> LAB : <code>backend = ad</code> 확인
NFS export root	<code>/volume1/share</code>	<code>/294t/dcloud/share</code> ; PoC는 별도 <code>test_krb</code> export
production export	storage IP별 <code>root_squash</code> , <code>sec=krb5:krb5i:krb5p</code> ; Synology <code>anonuid/anongid</code> , <code>insecure</code> option 포함	Linux <code>/etc/exports</code> 에서 <code>root_squash</code> , <code>sec=krb5</code> , <code>no_subtree_check</code> 를 명시
NFS service	Synology <code>/usr/sbin/svcsd</code> 와 <code>/usr/sbin/idmapd</code>	<code>nfs-server</code> 와 <code>rpc-svcgssd</code> 또는 배포판의 socket-activated GSS service
NFS keytab	<code>/etc/nfs/krb5.keytab</code> 이 ticket 검증 기준; <code>/etc/krb5.keytab</code> 도 동기화 검사	<code>/etc/krb5.keytab</code> 한 경로
NFS SPN 등록 계정	전용 user <code>svc-nfs-farm</code> ; <code>NAS\$</code> 와 분리	computer account <code>LAB-STORAGE\$</code>
추가 acceptor	같은 keytab의 AILAB <code>nfs/nas.ai lab.dgu@AILAB.DGU</code> 를 반드시 보존	현재 없음
운영 NFS version	v4.0	v4.1
keytab checker	<code>--profile farm</code> , 5분 user timer 운영	<code>--profile lab</code> ; profile은 준비됐지만 PoC 단계에서는 timer를 기본 활성화하지 않음

Synology에서 `getent` , `id` 또는 `wbinfo` 가 DECS UID/GID 대신 `96470xxx` 같은 내부 ID를 반환하면 RFC2307 경로가 정상인 상태가 아니다. 이때 symbolic `chown -R 'FARM\user':'FARM\group'` 을 실행하면 잘못 해석된 숫자가 filesystem에 저장된다. 먼저 DB-AD와 NAS의 numeric UID/GID가 일치하는지 확인하고, ownership 복구도 검증된 숫자로 수행한다.

FARM NAS에는 관리·호환성 목적으로 남은 `192.168.2.0/24 sec=sys,no_root_squash` export가 있을 수 있다. 이것은 production Kerberos data path가 아니며 FARM host나 container가 이 `sec=sys` export를 사용하도록 변경하면 안 된다.

7.2 읽기 전용 점검

FARM Synology NAS에서는 vendor 경로와 service 이름을 사용한다.

```
ssh -p 6954 jy@192.168.2.30

/usr/syno/sbin/synowin -getWorkgroup
SMB=/usr/local/packages/@appstore/SMBService/usr/bin
sudo "$SMB/net" ads testjoin
sudo "$SMB/wbinfo" --online-status

sudo klist -kte /etc/nfs/krb5.keytab
sudo klist -kte /etc/krb5.keytab
sudo exportfs -v | sed -n '/\/volume1\/share/,+8p'
ps -ef | grep -E 'svcgssd|idmapd' | grep -v grep
```

LAB Linux storage에서는 표준 Samba/NFS systemd unit과 keytab을 확인한다.

```
ssh -p 6953 jy@192.168.1.20

sudo net ads testjoin
wbinfo --online-status
getent passwd '<username>'

sudo klist -kte /etc/krb5.keytab \
  | grep -F 'nfs/lab-storage.lab.decs.internal@LAB.DECS.INTERNAL'
systemctl is-active winbind nfs-server
systemctl --no-pager --full status rpc-svcgssd.service 2>/dev/null || true
sudo exportfs -v | sed -n '/\/294t\/dcloud\/share/,+8p'
```

LAB 배포판에서는 `rpc-svcgssd` 가 독립 unit이 아니라 socket 또는 `nfs-server` 의 일부로 관리될 수 있다. unit 이름 하나만으로 실패를 판정하지 말고 process, service keytab과 실제 `kvno` /canary 결과를 함께 본다.

7.3 변경 작업에서 지킬 차이

- Synology 설정을 바꾸기 전에는 DSM/Samba/NFS 설정과 두 keytab을 먼저 backup한다. DSM 또는 SMB package가 관리하는 파일·unit을 일반 Ubuntu와 같다고 가정하지 않는다.
- FARM keytab repair는 AILAB principal과 이전 FARM KVNO를 보존하는 전용 `repair-farm-nas-nfs-keytab.sh` 을 사용한다. `svcgssd` 를 FARM principal 하나로 제한하는 `-p` 나 nameless acceptor `-n` 으로 시작하지 않는다.
- LAB storage는 표준 Samba join, `/etc/krb5.keytab` , `nfs-server` / `rpc-svcgssd` 와 Ansible desired state를 기준으로 한다. FARM repair/rotation script를 LAB에 실행하지 않는다.
- 두 환경 모두 identity/export 변경 후 `exportfs -ra` 와 필요한 NFS/RPC cache flush를 수행할 수 있지만, 먼저 활성 client와 D-state를 확인하고 유지보수 시간에만 실행한다.
- Synology 관리 IP와 LAB storage 관리 IP는 NFS source가 아니다. 변경 후 client `findmnt` 에서 각각 `nas.farm.decs.internal` 과 `lab-storage.lab.decs.internal` 이 유지되는지 확인한다.

환경별 keytab 상태는 같은 checker를 profile만 바꿔 읽기 전용으로 확인한다.

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile lab
```

server-side 전체 설정 절차는 [FARM canonical runbook](#)과 [LAB canonical runbook](#)을 각각 따른다.

8. FARM NAS service account와 KVNO 운영

현재 정책

- NFS SPN 등록 계정: `svc-nfs-farm`
- `NAS$`: domain membership용 `HOST/*`, `RestrictedKrbHost/*` 만 유지
- 자동 password expiration/rotation: 없음
- keytab drift 관측: 5분마다 읽기 전용
- repair/rotation: 관리자 명시 실행만 허용

과거 `NAS$` machine password의 주기적 변경으로 KVNO와 NAS NFS keytab이 어긋났기 때문에 전용 service account를 만들었다. 운영 문서에는 “KVNO가 주기적으로 바뀌어서 매번 새 계정을 만든다”가 아니라, **한 번 만든 전용 계정으로 자동 machine-account rotation에서 NFS SPN을 분리했다고** 기록한다.

읽기 전용 점검

```
cd /home/jy/server_manage/monitoring
health-checks/kerberos-nfs-keytab/script/check-nfs-keytab.sh --profile farm

systemctl --user status check-nfs-keytab@farm.timer --no-pager
systemctl --user list-timers 'check-nfs-keytab@farm.timer'
```

checker가 확인하는 항목:

- AD `svc-nfs-farm`의 현재 `msDS-KeyVersionNumber`
- NAS `/etc/krb5.keytab`, `/etc/nfs/krb5.keytab`에 현재 KVNO와 NFS principal 존재
- 공유 keytab에 AILAB acceptor가 보존되어 있는지
- 실제 `kvno -k` service-ticket 복호화 성공 여부
- `svcgssd` 실행 여부와 잘못된 `-p` 제한 여부

코드는 공용 checker, profile 값은 [FARM config](#)에서 확인한다.

Drift 수동 복구

```
cd /home/jy/server_manage/kerberos-nfs

./script/keytab/repair-farm-nas-nfs-keytab.sh --check
./script/keytab/repair-farm-nas-nfs-keytab.sh --repair
```

`--repair` 는 새 key를 export·검증하고 기존 AILAB principal과 이전 FARM KVNO를 보존하면서 NAS keytab을 원자 교체한다. 필요하다면 `svcgssd` 를 `-p` 없이 재시작하므로 활성 client 영향과 되돌릴 backup을 먼저 확인한다. 구현은 [repair script](#)에 있다.

계획된 rotation

```
./script/keytab/rotate-farm-nfs-service-key.sh --check

# 유지보수 창, AD replication, NAS/client 상태를 확인한 뒤에만 실행
./script/keytab/rotate-farm-nfs-service-key.sh --rotate --apply
```

rotation은 random password 설정, AD KVNO 변경, 새 key export, NAS keytab merge, `svcgssd` 반영, replica sync를 한 작업으로 수행한다. 이전 KVNO는 설정된 24시간 ticket lifetime보다 긴 최소 48시간 보존한다. 구현은 [rotation script](#)에 있다.

9. 모니터링에서 확인할 항목

무엇을 확인하는가	대표 상태/지표	코드
AD KVNO와 NAS keytab 일치	profile status JSON, checker exit 0/1/2	keytab health check
host keytab→kinit→kvno→rpc-gssd	<code>cluster_monitor_nfs_gss_ready</code> 와 stage	readiness script
실제 service path	<code>cluster_monitor_nfs_gss_canary_success</code>	canary probe
운영 mount	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , probe seconds	cluster collector
missing mount 복구	<code>cluster_monitor_nfs_gss_recovery_*</code>	guarded recovery
D-state/lease/hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code>	NFS forensics collector
경보 조건	<code>ClusterMonitorNFSGSS*</code> , <code>ClusterMonitorNFS*</code> , mount alerts	FARM Prometheus values

상세한 Prometheus/Grafana 운영은 [monitoring 운영 문서](#)를 참고한다.

10. 장애 진단 순서

사용자 한 명만 실패

1. DB UID/GID와 AD `uidNumber/gidNumber` 를 비교한다.
2. user keytab permission과 principal을 확인한다.
3. refresh service journal과 ccache `klist` 를 확인한다.
4. 동일 UID로 host NFS write를 시험한다.
5. container UID, `KRB5CCNAME` , ccache bind mount를 확인한다.
6. AD group 변경 직후라면 fresh ticket을 발급한다.

여러 host/user가 동시에 실패

1. `getent hosts <storage-fqdn>` 과 storage RPC 연결을 확인한다.
2. keytab checker에서 AD KVNO/NAS keytab drift를 확인한다.
3. `kvno nfs/<storage-fqdn>@<REALM>` 을 확인한다.
4. NAS `svcgssd` 와 service keytab을 확인한다.
5. mount source가 IP나 관리망 주소로 바뀌지 않았는지 확인한다.
6. AD replication 실패 시 실제 쓰기 대상 DC와 NAS가 조회하는 DC를 확인한다.

Mount가 없거나 응답하지 않음

1. `findmnt` 로 존재/source/security를 확인한다.
2. readiness 상태에서 처음 실패한 stage를 확인한다.
3. D-state, lease expiry, hung-task와 forensics snapshot을 확인한다.
4. mount가 **없는 경우에만** guarded recovery timer 상태를 확인한다.
5. 이미 healthy한 mount를 자동/수동으로 재마운트하지 않는다.
6. D-state caller가 남아 있으면 forced unmount 반복 대신 증거 보존 후 reboot를 검토한다.

11. 변경 전후 체크리스트

변경 전

- 대상 realm, DC, storage FQDN과 NFS principal 확인
- 활성 client와 유지보수 창 확인
- AD replication health 확인
- NAS keytab backup과 AILAB principal 확인
- 현재 `findmnt` , `klist` , checker status 보존

변경 후

- AD current KVNO와 NAS keytab KVNO 일치
- `kvno -k` 복호화 성공
- `svcgssd` 가 `-p` 없이 실행
- host readiness와 실제 사용자 NFS write 성공
- refresh timer enabled/active, ccache 유효
- Prometheus target/rules와 Grafana dashboard 정상
- 이전 KVNO를 최소 48시간 보존

12. 문서에 추가로 유지할 내용

설계와 운영이 다시 섞이지 않도록 다음 내용을 계속 분리해 기록한다.

- 설계 문서: trust boundary, principal naming, UID/GID invariant, ticket lifetime, FQDN 선택 근거, failure domain, Docker와 향후 Pod의 credential 모델
- 운영 문서: 현재 endpoint/KVNO가 아니라 확인 명령, timer/SLO, rotation 승인 조건, rollback, incident 진단 순서, 마지막 검증 날짜
- canonical runbook: 실제 host별 mount, 현재 적용 상태, 예외와 잔여 위험
- 실험 결과: 당시 kernel/NFS/security flavor와 재현 조건; 현재 default와 분리

keytab, password, local environment, incident 개인정보와 packet capture는 위키와 Git에 넣지 않는다.

Kerberos/NFS 디버깅 로그

이 문서는 Kerberos/NFS에서 발생한 장애와 재현 실험을 축적하는 인덱스다. 정상 상태를 만드는 절차는 [운영 문서](#)에 두고, 여기에는 장애가 어떻게 관측됐고 어떤 가설을 어떤 조건에서 검증했는지 기록한다.

기록 목록

검증일	상태	문제	핵심 결론
2026-07-20	관측, 인과 미확정	LAB9에서 <code>sec=krb5</code> 를 사용하는 이유	<code>krb5p</code> 에서 <code>krb5</code> 로 전환한 뒤 D-state 지속 시간, SUNRPC 적체와 mount probe 지연이 크게 감소함
2026-07-16	재현 완료, 과거 장애 원인 후보	NFSv4.1 session slot 고착	구형 storage kernel에서 지연된 idmap decode가 NFSv4.1 slot 하나를 영구 <code>INUSE</code> 상태로 남기는 경로를 재현함

상태는 다음 의미로 사용한다.

- **관측**: 증상과 증거는 있지만 원인 경로를 재현하지 못했다.
- **재현 완료**: 통제된 조건에서 같은 kernel 또는 protocol 경로를 다시 만들었다.

- **원인 후보:** 재현된 경로가 과거 장애를 설명하지만 장애 발생 시점의 직접 trace가 없어 동일 원인이었다고 확정할 수 없다.
- **원인 확정:** 장애가 시작된 시점의 증거와 재현 결과가 같은 경로를 가리킨다.

장애 발생 시 먼저 할 일

NFS **hard** mount 관련 process가 D-state에 들어간 경우에는 새로운 **find**, **du**, **stat**, canary I/O를 반복하지 않는다. 각 명령이 새로운 NFS request와 D-state process를 추가할 수 있다.

먼저 NFS 경로를 읽지 않는 local 상태만 수집한다.

```
date -Ins
uname -r
findmnt -rn -o TARGET,SOURCE,FSTYPE,OPTIONS

ps -e -o state=,pid=,ppid=,etimes=,comm=,wchan:48=,args= \
| awk '$1 ~ /^D/ {print}'

cat /proc/net/rpc/nfs 2>/dev/null || true
cat /run/decs-nfs-forensics/status.json 2>/dev/null || true
cat /var/lib/decs-nfs-gss/recovery.state 2>/dev/null || true
cat /var/lib/decs-nfs-gss/canary.state 2>/dev/null || true

systemctl --no-pager --full status rpc-gssd.service
journalctl -u rpc-gssd.service -b --no-pager -n 200
```

forensics가 배포된 host에서는 자동 snapshot이 생성됐는지 먼저 확인한다. 자동 trigger가 없었다면 local 상태만 읽는 수동 snapshot을 한 번 실행할 수 있다.

```
sudo systemctl start decs-nfs-forensics-snapshot.service
sudo cat /var/lib/decs-nfs-forensics/last_snapshot.state
```

이미 mount된 경로에 D-state caller가 있으면 **rpc-gssd** restart, forced unmount, 반복 remount를 먼저 실행하지 않는다. 증거를 보존한 뒤 storage-network 상태와 kernel wait path를 기준으로 복구 범위를 결정한다.

새 디버깅 문서 작성 형식

새 문제는 한 파일에 다음 내용을 남긴다.

1. **상태와 영향 범위:** 발생 시각, host, kernel, mount source/target/security
2. **증상:** 사용자 증상, process state, wait channel, alert
3. **확인한 사실과 가설:** 관측 사실과 아직 증명하지 못한 추론을 분리

4. **실험 목적**: 어떤 가설을 반증하거나 확인하려 했는지
5. **재현 조건**: server/client, version, cache, 동시성, fault와 안전장치
6. **실행 명령**: 사전 확인, fault 주입, workload, 관측, cleanup 순서
7. **결과와 판정 기준**: 성공/실패를 판단한 trace와 metric
8. **복구와 예방**: 임시 containment, 영구 수정, monitoring 변경
9. **원본 증거**: 결과 README, curated evidence, script와 checksum 링크

운영 환경에서 fault를 주입한 명령은 일반 runbook처럼 제시하지 않고 반드시 실행 일자와 승인·안전 조건을 붙인 **과거 실행 기록**으로 표시한다. 반복 실험은 가능하면 격리 VM harness로 옮긴다.

LAB9에서 **sec=krb5** 를 사용하는 이유

상태: LAB9에서 **sec=krb5p** 전환 전 24시간과 **sec=krb5** 전환 후 약 51시간을 비교한 운영 관측이다. 전환 뒤 NFS 지연과 적체가 크게 감소했지만, 관측 기간이 다르고 security flavor만 통제된 실험이 아니므로 **krb5p** 를 단일 원인으로 확정하지는 않는다.

1. 결론

LAB9의 Kerberos NFS 기본값은 **sec=krb5** 로 유지한다. 이 선택은 Kerberos principal 인증을 유지하면서 RPC payload 암호화 비용을 제외하고, LAB9에서 실제로 관측된 D-state 지속·SUNRPC 적체·mount probe 지연을 낮추기 위한 운영 기준이다.

krb5p 가 잘못된 방식이라는 뜻은 아니다. NFS payload 기밀성이 반드시 필요한 경로에서는 **krb5p** 를 사용해야 한다. 다만 현재 LAB9 사용자 공유의 기본 profile은 가용성과 지연 안정성을 우선해 **krb5** 를 사용한다. **krb5i** 또는 **krb5p** 는 요구사항이 있고 부하 검증을 통과한 경로에서만 명시적으로 선택한다.

2. security flavor의 차이

flavor	제공하는 보호	LAB9에서의 위치
sec=krb5	Kerberos principal 인증	기본값
sec=krb5i	인증 + RPC payload 무결성	변조 방지가 명시적으로 필요한 경로에서 검토
sec=krb5p	인증 + 무결성 + RPC payload 암호화	기밀성이 필요한 경로에서 성능 검증 후 사용

sec=krb5에서는 사용자와 NFS service가 Kerberos로 서로 인증되지만, NFS payload 자체는 암호화되지 않는다. 따라서 패킷 캡처와 forensics evidence는 root만 읽을 수 있게 보관하고, 보호되지 않은 네트워크에서 기밀 데이터를 전송해야 한다면 이번 성능 결과만으로 **krb5** 를 선택해서는 안 된다.

3. 왜 전환했나

`sec=krb5p` 사용 중 LAB9에서는 NFS 관련 D-state가 관측 기간 내내 존재했고, SUNRPC task와 transport reply 대기가 누적됐다. mount responsiveness probe도 설정된 10초 timeout 상한에 도달했다. 이 상태에서는 home 경로를 읽는 SSH나 사용자 process가 NFS 대기에 연쇄적으로 묶일 수 있다.

`krb5p` 는 각 RPC payload에 무결성 처리와 암호화를 추가한다. 이 비용이 장애의 단일 원인이라고 입증한 것은 아니지만, storage·kernel·network에서 이미 발생한 지연을 더 크게 보이게 하는 증폭 요인인지 확인할 운영상 이유가 있었다. 그래서 principal 인증은 유지하면서 privacy 처리를 제외하는 `sec=krb5` 로 전환하고 동일한 NFS health 지표를 계속 관측했다.

4. 전환 전후 결과

측정 대상은 LAB9다.

지표	<code>krb5p</code> 전환 전 24시간	<code>krb5</code> 전환 후 약 51시간
NFS D-state 존재 비율	100%	약 4.1%
D-state 프로세스 최대	28개	8개
최장 연속 D-state	83,339초(약 23.1시간)	1,236초(약 20분 36초)
<code>xprt_pending</code> 평균	5.35	1.29
RPC task 최대	3,696	525
마운트 probe 최대	10초	0.429초
lease 만료	없음	없음

변화 폭은 다음과 같다.

비교 항목	변화
NFS D-state 존재 비율	95.9%p 감소
D-state 프로세스 최대	약 71.4% 감소
최장 연속 D-state	약 98.5% 감소
<code>xprt_pending</code> 평균	약 75.9% 감소
RPC task 최대	약 85.8% 감소
마운트 probe 최대	약 95.7% 감소

가장 중요한 변화는 D-state가 “항상 존재하는 상태”에서 짧고 간헐적인 상태로 바뀐 점이다. `xprt_pending` 평균과 RPC task 최대도 함께 감소했으므로 단순히 D-state process 이름만 달라진 것이 아니라 NFS transport 적체 자체가 줄어든 방향과 일치한다.

전환 전 mount probe의 10초는 `cluster-monitor-exporter` 의 command timeout 상한과 같다. 따라서 이 값은 정상 응답에 10초가 걸렸다가보다 probe가 timeout 또는 기존 in-flight probe 상태에 도달했을 가능성이 크다. 전환 후 최대 0.429초는 관측 구간에서 10초 상한에 도달하지 않았음을 보여준다.

두 구간 모두 lease 만료가 없었다. 따라서 이번 비교에서 확인한 개선은 lease 만료 복구 여부보다 D-state 지속, RPC queue와 mount 응답성에 관한 것이다.

5. 지표의 의미

지표	판정 방법	구현
NFS D-state 존재 비율	유효 sample 중 NFS/RPC 관련 D-state process가 하나 이상인 sample 비율	<code>cluster_monitor_nfs_forensics_d_state_processes</code>
D-state 프로세스 최대	관측 구간의 NFS/RPC 관련 D-state process 최대값	같은 metric의 <code>max_over_time</code>
최장 연속 D-state	가장 오래 연속 관측된 NFS/RPC 관련 D-state process의 시간	<code>cluster_monitor_nfs_forensics_d_state_oldest_seconds</code>
<code>xprt_pending</code> 평균	transport reply를 기다리는 SUNRPC task 수의 구간 평균	<code>cluster_monitor_nfs_xprt_pending</code>
RPC task 최대	NFS RPC client task 수의 구간 최대	<code>cluster_monitor_nfs_rpc_tasks</code>
마운트 probe 최대	required mount에 수행한 bounded <code>statfs</code> 시간의 구간 최대	<code>cluster_monitor_host_mount_probe_seconds</code>
lease 만료	local NFSv4 lease deadline을 넘긴 시간	<code>cluster_monitor_nfs_lease_expired_seconds</code>

수집 경로는 다음과 같다.

```
LAB9 kernel:/proc 상태
-> NFS forensics watcher status.json
-> cluster-monitor-exporter metrics
-> FARM Prometheus
-> 같은 server="LAB9" label의 전환 전후 구간 집계
```

구현은 `forensics watcher`, `forensics metric adapter`, `mount/D-state collector`에서 확인한다.

6. 이 결과로 말할 수 있는 것과 없는 것

말할 수 있는 범위는 다음과 같다.

- LAB9에서 `krb5` 전환 뒤 모든 주요 NFS 적체 지표가 같은 방향으로 개선됐다.
- 약 51시간의 전환 후 구간에는 24시간 전환 전 구간과 같은 장시간 고착이 관측되지 않았다.

- Kerberos 인증을 유지한 `krb5` 가 LAB9의 현재 운영 baseline으로 더 안정적이었다.

아직 확정할 수 없는 범위는 다음과 같다.

- `krb5p` 의 암호화 비용만으로 전환 전 D-state가 발생했다는 인과관계
- 같은 시기에 달라졌을 수 있는 workload, storage queue, network, kernel 상태의 영향
- FARM/LAB의 다른 host에서도 감소 폭이 동일할 것이라는 일반화
- 51시간보다 긴 기간이나 peak workload에서도 같은 결과가 유지된다는 보장

따라서 이 기록의 판정은 **LAB9 운영 선택을 지지하는 강한 전후 관측**이며, 통제된 benchmark나 원인 확정 실험은 아니다. 원인을 더 좁히려면 동일 workload와 동일 시간대에 `krb5` / `krb5p` 를 반복 교차하는 격리 실험이 필요하다.

7. 운영 기준과 확인 방법

LAB9의 desired mount option은 `sec=krb5` 다. mount source는 IP가 아니라 NFS service principal과 일치하는 FQDN을 사용한다. D-state가 이미 존재할 때는 확인을 위해 mount를 반복해서 읽거나 즉시 remount하지 말고 먼저 local forensics 상태를 보존한다.

현재 mount의 security flavor는 다음처럼 정확히 확인한다.

```
LAB9_MOUNT_TARGET=/home/tako9/share

findmnt -T "$LAB9_MOUNT_TARGET" -n -o TARGET,SOURCE,FSTYPE,OPTIONS
findmnt -T "$LAB9_MOUNT_TARGET" -n -o OPTIONS \
| tr ' ' '\n' \
| grep -Fx 'sec=krb5'
```

`grep -F 'sec=krb5'` 만 사용하면 `sec=krb5p` 도 일치하므로 option을 쉼표 단위로 분리해 exact match한다. 전환 이후에도 최소한 다음 조건을 함께 확인한다.

- NFS D-state 존재 비율과 최장 연속 시간이 다시 증가하지 않는가
- `xprt_pending` 평균과 RPC task 최대가 전환 전 수준으로 돌아가지 않는가
- mount probe가 10초 timeout 상한에 다시 도달하지 않는가
- `cluster_monitor_nfs_lease_expired_seconds` 가 계속 0인가
- payload 기밀성 요구가 새로 생겨 `krb5p` 가 필요한 경로가 되지 않았는가

기본 security policy는 Kerberos/NFS 운영 저장소, 배포 값은 `monitoring exporter` 변수를 기준으로 한다. 상세 장애 채증과 D-state 대응은 [디버깅 로그 개요](#)의 절차를 따른다.

8. 다음 측정에서 함께 남길 값

현재 비교표에는 구간 길이만 있고 정확한 시작·종료 timestamp와 원본 query/export artifact가 없다. 다음 재측정부터는 아래 값을 함께 보관해야 같은 결과를 재계산할 수 있다.

- 전환 시각과 두 구간의 시작·종료 시각(타임존 포함)
- LAB9의 source FQDN, target, NFS version과 전체 mount option
- client/server kernel, storage 상태와 구간별 workload
- 사용한 PromQL 또는 `samples.tsv` 집계 script와 원본 결과
- exporter command timeout과 scrape interval
- 전환과 동시에 수행한 다른 설정 변경

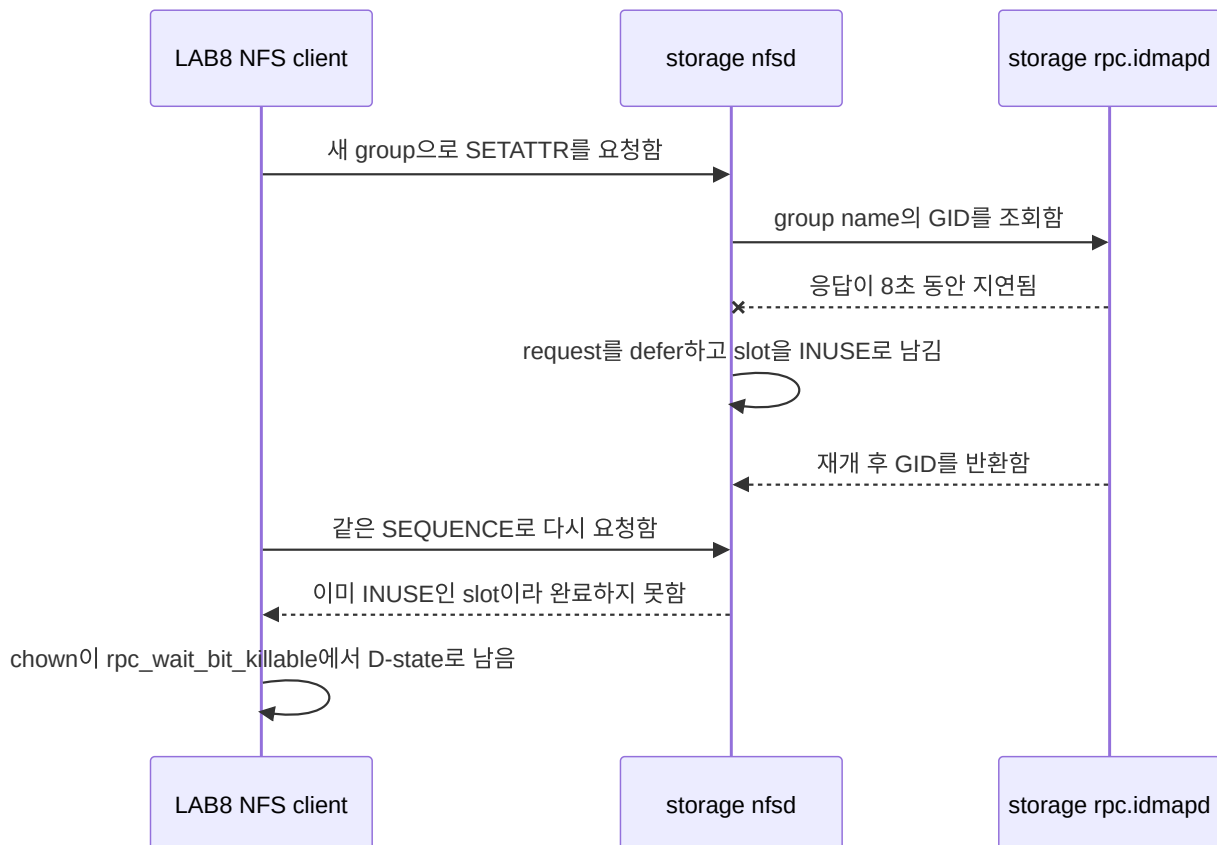
원본 artifact를 저장소에 추가하면 이 문서의 표 아래에 commit 고정 링크를 추가한다.

NFSv4.1 session slot 고착

상태: 2026-07-16에 slot leak 경로 재현 완료. 2026-06-29 LAB5 장애의 강한 원인 후보이지만, LAB5 장애 시작 시점의 storage trace가 없어 과거 장애의 최초 원인으로 확정하지는 않는다.

1. 요약

LAB storage의 구형 kernel `3.10.0-862.el7.x86_64` 에서 NFSv4 `SETATTR` 가 새로운 group 이름을 UID/GID로 해석하던 중 `rpc.idmapd` 응답이 지연되면 request가 defer될 수 있다. 이때 NFSv4.1 `SEQUENCE` 는 slot을 `INUSE` 로 표시하지만 response encode와 `nfsd4_sequence_done()` 이 실행되지 않아 slot이 반환되지 않는 경로를 실제로 재현했다.



한 slot만 leak한 최소 재현에서는 `ForeChannel` waiter가 0일 수도 있다. 따라서 waiter가 없다는 사실만으로 slot 문제를 배제하면 안 된다. session reset이나 recovery가 모든 slot의 drain을 기다리는 상황에서는 고착된 한 slot이 전체 복구를 막을 수 있다.

2. 어떤 증상이었나

2026-07-16 재현에서는 LAB8의 `chown` worker가 173초 이상 D-state에 남았다.

```
chown
-> nfs4_proc_setattr
-> nfs4_call_sync_sequence
-> rpc_wait_bit_killable
```

storage trace에서는 첫 처리와 재방문이 다음처럼 달랐다.

단계	관측
첫 idmap decode	<code>RQ_USEDEFERRAL</code> , <code>RQ_DROPME</code> 가 설정됨
첫 SEQUENCE 확인	<code>seqid=0x5f</code> , <code>slot_seqid=0x5e</code> , <code>INUSE=0</code>
첫 request 종료	dispatcher가 0을 반환했고 encode와 <code>sequence_done</code> 이 실행되지 않음
<code>rpc.idmapd</code> 재개 후	같은 request가 <code>seqid=0x5f</code> , <code>slot_seqid=0x5f</code> , <code>INUSE=1</code> 로 재방문
이후 retry	약 15초마다 같은 <code>INUSE=1</code> 을 확인하고 완료하지 못함

3. 왜 이 실험을 했나

조사는 다음 순서로 가설을 좁혔다.

- 2026-06-29 LAB5에서는 NFS user home을 읽는 `sshd: ... [priv]` 가 D-state에 누적됐다.
- 2026-07-10 LAB8에서 TCP/2049 transport를 일시 차단하자 같은 `rpc_wait_bit_killable` 계열 증상을 재현했지만, network가 복구된 뒤 process도 자연 복구됐다.
- keytab, `rpc-gssd`, readiness와 mount 순서를 바로잡은 clean boot에서도 구형 storage kernel의 session state가 영구 고착될 수 있는지는 별도 검증이 필요했다.
- upstream의 idmap deferral 수정 내용이 “NFSv4 decode 중 request drop으로 `NFSD4_SLOT_INUSE` 가 해제되지 않는다”는 경로를 설명하므로, 실제 LAB kernel과 mount에서 그 primitive를 최소 workload로 확인했다.

실험 목적은 많은 client를 동시에 멈추게 하는 것이 아니라, **idmap miss 하나가 slot 하나를 반환 불가능하게 만들 수 있는지**를 확인하는 것이었다.

4. 실험 결과

4.1 실제 LAB 환경의 최소 재현

첫 pass에서 `RQ_DROPME` 가 설정된 뒤 slot이 `INUSE` 로 바뀌었지만 encode와 `nfsd4_sequence_done()` 이 호출되지 않았다. 8초 뒤 `rpc.idmapd` 가 재개된 후에도 같은 sequence는 이미 사용 중인 slot으로 판단됐다. client retry도 동일 상태를 반복했고 worker는 D-state에서 끝나지 않았다.

따라서 다음 좁은 결론은 확정할 수 있다.

- 당시 storage kernel에서는 idmap decode deferral이 NFSv4.1 session slot 하나를 leak할 수 있다.
- keytab 발급, `rpc-gssd` 시작 순서나 client mount 순서만 고쳐서는 이 server-side kernel 경로를 제거할 수 없다.
- 한 slot leak은 재현했지만 LAB5에서 나중에 관측한 전체 session recovery/drain stall까지 이 최소 실험 하나로 재현한 것은 아니다.
- LAB5 장애 시작 시점의 trace가 없으므로 어떤 실제 operation이 최초 idmap miss를 만들었는지는 알 수 없다.

4.2 A-G: 8초 idmap 지연 kernel matrix

최소 재현과 같은 `idmap-setattr`, NFSv4.1, `sec=krb5p` 조건에서 server와 client kernel 조합을 바꿔 각 5회 실행했다. client OS는 모두 Ubuntu 22.04.5 LTS이며, F/G만 client kernel을 `6.18.38` 로 올렸다.

Case	NFS server OS	Server kernel	Client kernel	결과
A	CentOS Linux 7.9.2009	<code>3.10.0-862.el7.x86_64</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5
B	CentOS Linux 7.9.2009	<code>3.10.0-1160.el7.x86_64</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5
C	CentOS Stream 10	<code>6.12.74</code>	<code>6.8.0-134-generic</code>	slot 고착 5/5
D	CentOS Stream 10	<code>6.12.75</code>	<code>6.8.0-134-generic</code>	자동 복구 5/5
E	CentOS Stream 10	<code>6.12.0-248.el10.x86_64</code>	<code>6.8.0-134-generic</code>	자동 복구 5/5
F	CentOS Linux 7.9.2009	<code>3.10.0-862.el7.x86_64</code>	<code>6.18.38</code>	slot 고착 5/5
G	CentOS Linux 7.9.2009	<code>3.10.0-1160.el7.x86_64</code>	<code>6.18.38</code>	slot 고착 5/5

구형 server kernel에서는 client kernel을 `6.18.38` 로 올린 F/G도 모두 고착됐다. 반대로 idmap/slot fix가 포함된 D와 signed Stream kernel E는 모두 자동 복구했다. 이 결과는 문제와 복구 여부를 가르는 주된 변수가 client가 아니라 **NFS server kernel의 idmap deferral 처리**임을 지지한다.

4.3 D/E: 120·300·600초 장시간 idmap 지연 추가 실험

8초 지연만으로는 수정된 kernel이 짧은 지연에서만 우연히 복구한 것인지 판단하기 어려웠다. 그래서 자동 복구한 D/E만 대상으로 idmap cache miss 응답을 120초, 300초, 600초까지 지연하고 각 조건을 3회씩 다시 실행했다.

Case	Server kernel	idmap 지연	원래 chgrp	신규 RPC	최종 snapshot D / FC / lease
D-120s	6.12.75-nfs-slot	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0*
D-300s	6.12.75-nfs-slot	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
D-600s	6.12.75-nfs-slot	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0
E-120s	6.12.0-248.el10.x86_64	120초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-300s	6.12.0-248.el10.x86_64	300초	성공 3/3	성공 3/3	모두 0 / 0 / 0
E-600s	6.12.0-248.el10.x86_64	600초	EINVAL 3/3	성공 3/3	모두 0 / 0 / 0

여기서 **FC** 는 NFSv4.1 ForeChannel waiter 수다. 600초 조건의 **EINVAL** 은 원래 **chgrp** 요청의 application 결과이며, slot 고착 판정과 분리해야 한다. 같은 run에서 idmap 지연을 해제한 뒤 실행한 신규 RPC는 모두 성공했고, 최종 snapshot의 D-state, ForeChannel waiter와 **lease_expired** 도 모두 0이었다. 따라서 600초에서 원래 작업이 실패했더라도 **session slot**은 반환됐고 후속 요청을 막지 않았다.

* D-120s 2회차의 2초 sampler는 종료 직전 D-state **1** 을 한 번 기록했다. 그러나 직후 final snapshot은 D-state **0** 이었고, 신규 RPC 성공, ForeChannel **0** , **lease_expired=0** , analyzer **verdict=pass** 였다. 지속된 NFS slot 고착으로 판정하지 않았다.

5. 실제 재현 조건

항목	2026-07-16 조건
client	LAB8, 100.100.100.108
server	lab-storage.lab.decs.internal , 100.100.100.100
mount	/home/tako8/share , NFSv4.1, sec=krb5p , hard
storage kernel	3.10.0-862.el7.x86_64
client 시작 상태	D-state 0, NFS RPC task 0, readiness ready
test identity	LAB8 local group decs_idmap_repro_424242 , GID 424242
test object	/home/tako8/share/.decs-idmap-slot-repro-20260716
fault	storage rpc.idmapd 에 8초간 SIGSTOP
workload	test file에 GID 424242 를 지정하는 chown(2) 한 번
안전장치	storage-local 8초·20초 SIGCONT timer를 먼저 예약
금지한 복구	재현 후 client reboot, unmount, signal, service restart를 하지 않음

boot 순서도 별도로 확인했다. LAB8은 `rpc-gssd` active → Kerberos/NFS readiness 완료 → mount 순서였고, 기존 `/etc/krb5.keytab`의 KVNO는 2였다. 따라서 이번 결과를 keytab 생성 누락이나 잘못된 mount 시작 순서로 설명할 수 없다.

6. 당시 실행한 명령

경고: 아래 명령은 2026-07-16의 승인된 실제 실행 기록이다. storage의 `rpc.idmapd`를 멈추므로 일반 점검이나 운영 재현 절차로 다시 실행하면 안 된다. 재실험은 8절의 격리 VM harness를 사용한다. 특히 `kprobe` 인자 register는 당시 x86_64 kernel symbol에만 맞춘 값이라 다른 kernel에서 그대로 사용하지 않는다.

6.1 client 사전 확인과 test 값 준비

LAB8에서 D-state와 RPC task가 없는지 확인한 뒤, storage에 없던 group 이름을 client가 encode하도록 local group과 test file을 준비했다.

```
ps -eo stat= | awk '$1 ~ /^D/ {n++} END {print "d_state=" (n+0)}'
grep -c '^RPC task' /proc/net/rpc/nfs 2>/dev/null || true
findmnt -T /home/tako8/share -n -o TARGET,SOURCE,FSTYPE,OPTIONS

sudo groupadd -g 424242 decs_idmap_repro_424242

sudo bash -c '
p=/home/tako8/share/.decs-idmap-slot-repro-20260716
rm -f "$p"
touch "$p"
chmod 0666 "$p"
stat -c "path=%n uid=%u gid=%g mode=%a ino=%i" "$p"
'
```

fault 전에 GID `424241`을 사용한 정상 `chown`을 실행해 같은 경로가 idmap 지연 없이는 encode와 `sequence_done`까지 완료되는 것을 먼저 확인했다.

```
sudo python3 -c 'import os; p="/home/tako8/share/.decs-idmap-slot-repro-20260716"; print("before",
os.stat(p).st_gid); os.chown(p, -1, 424241); print("after", os.stat(p).st_gid)'
```

6.2 storage trace 설정

당시 storage에서 idmap decode, slot 검사, dispatcher return, response encode와 slot 반환 여부를 한 trace instance에 기록했다.

```

T=/sys/kernel/debug/tracing
I="$T/instances/decs-idmap-slot-repro"
sudo mkdir -p "$I"

for event in uid gid check_slot dispatch dispatch_ret encode sequence sequence_done; do
    echo "-:decs_idmap/$event" | sudo tee -a "$T/kprobe_events" >/dev/null || true
done

echo 'p:decs_idmap/uid nfsd_map_name_to_uid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/gid nfsd_map_name_to_gid rqstp=%di name=+0(%si):string namelen=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/check_slot check_slot_seqid seqid=%di:u32 slot_seqid=%si:u32 slot_inuse=%dx:u32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/dispatch nfsd_dispatch rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'r:decs_idmap/dispatch_ret nfsd_dispatch ret=$retval:s32' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/encode nfs4svc_encode_compoundres rqstp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence nfsd4_sequence rqstp=%di cstate=%si seq=%dx' \
    | sudo tee -a "$T/kprobe_events" >/dev/null
echo 'p:decs_idmap/sequence_done nfsd4_sequence_done resp=%di' \
    | sudo tee -a "$T/kprobe_events" >/dev/null

echo 0 | sudo tee "$I/tracing_on" >/dev/null
echo 0 | sudo tee "$I/events/enable" >/dev/null
sudo sh -c ": > '$I/trace'"
echo 8192 | sudo tee "$I/buffer_size_kb" >/dev/null

for event in \
    decs_idmap/uid decs_idmap/gid decs_idmap/check_slot decs_idmap/dispatch \
    decs_idmap/dispatch_ret decs_idmap/encode decs_idmap/sequence \
    decs_idmap/sequence_done sunrpc/svc_process sunrpc/svc_defer \
    sunrpc/svc_revisit_deferred sunrpc/svc_drop sunrpc/svc_send
do
    echo 1 | sudo tee "$I/events/$event/enable" >/dev/null
done
echo 1 | sudo tee "$I/tracing_on" >/dev/null

```

6.3 fault와 단일 workload 실행

storage에서 반드시 자동 재개 timer 두 개가 active인지 확인한 뒤 `rpc.idmapd`를 멈췄다.

```
pid="$(pgrep -xo rpc.idmapd)"

sudo systemd-run --unit=decs-idmapd-auto-resume-8s \
  --on-active=8s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"
sudo systemd-run --unit=decs-idmapd-auto-resume-20s \
  --on-active=20s --timer-property=AccuracySec=1s \
  /bin/kill -CONT "$pid"

sudo systemctl is-active --quiet decs-idmapd-auto-resume-8s.timer
sudo systemctl is-active --quiet decs-idmapd-auto-resume-20s.timer
sudo kill -STOP "$pid"
```

그 직후 LAB8의 별도 systemd worker에서 `chown(2)` 한 번만 실행했다.

```
sudo systemd-run --unit=decs-idmap-slot-repro-worker --no-block \
  /usr/bin/python3 -c \
  "import os; os.chown('/home/tako8/share/.decs-idmap-slot-repro-20260716', -1, 424242)"
```

6.4 결과 확인과 trace 보존

새로운 NFS I/O를 만들지 않고 worker, local kernel state와 이미 동작 중이던 forensics state를 확인했다.

```
systemctl --no-pager --full status decs-idmap-slot-repro-worker.service
ps -eo pid=,ppid=,stat=,comm=,wchan:32=,args= \
  | awk '$3 ~ /^D/ {print}'
grep -E 'lease_expired|RPC task|xprt_pending|ForeChannel' \
  /proc/net/rpc/nfs 2>/dev/null || true

cat /run/decs-nfs-forensics/status.json
cat /var/lib/decs-nfs-gss/recovery.state
cat /var/lib/decs-nfs-gss/canary.state
```

storage trace는 다음 pattern을 기준으로 추렸다.

```
I=/sys/kernel/debug/tracing/instances/decs-idmap-slot-repro
sudo sh -c "echo 0 > '$I/tracing_on'"
sudo cp "$I/trace" /var/tmp/decs-idmap-slot-repro-20260716.trace

sudo grep -E \
  'decs_idmap_gid|svc_defer|svc_revisit_deferred|svc_drop:|check_slot|sequence_done|dispatch_ret' \
  /var/tmp/decs-idmap-slot-repro-20260716.trace
```

7. Monitoring과 운영 대응

재현 당시 guardrail은 의도대로 동작했다.

- forensics watcher가 `nfs_d_state` 이유로 incident snapshot을 보존했다.
- recovery는 `status=blocked`, `diagnosis=nfs_d_state_detected`, `retry_inhibited=1`로 바뀌었다.
- canary도 같은 진단으로 차단돼 추가 NFS request를 만들지 않았다.
- packet ring과 kernel trace는 계속 증거를 수집했다.

운영에서 같은 증상이 보이면 다음 순서를 따른다.

1. NFS path를 읽는 `find`, `du`, `stat`, canary를 추가 실행하지 않는다.
2. D-state stack, `/proc/net/rpc/nfs`, forensics status와 snapshot을 보존한다.
3. mount가 이미 있으면 자동 remount와 `rpc-gssd` 반복 restart를 차단한다.
4. storage kernel, `rpc.idmapd`, network와 NFS session 증거를 함께 확인한다.
5. 상태가 자연 복구되지 않으면 유지보수 창외 client reboot 또는 storage 조치를 수동 결정한다. reboot는 상태를 지울 뿐 원인 수정은 아니다.
6. 영구 조치는 idmap deferral fix가 포함된 storage kernel로 올리고 격리 회귀 실험을 통과시키는 것이다.

운영 mount를 `soft`로 바꾸는 방식은 timeout을 application error로 노출하고 data integrity 위험을 만들 수 있어 해결책으로 사용하지 않는다.

8. 격리 VM에서 다시 검증하는 방법

반복 실험은 LAB8 local disk의 `decs-nfs-slot-isolated` network에서 수행한다. 이 network는 forwarding이 없고 `100.100.100.0/24`, `192.168.1.0/24`와 운영 storage hostname을 거부한다. VM disk, trace와 result도 NFS share가 아니라 LAB8 local filesystem에 둔다.

kernel 비교 기준은 다음과 같다.

Case	Server kernel	Client kernel	목적
A	CentOS 7 <code>3.10.0-862</code>	Ubuntu <code>6.8.0-134</code>	운영 취약 baseline
B	CentOS 7 <code>3.10.0-1160</code>	Ubuntu <code>6.8.0-134</code>	CentOS 7 최종 kernel 비교

Case	Server kernel	Client kernel	목적
C	upstream 6.12.74	Ubuntu 6.8.0-134	fix 이전 positive control
D	upstream 6.12.75	Ubuntu 6.8.0-134	idmap/slot fix 비교 control
E	signed Stream 6.12.0-248.el10	Ubuntu 6.8.0-134	운영 후보
F	CentOS 7 3.10.0-862	kernel.org 6.18.38	새 client kernel 비교
G	CentOS 7 3.10.0-1160	kernel.org 6.18.38	새 client kernel 비교

준비와 격리는 VM lab README의 절차를 따른다.

```
cd /home/jy/server_manage/kerberos-nfs/test/lab-kerberos-poc/vm-slot-regression

sudo ./bin/provision_lab8_vms.sh "$PWD"
./bin/configure_vm_stack.sh
sudo ./bin/prefetch_kernel_matrix.sh
sudo ./bin/verify_kernel_sources.sh
sudo ./bin/build_upstream_kernels_vm.sh
sudo ./bin/prepare_centos7_domains.sh "$PWD"
sudo ./bin/isolate_test_vms.sh
```

단일 비교는 같은 idmap-setattr, sec=krb5p, 8초 조건을 case별로 실행한다.

```
./bin/switch_kernel_case.sh A
NFS_SLOT_EXPECT_VULNERABLE=1 \
./bin/run_single_case.sh A krb5p idmap-setattr 8 1

./bin/switch_kernel_case.sh D
NFS_SLOT_EXPECT_VULNERABLE=0 \
./bin/run_single_case.sh D krb5p idmap-setattr 8 1
```

먼저 실행 수만 확인하려면 plan-only mode를 쓴다.

```
MATRIX_PLAN_ONLY=1 \
MATRIX_CASES='A C D E' \
MATRIX_SECURITY='krb5p' \
MATRIX_SCENARIOS='idmap-setattr' \
MATRIX_DURATIONS='8' \
MATRIX_REPETITIONS=3 \
./bin/run_matrix.sh
```

취약 case가 고착되면 harness는 evidence를 archive한 뒤 **격리 VM만** reset한다. 그 reset은 자동 복구 성공으로 계산하지 않는다. 운영 LAB8 host와 운영 NFS mount는 이 harness의 복구 대상이 아니다.

9. 원본 증거와 코드

- 2026-07-16 실제 재현 결과
- curated trace evidence
- 2026-07-10 NFS transport fault 실험
- 격리 VM slot regression lab
- single-case harness
- result analyzer
- upstream stable patch 설명
- NFS forensics 구현

보존된 storage raw trace의 SHA-256은 `5fe7fbab2b8aa712ff71afe5156438b4e880edcc39cf16a00592e22d578bac59` 다.

Monitoring

원본: md/system/monitoring/index.md, md/system/monitoring/design.md, md/system/monitoring/operations.md

역할: FARM/LAB 서버, NFS, GPU와 사용자 container의 실행 상태를 반복 관측하고, Prometheus·Grafana·Alertmanager를 통해 시각화와 경보를 제공한다.

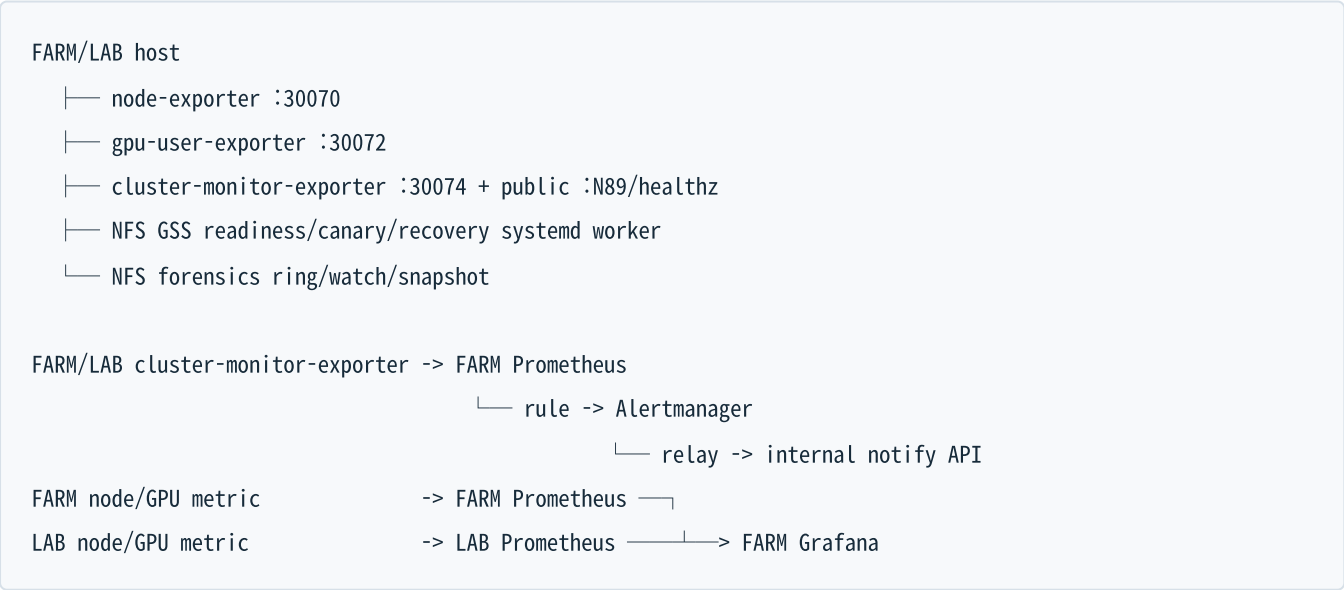
이 문서는 monitoring 문서의 시작점이다. **설계**는 수집 데이터가 어떤 구성 요소를 거쳐 metric과 alert가 되는지 설명하고, **운영**은 배포·점검·장애 대응 명령과 코드 위치를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위, 핵심 구성 요소와 운영 원칙
설계	exporter, Prometheus, Grafana, Alertmanager, GSS health와 forensics의 데이터 흐름
운영	배포, endpoint 점검, metric/alert 코드 링크, 장애별 진단 순서

처음 보는 사람은 **설계 2절의 전체 구성**에서 배치 영역과 데이터 흐름을 먼저 확인하고, 실제 점검이나 배포가 필요할 때 **운영 문서**로 이동한다. 장애 대응 중 설계 문서의 구성 요소 설명과 운영 문서의 명령을 번갈아 읽지 않도록, 반복해서 사용하는 endpoint·metric·진단 순서는 운영 문서에 모았다.

한눈에 보는 구조



핵심 구성 요소

구성 요소	endpoint/산출물	역할
node-exporter	:30070/metrics	CPU, memory, filesystem, disk와 network 같은 일반 host 자원 노출
gpu-user-exporter	:30072/metrics , :30072/-/healthy	GPU process를 DB의 실제 사용자와 container에 연결
cluster-monitor-exporter	:30074/metrics , :30074/healthz	mount, GSS, GPU, Docker, container, 연결성, D-state 수집
public health listener	서버별 N89/healthz	외부에서 NAT와 host 도달 가능성 확인
NFS GSS worker	/run · /var/lib state	readiness, canary와 guarded missing-mount recovery
NFS forensics	status JSON, local incident snapshot	제한된 packet/kernel trace를 보존
keytab health check	profile status JSON	AD KVNO, storage keytab과 GSS service drift를 읽기 전용 확인
FARM Prometheus/Alertmanager	FARM Kubernetes release	모든 cluster-monitor-exporter 를 중앙 수집하고 rule 평가와 alert routing 수행
LAB Prometheus	LAB Kubernetes release, NodePort 30073	LAB node/GPU metric을 별도 저장; Alertmanager와 Grafana는 비활성화
Grafana	FARM NodePort 30080	FARM/LAB Prometheus datasource를 한 UI에서 조회

운영 경계

- 지속 관측은 `monitoring` , 부팅 orchestration은 `remote-operations` , host desired state는 `server-state` 에서 관리한다.
- `cluster-monitor-exporter` 가 직접 mount하지 않는다. 별도 systemd recovery worker가 fstab, DNS/RPC, Kerberos readiness, canary와 D-state gate를 통과한 **missing mount**만 복구한다.
- keytab checker는 AD password, NAS keytab, service와 mount를 변경하지 않는다.
- forensics는 증거를 수집할 뿐 mount/restart/reboot를 수행하지 않는다.
- 자동 복구는 stopped container 시작, SSH 시작, 안전성이 확인된 NVML symlink repair처럼 영향 범위가 제한된 작업에만 사용한다.
- FARM과 LAB Prometheus는 장애와 저장소를 분리하고, Grafana dashboard만 공통 UI에서 datasource UID와 cluster label로 구분한다.

주요 코드

- `cluster-monitor-exporter`

- [gpu-user-exporter](#)
- [Prometheus/Alertmanager 설정](#)
- [Grafana dashboards](#)
- [monitoring Ansible](#)
- [NFS forensics](#)
- [Kerberos/NFS keytab checker](#)

Kerberos credential과 FQDN mount 원칙은 [Kerberos/NFS 설계](#), KVNO·timer·mount 장애 절차는 [Kerberos/NFS 운영](#)을 함께 참고한다.

Monitoring 설계

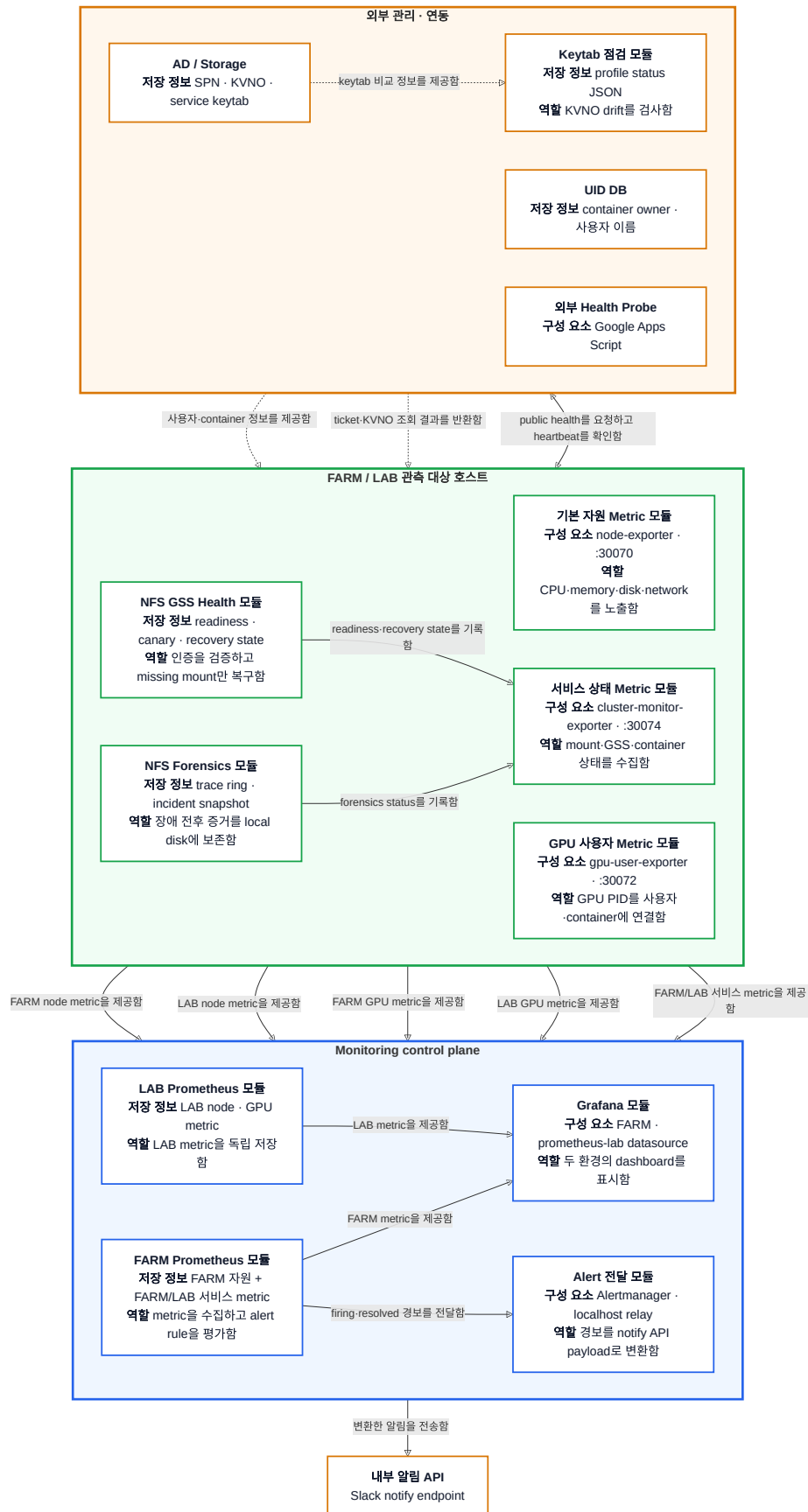
이 문서는 host의 원시 상태가 metric, alert와 dashboard가 되기까지의 흐름과 자동 복구의 안전 경계를 설명한다.

1. 설계 목표

- FARM/LAB host, GPU, container와 storage 상태를 동일한 metric 체계로 관측한다.
- HTTP process가 살아 있는지와 실제 collection이 갱신되는지를 구분한다.
- NFS **hard** mount와 D-state 상황에서도 monitoring loop 전체가 멈추지 않게 한다.
- alert에는 원인 진단에 필요한 stage와 label을 포함하되 cardinality를 제한한다.
- 관측, 증거 수집과 상태 변경을 분리하여 monitoring이 장애 반경을 넓히지 않는다.
- FARM/LAB Prometheus failure domain은 분리하고 공통 Grafana에서 함께 조회한다.

2. 전체 데이터와 알림 흐름

다이어그램의 **굵은 바깥 제목은 배치 영역**이고, 각 카드의 **굵은 첫 줄은 모듈 이름**이다. 카드 안의 **저장 정보 / 구성 요소**에는 명사만 적고, **역할**에는 주체가 무엇을 하는지 동사로 적었다. 화살표에는 출발 모듈이 요청하거나 제공하는 동작을 표시했으며, 점선은 외부 값을 읽기 전용으로 조회하는 흐름이다.



구성요소별 책임

배치 영역	모듈	내부 구성 요소·정보	역할
FARM/LAB host	기본 자원 Metric	<code>node-exporter</code> , <code>:30070</code>	CPU, memory, filesystem, disk와 network metric 노출
FARM/LAB host	GPU 사용자 Metric	<code>gpu-user-exporter</code> , <code>nvidia-smi</code> , Docker, UID DB cache	GPU process를 실제 사용자·container·GPU에 연결
FARM/LAB host	서비스 상태 Metric	<code>cluster-monitor-exporter</code> , <code>:30074</code> , public <code>:N89</code>	mount, GSS, GPU, Docker, container, 연결성과 D-state 수집
FARM/LAB host	NFS GSS Health	readiness, canary, guarded recovery worker와 state	인증 stage를 검증하고 안전 조건을 통과한 missing mount만 복구
FARM/LAB host	NFS Forensics	packet/ftrace ring, watcher, snapshot	user share를 읽거나 복구하지 않고 장애 전후 증거 보존
관리 host	Keytab 점검	profile checker, user systemd timer, status JSON	AD KVNO, storage keytab과 service ticket drift를 읽기 전용 검사
FARM control plane	FARM Prometheus/Alert	Prometheus, rule, Alertmanager, relay	FARM 자원과 FARM/LAB 서비스 metric을 수집하고 중앙 경고 전달
LAB control plane	LAB Prometheus	Prometheus, local PV, <code>:30073</code>	LAB node/GPU metric을 별도 저장; Grafana와 Alertmanager는 실행하지 않음
FARM control plane	Grafana	FARM/LAB datasource와 dashboard	두 Prometheus의 시계열을 공통 UI에서 조회

데이터 plane과 control plane

종류	예	설계 원칙
관측 data plane	<code>/proc</code> , <code>nvidia-smi</code> , Docker inspect, mountinfo, state JSON	bounded timeout, read-only 우선
metric plane	exporter <code>/metrics</code> → Prometheus	scrape 실패와 collection stale을 별도 표시
alert plane	Prometheus rule → Alertmanager → relay	secret은 Kubernetes Secret, alert label로 credential 전달 금지
visualization plane	FARM/LAB datasource → Grafana	datasource UID와 cluster label로 환경 구분
제한된 control plane	container start, SSH start, NVML repair, missing mount worker	명시적 enable, 사전조건, retry/inhibit 필요

3. Host metric exporter 구성

3.1 node-exporter: 일반 host 자원

`node-exporter` 는 CPU, memory, filesystem, disk I/O와 network 같은 일반 host 자원을 `:30070/metrics` 로 노출한다. FARM과 LAB Prometheus가 각 환경의 node metric을 저장하며, 서비스별 진단은 `cluster-monitor-exporter` 가 담당한다. 따라서 CPU 사용량이 보인다는 사실만으로 NFS readiness나 container SSH가 정상이라고 판단하지 않는다.

`node-exporter` 는 host systemd 서비스로 배포하는 두 custom exporter와 달리 `kube-prometheus-stack` 의 DaemonSet/service 설정으로 배포된다. 포트와 scrape 설정은 FARM/LAB Prometheus values의 `prometheus-node-exporter` 에서 확인한다.

3.2 cluster-monitor-exporter: 서비스 상태

Background collection과 freshness

exporter는 HTTP 요청 때마다 무거운 host 명령을 실행하지 않고 background loop의 마지막 결과를 text format으로 제공한다. `/healthz` 는 process 응답뿐 아니라 마지막 성공 collection이 `metrics_stale_after` 이내인지 확인한다.

이 구분이 필요한 이유는 HTTP server는 살아 있지만 child command나 collection goroutine이 멈춘 상태를 `up=1` 만으로는 찾을 수 없기 때문이다. 대표 지표는 `cluster_monitor_exporter_last_collection_timestamp_seconds` 와 `cluster_monitor_exporter_metrics_stale_after_seconds` 다.

수집/렌더링 구현은 `main.go`에서 확인한다.

관측 범위

- required mount의 source/target/filesystem/security option과 응답 시간
- storage host 및 peer 연결성, mount failure diagnosis
- 모든 thread를 포함한 host D-state process와 command별 count
- kernel hung-task 증가, NFS lease와 forensic 상태
- host `nvidia-smi` , Docker daemon
- 대상 image container의 running/SSH/GPU readiness
- NVML driver/library mismatch와 제한된 repair 결과
- 외부 connectivity heartbeat와 public inbound health
- NFS GSS readiness/canary/recovery state와 `rpc-gssd` journal

Container 실행 상태와 GPU alert gating

container 내부 GPU readiness는 container가 실행 중일 때만 의미가 있다. collector는 정지한 대상 container에 대해 `cluster_monitor_container_running=0` 과 `cluster_monitor_container_gpu_up=0` 을 함께 노출하지만, 이때 `gpu_up=0` 은 GPU 장애가 아니라 검사를 실행할 수 없다는 뜻이다.

따라서 GPU alert는 다음처럼 running metric을 선행조건으로 결합한다. `instance` , `server` , `container` , `image` label로 vector를 매칭하여 다른 host의 동명 container가 결합되지 않게 한다.

```
cluster_monitor_container_gpu_up{job="cluster-monitor-exporter"} == 0
and on (instance, server, container, image)
cluster_monitor_container_running{job="cluster-monitor-exporter"} == 1
```

Container 상태	running	gpu_up	발생해야 하는 alert
정지	0	0	container stopped alert만 발생
실행 중, GPU 정상	1	1	없음
실행 중, GPU 실패가 5분 지속	1	0	container GPU alert 발생

이 gating은 하나의 원인인 container 정지가 stopped와 GPU failure로 중복 통지되는 것을 막고, container가 실제로 실행 중인데 `nvidia-smi` 가 실패하는 경우만 GPU 경보로 분류한다.

D-state와 명령 timeout

NFS `hard` mount에서 child process가 D-state에 들어가면 `SIGKILL` 에도 즉시 끝나지 않고 `cmd.Wait()` 가 무기한 block될 수 있다. exporter는 외부 명령마다 timeout을 두고 collection loop가 한 NFS probe 때문에 완전히 멈추지 않게 한다.

D-state는 발생 즉시 total과 command별 metric으로 기록하되 alert는 30분 연속 지속을 기다린다. `sshd`, `find`, `du`, `chown`, `chmod`, `codex`, `claude`, `python`, `python3` 은 0인 시계열도 유지하고, 다른 command는 실제 관측될 때만 동적으로 노출한다. hung-task alert는 과거 누적 line이 아니라 최근 5분 증가분을 사용한다.

제한된 container 복구

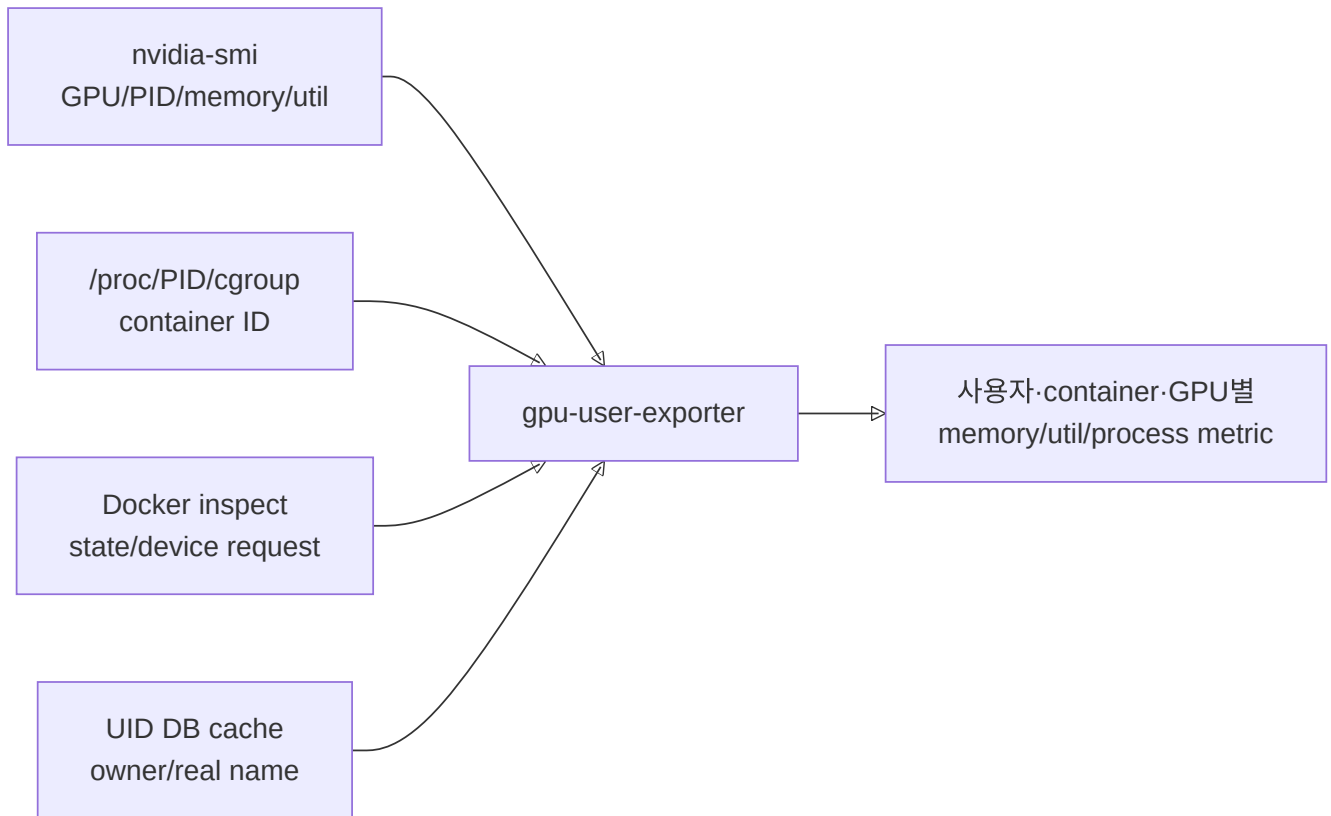
설정으로 허용된 경우에만 다음을 수행한다.

- stopped target container 시작
- container 내부 SSH service 시작
- host driver version 검증 뒤 `libnvidia-ml.so.1` symlink repair

임의 image/container, user data, host mount를 변경하지 않는다. 복구 시도와 결과는 별도 metric으로 남겨 “정상”과 “복구되어 정상”을 구분한다.

4. gpu-user-exporter

`nvidia-smi` 는 PID와 GPU 사용량만 제공하므로 실제 사용자와 container를 알기 위해 여러 정보를 join한다.



주요 지표:

- `docker_gpu_user_memory_used_bytes`
- `docker_gpu_user_sm_utilization_percent`
- `docker_gpu_user_process_count`
- device 전체 utilization/memory
- ignored process count
- DB cache entry/refresh success
- exporter scrape success/duration

DB에 없는 process를 임의의 사용자와 연결하지 않고 ignored count로 분리한다. 매 scrape마다 DB를 조회하지 않도록 활성 container owner를 cache하고 refresh 성공과 cache 크기를 metric으로 노출한다. 구현은 [gpu-user-exporter main.go](#)에 있다.

5. NFS GSS readiness, canary와 recovery

Readiness

부팅 시 NFS mount가 Kerberos host identity보다 먼저 실행되는 race를 막기 위해 다음 stage를 순서대로 확인한다.

```

host keytab -> machine kinit -> NFS service kvno
-> rpc_pipefs -> rpc-gssd -> (환경별 canary)
  
```

각 stage와 diagnosis는 state file에 기록되고 exporter가 metric으로 변환한다. 구현은 [readiness script](#)와 [NFS GSS collector](#)에 있다.

Canary

canary는 실제 user share가 아니라 전용 read-only export에서 service path를 확인한다. LAB은 canary를 사용하며 FARM은 전용 export가 준비되기 전까지 canary gate만 비활성화하고 나머지 readiness를 유지한다.

Guarded missing-mount recovery

exporter는 recovery state를 관측하고 loopback API로 worker를 queue할 수 있지만 실제 `mount` 는 별도 systemd service가 수행한다. worker gate는 다음과 같다.

1. 기대 source/filesystem/security로 이미 mount되어 있으면 즉시 healthy로 종료한다.
2. mount가 없을 때만 fstab 기대 entry를 확인한다.
3. NFS/RPC D-state가 있으면 새 mount 시도를 보류한다.
4. DNS와 storage RPC를 확인한다.
5. Kerberos readiness를 확인한다.
6. 환경에서 요구하는 canary를 확인한다.
7. retry/backoff/inhibit가 허용할 때만 `mount <target>` 을 실행한다.

healthy mount를 unmount/remount하지 않으며, mount timeout이나 관련 D-state가 생기면 현재 boot에서 반복 시도를 inhibit할 수 있다. 구현은 [mount recovery script](#)에 있다.

6. NFS forensics

NFS 장애는 재시작 후 증거가 사라질 수 있으므로 recovery와 독립된 passive forensics를 둔다.

- TCP/2049 packet의 크기 제한 ring
- 저용량 ftrace instance
- 5초 간격 kernel/NFS/D-state 상태 관측
- incident threshold에서 pre-incident ring을 local snapshot으로 보존
- PID와 process start time을 함께 사용해 PID 재사용으로 duration이 이어지지 않게 함

forensics는 mount/unmount, user share 읽기, NFS/GSS restart, recovery, reboot를 하지 않는다. exporter는 `/run/decs-nfs-forensics/status.json` 만 읽는다. `sec=krb5` 는 payload privacy를 제공하지 않으므로 capture와 incident directory는 root-only다.

구현은 [trace buffer](#), [watcher](#), [snapshot](#), [metric adapter](#)에서 확인한다.

7. Keytab health check

AD/NAS service keytab은 exporter의 자동 복구 대상이 아니다. 별도 user systemd timer가 profile별 checker를 실행한다.

```
AD KVNO + required SPN
vs storage keytab KVNO/principal
+ 실제 service ticket 복호화
+ GSS service process/forbidden option
-> profile status JSON
```

exit code는 정상 0, 점검 오류 1, drift 2 다. keytab이나 credential 원문은 status에 기록하지 않는다. 코드는 [check-nfs-keytab.sh](#)에 있다.

8. Prometheus, Alertmanager와 Grafana

Cluster 분리

- FARM: kube-prometheus-stack, Prometheus/Grafana local PV, 공통 Grafana 제공
- LAB: LAB8 control plane의 독립 Prometheus, NodePort 30073
- Grafana: LAB datasource UID prometheus-lab 으로 LAB dashboard 조회

retention, storage, target, Alertmanager routing과 control-plane failure domain이 다르므로 Prometheus release와 values를 분리한다. dashboard는 운영자가 한 UI에서 비교하므로 공통 디렉터리에 둔다.

Alert delivery

Prometheus rule은 Alertmanager로 전달되고, Alertmanager는 localhost relay sidecar에 native payload를 보낸다. relay가 내부 notify API contract로 변환하여 /api/internal/slack/notify 로 전달한다. Grafana password와 Slack webhook은 values가 아니라 Kubernetes Secret으로 관리한다.

FARM과 LAB의 ClusterMonitor.* 경보는 cluster별 receiver로 나누지 않고 하나의 receiver를 사용한다. cluster label은 grouping과 진단 표시에만 사용하며 Slack channel 선택에는 사용하지 않는다. relay가 모든 경보에 cluster-monitor-slack-webhook-farm Secret을 적용하므로 LAB 전용 receiver와 webhook Secret은 필요하지 않다. relay는 alert fingerprint별 마지막 firing/resolved 상태를 persistent state에 기록하여 동일 상태의 반복 전송도 제거한다.

canonical FARM rule과 routing은 prometheus-farm-values.yaml, relay 구현은 Alertmanager relay ConfigMap에서 확인한다.

9. 설정과 코드 위치

관심사	기준 파일
exporter 공통 기본값, FQDN mount/SPN, GSS/forensics option	group_vars/exporters.yaml
exporter build/install/systemd config	deploy_exporters.yaml
GSS readiness/canary/recovery 설치	deploy_nfs_gss_health.yaml

관심사	기준 파일
forensics 설치	deploy_nfs_forensics.yml
FARM Prometheus/Alertmanager rules	prometheus-farm-values.yaml
LAB Prometheus values	prometheus-lab-values.yaml
GPU/NIC/storage dashboards	grafana/dashboards

10. 설계 변경 시 확인할 항목

- 새 metric의 이름, type, help, label cardinality와 실패 시 값
- command timeout과 NFS D-state에서 collection loop가 유지되는지
- alert `for` 시간이 일시적 부팅/배포 지연보다 충분히 긴지
- recovery의 enable flag, precondition, retry/backoff, inhibit와 audit metric
- user data를 읽지 않는 canary/probe 경로
- FARM/LAB datasource, job, label과 dashboard query의 일치
- credential, webhook, DB password가 Git/metric/label/log에 노출되지 않는지

Monitoring 운영

이 문서는 monitoring 구성 요소를 배포하고 endpoint, metric, alert와 incident를 점검하는 방법을 설명한다.

운영 원칙

- `up=1` 과 실제 collection freshness를 구분한다. HTTP process가 응답해도 마지막 성공 collection이 오래됐으면 정상으로 판단하지 않는다.
- metric 수집, 증거 보존과 상태 변경을 분리한다. exporter는 mount하지 않고, forensics는 service restart나 reboot를 수행하지 않는다.
- NFS caller가 D-state이면 `find`, `du`, `stat`, canary나 recovery를 반복 실행하지 않고 local state와 이미 수집된 snapshot을 먼저 보존한다.
- 자동 복구 결과는 원래 정상 상태와 구분해 metric과 state에 남긴다. inhibit 또는 backoff가 설정된 worker를 endpoint 호출로 우회하지 않는다.
- 배포는 `monitoring/ansible_playbook/` 을 기준으로 하고 target host의 개별 install script를 운영 entry point로 사용하지 않는다.
- DB password, Grafana password, webhook과 Kerberos credential은 metric, alert label, Git values와 일반 journal에 기록하지 않는다.

1. 배포 순서

새 host 또는 NFS GSS monitoring을 처음 적용할 때는 다음 순서를 권장한다.

1. inventory와 host FQDN/SPN/mount source 확인
2. NFS GSS readiness/canary/recovery worker 설치
3. NFS forensics 설치
4. exporter 배포
5. Prometheus target/rule 반영
6. Grafana datasource/dashboard 확인
7. alert delivery test

운영 배포 entry point는 exporter 내부 개별 shell script가 아니라 `monitoring/ansible_playbook/` 이다.

2. Exporter 배포

```
cd /home/jy/server_manage/monitoring
```

두 exporter를 한 host에 배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=farm8
```

FARM 전체 또는 FARM/LAB 전체:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml \  
-e exporter_hosts=FARM
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_exporters.yml
```

한 exporter만 전체 재배포:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_cluster_monitor_exporter_all.yml
```

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_gpu_user_exporter_all.yml
```

target에는 다음 systemd service와 endpoint가 생긴다.

```
cluster-monitor-exporter.service -> :30074/metrics, :30074/healthz, :N89/healthz
gpu-user-exporter.service        -> :30072/metrics, :30072/-/healthy
```

배포 기준과 요구 조건은 [Ansible README](#), 실제 task는 [deploy_exporters.yml](#)에서 확인한다.

3. NFS GSS와 forensics 배포

NFS GSS readiness/canary/recovery worker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_gss_health.yml \
-e nfs_gss_health_hosts=lab8
```

NFS forensics:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \
ansible-playbook ansible_playbook/deploy_nfs_forensics.yml
```

중요한 systemd unit과 state:

```
decs-kerberos-nfs-ready.service
decs-nfs-gss-canary-probe.service/.timer
decs-nfs-gss-mount-recovery.service/.timer
/run/decs-nfs-gss/ready.state
/var/lib/decs-nfs-gss/canary.state
/var/lib/decs-nfs-gss/recovery.state

decs-nfs-forensics-buffer.service
decs-nfs-forensics-watch.service/.timer
decs-nfs-forensics-snapshot.service
/run/decs-nfs-forensics/status.json
/var/lib/decs-nfs-forensics/
```

FARM은 전용 read-only canary export가 준비될 때까지 canary만 비활성화하며 keytab/kinit/kvno/rpc-gssd readiness와 guarded recovery는 유지한다. LAB canary는 `sec=krb5` 전용 read-only export를 사용한다.

4. Keytab checker 배포

FARM 읽기 전용 checker:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml
```

LAB을 상시 점검 대상으로 전환한 뒤에만 두 profile을 활성화한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_kerberos_nfs_keytab_check.yml \  
-e '{"keytab_check_profiles":["farm","lab"]}'
```

```
systemctl --user list-timers 'check-nfs-keytab@*.timer'  
systemctl --user status check-nfs-keytab@farm.service --no-pager
```

checker는 drift를 고치지 않는다. KVNO history, repair와 rotation은 [Kerberos/NFS 운영](#)을 따른다.

5. Prometheus/Alertmanager 배포

먼저 server-side render와 cluster precondition만 검증한다.

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml
```

변경 적용:

```
ANSIBLE_CONFIG=ansible_playbook/ansible.cfg \  
ansible-playbook ansible_playbook/deploy_prometheus.yml \  
-e prometheus_dry_run=false
```

한 환경만 배포하려면 `prometheus_farm_enabled=false` 또는 `prometheus_lab_enabled=false` 를 사용한다. playbook은 kubeconfig 대상 cluster, node Ready/DiskPressure, 필수 Secret 이름을 확인한 후 Helm을 실행한다.

FARM에서 필요한 Secret:

- `monitoring-grafana-admin`: `admin-user`, `admin-password`
- `cluster-monitor-slack-webhook-farm`: `url`

값은 Git values에 넣지 않는다. Alertmanager는 Slack webhook으로 직접 보내지 않고 localhost relay를 거쳐 internal notify API로 보낸다.

현재 control plane의 역할은 다음처럼 구분한다.

- FARM Prometheus는 FARM 자원 metric과 FARM/LAB의 `cluster-monitor-exporter` 를 수집하고 중앙 alert rule을 평가한다.

- FARM Alertmanager와 relay는 FARM/LAB 서비스 경보를 내부 notify API로 보낸다.
- LAB Prometheus는 LAB node/GPU metric을 독립 저장한다. LAB release의 Grafana와 Alertmanager는 비 활성화되어 있다.
- FARM Grafana는 기본 FARM datasource와 `prometheus-lab` datasource를 함께 사용한다.

6. 배포 후 endpoint 확인

target host에서:

```
systemctl status cluster-monitor-exporter gpu-user-exporter --no-pager
journalctl -u cluster-monitor-exporter -n 100 --no-pager
journalctl -u gpu-user-exporter -n 100 --no-pager

curl -fsS http://127.0.0.1:30074/healthz
curl -fsS http://127.0.0.1:30074/metrics | head
curl -fsS http://127.0.0.1:30072/-/healthy
curl -fsS http://127.0.0.1:30072/metrics | head
curl -fsS http://127.0.0.1:30070/metrics | head
```

:30070의 `node-exporter`는 `kube-prometheus-stack`에서 배포하므로 앞의 두 custom exporter처럼 host systemd unit을 확인하지 않는다. endpoint가 없으면 그 cluster의 DaemonSet, Service와 node scheduling 상태를 확인한다.

public health port는 서버 번호 `N`에 대해 `9000 + N*100 - 11`이다. 예를 들어 FARM8은 `9789/healthz`다. 이 endpoint는 exporter process/collection freshness와 외부 NAT 도달 경로를 확인하는 용도이며, 모든 dependency가 준비됐음을 보장하지는 않는다.

NFS GSS host-only API:

```
curl -fsS http://127.0.0.1:30074/nfs-gss/ready
curl -fsS http://127.0.0.1:30074/nfs-gss/health
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/probe
curl -fsS -X POST http://127.0.0.1:30074/nfs-gss/recover
```

`probe`와 `recover`는 loopback 요청만 허용하며 worker를 queue한 뒤 즉시 응답한다. `recover`를 반복 호출하기 전에 recovery state와 inhibit reason을 확인한다.

7. 무엇을 어디에서 보는가

관측 대상	대표 metric/alert	구현/설정 링크
CPU, memory, filesystem, disk, network	<code>node_cpu_*</code> , <code>node_memory_*</code> , <code>node_filesystem_*</code> , <code>node_disk_*</code> , <code>node_network_*</code>	FARM values, LAB values
exporter process와 freshness	<code>up</code> , <code>cluster_monitor_exporter_last_collection_timestamp_seconds</code> , <code>ClusterMonitorExporter*</code>	collector, rules
required mount와 latency	<code>cluster_monitor_host_mount_up</code> , <code>..._responsive</code> , <code>..._probe_seconds</code>	mount collector, storage dashboards
Kerberos NFS readiness	<code>cluster_monitor_nfs_gss_ready</code> , <code>ClusterMonitorNFSGSSReadinessFailed</code>	readiness, GSS collector
canary/recovery	<code>cluster_monitor_nfs_gss_canary_*</code> , <code>..._recovery_*</code>	canary, recovery
D-state, lease, hung task	<code>cluster_monitor_host_dstate_*</code> , <code>cluster_monitor_nfs_*</code> , <code>ClusterMonitorNFS*</code>	forensics adapter, forensics scripts
host GPU/Docker/container	<code>cluster_monitor_host_gpu_up</code> , <code>...docker_daemon_up</code> , <code>...container_*</code>	cluster collector
사용자별 GPU	<code>docker_gpu_user_memory_used_bytes</code> , <code>...sm_utilization_percent</code> , <code>...process_count</code>	gpu-user-exporter, GPU dashboard
외부 연결	<code>cluster_monitor_external_connectivity_*</code> , public health	exporter, Apps Script
AD/NAS keytab	checker JSON/exit code	keytab checker

canonical alert rule은 [prometheus-farm-values.yaml](#)이다. exporter 내부

`prometheus/cluster_monitor_alerts.yml` 은 metric 이름을 보여 주는 reference이며 실제 FARM release와 값이 다를 수 있다.

8. Alert별 진단 순서

Exporter down 또는 stale

1. Prometheus target에서 connection error와 scrape timestamp를 구분한다.
2. target systemd status/journal을 확인한다.
3. `:30074/healthz` 와 `/metrics` 를 각각 확인한다.
4. process는 살아 있지만 stale이면 마지막 실행 command와 D-state를 확인한다.
5. 변경된 env/config와 binary 배포 시간을 확인한 뒤 exporter만 재배포한다.

Required mount down/slow/unresponsive

1. `findmnt` 로 mount 존재, FQDN source, filesystem과 `sec` 를 확인한다.
2. mount probe와 storage ping/peer diagnosis label을 확인한다.
3. GSS readiness에서 처음 실패한 stage를 확인한다.
4. D-state, lease expiry, hung-task와 local forensic snapshot을 확인한다.
5. mount가 없을 때만 recovery state를 확인한다.
6. mount가 이미 있거나 D-state caller가 있으면 강제 remount를 반복하지 않는다.

Kerberos source는 IP가 아니라 FQDN이어야 한다. 자세한 기준은 [Kerberos/NFS 운영의 mount 절](#)을 따른다.

GSS readiness/canary/recovery

```
systemctl status decs-kerberos-nfs-ready.service --no-pager
systemctl status decs-nfs-gss-canary-probe.timer --no-pager
systemctl status decs-nfs-gss-mount-recovery.timer --no-pager
journalctl -u decs-kerberos-nfs-ready.service -n 100 --no-pager
cat /run/decs-nfs-gss/ready.state
cat /var/lib/decs-nfs-gss/canary.state
cat /var/lib/decs-nfs-gss/recovery.state
```

- keytab/kinit stage: host machine credential 확인
- kvno stage: DNS/FQDN/SPN/KDC 확인
- rpc-gssd stage: service와 잘못된 `-n` override 확인
- canary stage: 전용 export와 user share를 혼동하지 않았는지 확인
- inhibited recovery: D-state 또는 mount timeout 증거를 보존하고 원인을 처리

GPU 사용자 mapping 이상

1. `nvidia-smi` 와 exporter scrape success를 확인한다.
2. ignored process count가 증가했는지 확인한다.
3. PID의 cgroup container ID와 Docker inspect를 비교한다.
4. DB cache refresh success와 cache age/entry를 확인한다.
5. DB record와 실제 running container가 어긋났다면 user-lifecycle에서 관리하는 절차로 고친다.

Container SSH/GPU alert

1. container가 running인지 먼저 확인한다.
2. exporter가 start/SSH/NVML recovery를 시도했는지 metric과 journal을 확인한다.
3. container image가 configured regex 대상인지 확인한다.
4. NVML mismatch가 아니라 application/GPU allocation 문제면 자동 symlink repair를 반복하지 않는다.

9. Grafana 점검

- LAB dashboard datasource UID가 `prometheus-lab` 인지 확인한다.
- dashboard query의 `cluster`, `server_id`, `job` label이 scrape config와 일치하는지 확인한다.
- storage dashboard에서 mount probe, responsive, D-state, blocked process, hung-task를 같은 시간축으로 비교한다.
- GPU dashboard에서 DB-known 사용자 metric과 device 전체 metric을 함께 본다.
- 새 server 추가 시 dashboard에 hard-coded server/GPU panel 누락이 없는지 확인한다.

dashboard 원본:

- [FARM storage latency](#)
- [LAB storage latency](#)
- [GPU usage](#)
- [Network traffic](#)

10. 테스트

```
cd /home/jy/server_manage/monitoring

(cd prometheus/exporters/cluster-monitor-exporter && go test ./...)
(cd prometheus/exporters/gpu-user-exporter && go test ./...)
bash nfs-forensics/tests/test_watch.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_ready.sh
bash prometheus/exporters/cluster-monitor-exporter/tests/test_nfs_gss_mount_recovery.sh
python3 prometheus/config/tests/test_slack_notify_relay.py
```

변경 위험에 맞게 최소한 다음을 함께 검증한다.

- metric 이름/type/help와 0/failure path
- recovery가 healthy mount를 건드리지 않는지
- D-state에서 mount attempt가 차단되지만 existing mount는 healthy로 처리되는지
- Alertmanager relay가 secret을 출력하지 않고 internal API payload로 변환하는지
- FARM/LAB dashboard datasource가 바뀌지 않았는지

11. 새 서버 추가 체크리스트

1. `ansible_playbook/inventory.ini` 에 `farmN` / `labN` 형식으로 추가
2. `group_vars/exporters.yml` 의 mount source, SPN과 환경 기본값 확인
3. host share target와 public `N89` port 계산 확인

4. GSS readiness/forensics/exporter 배포
5. Prometheus `:30070`, `:30072`, `:30074` scrape target 추가
6. 방화벽/NAT public health 확인
7. Grafana dashboard server/GPU panel 확인
8. alert rule label과 Alertmanager route 확인
9. endpoint와 실제 alert delivery 검증

12. 운영 문서에 계속 추가할 내용

설계와 운영을 분리한 뒤 다음 정보를 운영 문서에 누적하면 좋다.

- 서비스별 owner, endpoint, systemd/Kubernetes resource와 dependency
- metric별 정상 범위, alert `for` 와 runbook link
- 최근 배포 version/commit과 마지막 검증 시각
- FARM/LAB retention, PV 용량과 capacity threshold
- alert silence 승인/만료 기준과 false-positive 기록
- 자동 복구별 시도 횟수, inhibit 해제와 rollback 절차
- incident snapshot 보존 기간, 접근 권한과 개인정보 취급
- 새 server/metric/dashboard 추가 checklist

설계 문서에는 구성 요소의 관계와 안전 경계를 유지하고, host별 현재 값과 일회성 incident 결과는 canonical config/runbook 또는 별도 evidence에 기록한다.

remote-operations

원본: md/system/remote-operations/index.md, md/system/remote-operations/design.md, md/system/remote-operations/operations.md, md/system/remote-operations/config.md

역할: FARM/LAB 서버를 깨우고, 부팅 시 한 번 상태를 확인한 뒤 기존 컨테이너를 시작한다.

이 문서는 remote-operations 문서의 시작점이다. **설계**는 부팅 orchestration의 책임 범위와 각 스크립트의 역할을 설명하고, **운영**은 실제 서버 관리자가 배포·점검·장애 대응에 사용하는 명령을 설명하며, **설정**은 config 파일을 어떻게 준비하는지 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	디렉터리 지도, 부팅 상태 머신, 핵심 기능, 설계상 지켜야 할 경계
운영	사용법, dry-run과 검증, 실패 처리 원칙, 새 서버 추가
설정	config 파일 준비와 설정 모델

처음 보는 사람은 **설계 문서**에서 전체 구조를 먼저 확인하고, 실제 서버를 켜거나 점검해야 할 때 **운영 문서**로, config 파일을 새로 준비해야 할 때 **설정 문서**로 이동한다.

전체 서버 동시 부팅 전환과 Slack 알림 백엔드 큐 연동 검증은 완료되어 이 문서들에 반영됨. `.sh` → `.py` 전환 작업은 아직 진행 전이며, 반영되면 이 페이지와 설계/운영 문서 내용도 함께 갱신해야 한다.

remote-operations 설계

개요 · 운영 · 설정

역할: FARM/LAB 서버를 깨우고, 부팅 시 한 번 상태를 확인한 뒤 기존 컨테이너를 시작한다.

1. 책임 범위

`remote-operations` 는 management host의 `remote-boot.service` 가 실행하는 부팅 orchestration이다. Wake-on-LAN, 전체 대상 host health gate, 1회 host health check와 container post-check를 소유한다.

주기적인 mount/GPU/container 관측은 `monitoring` 에 있다. 이 경계를 나눈 이유는 부팅 시 복구와 상시 self-healing이 서로 다른 retry 정책과 장애 의미를 가지기 때문이다. 이 디렉터리에서 관리하는 systemd unit은 `remote-boot.service` 하나다.

2. 디렉터리 지도

경로	핵심 기능
<code>script/run_remote_boot.sh</code>	전체 부팅 흐름의 entry point
<code>script/wake_targets.sh</code>	target 선택과 WOL magic packet 전송
<code>script/wait_for_priority_servers.sh</code>	선택된 대상 전체의 host health gate
<code>script/check_server_boot_health.sh</code>	host SSH 도달성, mount, host GPU 점검
<code>script/restart_all_remote_containers.sh</code>	기존 container 시작과 SSH/GPU post-check
<code>script/create_test_container.sh</code>	(부팅 흐름에서 호출되지 않는 독립 도구) 임시 GPU container 생성
<code>script/delete_test_container.sh</code>	(부팅 흐름에서 호출되지 않는 독립 도구) 임시 container 정리
<code>script/common.sh</code>	config, target, Ansible, log, alert 공통 함수
<code>script/install_remote_boot_service.sh</code>	systemd unit 설치·활성화
<code>test/dry_run_remote_boot.sh</code>	WOL/health/container/full flow simulation
<code>test/integration_smoke_test.sh</code>	수동 Ansible/Docker/GPU 통합 확인
<code>test/test_slack_notification.sh</code>	Slack webhook 설정과 알림 발송 테스트
<code>config/remote_boot.example.env</code>	공개 가능한 설정 구조
<code>config/remote_boot.local.env</code>	실제 MAC·webhook 등을 담는 ignored 설정

3. 부팅 상태 머신

```
pre-delay
|
v
wake all selected targets (동시)
|
v
health gate for all selected targets -- failure --> stop + alert
|
v
start stopped target containers
|
v
per-container SSH/GPU post-check
```

과거에는 LAB1 / FARM1 에 있는 사용자 관리 DB(사용자, UID/GID, 할당 port와 사용자 container record를 관리하는 기준 데이터)를 먼저 기동하기 위해 LAB1 / FARM1 을 priority target으로 먼저 깨우고 health gate를 통과시킨 뒤 나머지 서버를 기동하는 2단계 구조였다. 이 priority/remaining 분리 로직은 제거되었고, 지금은 선택된 대상을 항상 동시에 wake + gate한다.

4. 핵심 기능

4.1 target 정규화

설정은 FARM/LAB 전체 목록과 기본 대상을 가진다. 명령행 target이 있으면 기본 대상을 덮어쓴다. all, domain group, 개별 FARM1 / LAB10 을 동일한 parser로 해석하여 중복을 제거한다.

MAC address와 broadcast IP는 WOL에만 쓰고, 상태 확인과 원격 명령은 Ansible inventory를 사용한다. 네트워크 주소와 논리 server ID를 분리하면 SSH port나 주소가 바뀌어도 workflow 이름을 유지할 수 있다.

4.2 1회 host health check

check_server_boot_health.sh 는 target에 대해 다음을 확인한다.

- Ansible/SSH 도달성
- 필수 FARM/LAB share mount
- host nvidia-smi

과거에는 이 뒤에 create_test_container.sh 로 임시 GPU container를 만들어 container 내부의 GPU/SSH까지 확인하고 delete_test_container.sh 로 정리하는 단계가 있었다. 여러 서버가 동시에 부팅되면 이 단계가 서버마다 GPU 전체와 메모리 192g를 잡고 무거운 이미지를 pull하는 부담으로 이어져 제거했다. 이제 host 레벨 확인까지만 하고,

container GPU 연동이 실제로 살아있는지는 `restart_all_remote_containers.sh`가 기존 사용자 container를 대상으로 재확인한다 (§ 4.3). `create_test_container.sh` / `delete_test_container.sh` 자체는 삭제하지 않았으므로, 필요하면 독립적으로 실행할 수 있다.

4.3 기존 container 재시작과 post-check

선택된 서버의 stopped target container를 시작한 뒤 각 container의 SSH와 GPU를 제한 시간 동안 확인한다. post-check 실패는 container별로 기록하고 전체 stage 실패 정보에 포함한다.

container를 무조건 재생성하지 않는 이유는 사용자 홈과 DB record, 고정 포트, 실행 옵션의 권위가 `user-lifecycle`에 있기 때문이다. 부팅 모듈은 기존 container를 시작하고 준비 여부만 확인한다.

4.4 alert suppression과 로그

실패는 구성된 경우 내부 notify API를 통해 Slack으로 보내고, 그렇지 않으면 stub log에 남긴다. 동일 실패의 반복 알림을 줄이기 위한 alert state가 별도 디렉터리에 저장된다. `reset_remote_boot_alert_state.sh`는 이 suppression 상태만 초기화한다.

service log, host health log와 alert state를 나눈 이유는 실행 이력과 알림 deduplication 상태를 독립적으로 보존하기 위해서다.

5. 설계상 지켜야 할 경계

- systemd unit을 추가하기 전에 부팅 1회 책임인지 지속 monitor 책임인지 구분한다.
- 공통 shell helper를 수정하면 모든 stage의 dry-run과 alert redaction을 확인한다.
- 실제 secret을 example 또는 log에 쓰지 않는다.
- retry를 늘리는 것으로 storage/GSS 장애를 감추지 않는다.
- container 생성/삭제 비즈니스 규칙은 `user-lifecycle` CLI를 통해 수행한다.

remote-operations 운영

개요 · 설계 · 설정

1. 사용법

target 확인과 수동 WOL (`admin_infra_server/remote-operations`에서 실행):

```
./script/wake_targets.sh --list-targets
./script/wake_targets.sh FARM1 LAB1
```

한 서버의 boot health:

```
./script/check_server_boot_health.sh --server-id FARM1
```

한 서버의 기존 container 시작/post-check:

```
./script/restart_all_remote_containers.sh FARM1
```

systemd 설치와 확인:

```
./script/install_remote_boot_service.sh  
sudo systemctl start remote-boot.service  
systemctl status remote-boot.service  
journalctl -u remote-boot.service -b
```

2. dry-run과 검증

```
./test/dry_run_remote_boot.sh wake FARM1 LAB1  
./test/dry_run_remote_boot.sh health FARM1  
./test/dry_run_remote_boot.sh containers FARM1  
./test/dry_run_remote_boot.sh full FARM1 LAB1
```

dry-run은 sleep, WOL, Ansible 변경과 Docker 작업 대신 계획을 기록한다. 설정 파싱, target 분할과 단계 순서를 production 영향 없이 검증하기 위한 공개 계약이다.

실제 연결을 확인할 때:

```
./test/integration_smoke_test.sh
```

Slack 알림 설정을 확인할 때:

```
./test/test_slack_notification.sh --server-id FARM1
```

`REMOTE_BOOT_SLACK_ENABLED` 와 webhook URL이 설정돼 있어야 하며, 테스트 메시지를 실제로 전송해 alert 경로가 살아있는지 확인한다.

운영 전에는 최소한 다음을 확인한다.

- MAC/broadcast IP와 Ansible host가 같은 물리 서버를 가리키는지
- gate timeout이 선택된 서버 전체가 동시에 부팅되는 시간을 감안해도 충분한지
- 실패 알림에 비밀값이 포함되지 않는지

3. 실패 처리 원칙

- 선택된 대상 중 gate를 통과하지 못한 서버가 있으면 container 재시작 단계로 넘어가지 않는 것이 기본이다.
- gate 단계에서 서버별 통과/대기 상태를 구분해 기록한다 (`wait_for_priority_servers.sh` 가 통과한 서버와 재시도 중인 서버를 분리해서 로그로 남긴다).
- mount 또는 GPU 문제를 부팅 script에서 무제한 복구하지 않는다. 반복 상태는 `monitoring` 으로 넘기고, Kerberos/NFS 고위험 복구는 해당 runbook을 따른다.
- 실제 사용자 container의 DB record나 Docker 옵션을 부팅 script에서 다시 만들지 않는다.

4. 새 서버 추가

1. `user-lifecycle/server_info/servers.jsonl` 과 Ansible inventory에 서버를 등록한다.
2. local env의 FARM/LAB target 목록과 MAC을 추가한다.
3. 필요한 경우 필수 mount template를 정한다.
4. WOL dry-run과 개별 WOL을 확인한다.
5. 개별 boot health check를 실행한다.
6. container restart를 한 서버에 제한해 검증한다.
7. `monitoring` 의 scrape target/exporter 배포를 별도로 완료한다.

remote-operations 설정

개요 · 설계 · 운영

설정 모델

설정은 `remote_boot.example.env` 를 복사하여 local 파일에서 관리한다 (`admin_infra_server/remote-operations` 에서 실행).

```
cp config/remote_boot.example.env config/remote_boot.local.env
```

주요 설정 그룹:

그룹	예
target	FARM/LAB 목록, 기본 target
순서/gate	pre-delay, gate timeout/poll
container	restart enable, timeout, post-check poll

그룹	예
network	Ansible inventory, broadcast IP, MAC address
health	필수 mount와 host share template
test container (독립 도구용)	<code>create_test_container.sh</code> / <code>delete_test_container.sh</code> 가 쓰는 image, UID/GID, mount, memory, runtime — 부팅 흐름에서는 더 이상 쓰이지 않음
logging/alert	log path, rotate count, notify API와 webhook

실제 MAC, password와 webhook은 `remote_boot.local.env`에만 둔다. example에는 변수 이름과 안전한 placeholder만 유지한다.

첫 설정 체크리스트

이 저장소를 처음 pull한 뒤 dry-run 검증까지 마치는 순서:

1. 개인 SSH 공개키를 각 FARM/LAB 서버의 `~/.ssh/authorized_keys`에 등록
2. `~/ansible/inventory.ini` 준비 (`ansible_user`를 자신의 계정으로)
3. `remote_boot.example.env` → `remote_boot.local.env` 복사
4. `REMOTE_BOOT_ANSIBLE_INVENTORY`를 자신의 inventory 경로로 설정
5. `REMOTE_BOOT_MAC_*` 값을 실제 MAC으로 채움
6. `REMOTE_BOOT_SLACK_ENABLED=true`와 `REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM` 설정
7. 운영 문서의 dry-run 명령으로 검증

개인 Ansible inventory 준비

관리 데스크탑은 여러 관리자가 공유한다. Ansible inventory는 host 그룹당 `ansible_user`를 하나만 가질 수 있어서, `/etc/ansible/inventory.ini` (특정 관리자 계정으로 고정)를 그대로 쓰면 다른 관리자는 그 계정으로 원격 서버에 접속을 시도하다가 SSH 인증에 실패한다. 그래서 관리자마다 자신의 계정으로 접속하는 개인 inventory가 필요하다. (UID_GID_Management_System 5.2.4 내용을 보고 inventory.ini 파일을 `~/uid_gid/inventory.ini`에 위치시킨 경우 지금 설명하는 단계를 진행해야 한다. inventory.ini의 위치가 변경되기 때문이다.)

- 위치: `~/ansible/inventory.ini`. 특정 프로젝트에 속하지 않는 개인 경로다. remote-operations와 `uid_gid` 등 이 데스크탑의 여러 도구가 각자의 설정에서 이 경로를 가리키기만 하고, 어느 한쪽도 이 파일을 소유하지 않는다.
- 준비: `/etc/ansible/inventory.ini`를 복사해서 `ansible_user`만 자신의 계정으로 바꾼다.
- 원격 서버의 `~/.ssh/authorized_keys`에 자신의 공개키를 등록해야 실제 SSH 인증이 된다. inventory 파일만으로는 접속 권한이 생기지 않는다.
- (선택) `~/ansible/ansible.cfg`를 만들어두면 `-i` 없이 `ansible <host> ...` 명령이 기본으로 이 inventory를 사용한다.


```
[defaults]
inventory = /home/<본인계정>/ansible/inventory.ini
interpreter_python = auto_silent
host_key_checking = False
retry_files_enabled = False
```

Ansible 설정이 완료되면 `remote_boot.local.env` 에서 REMOTE_BOOT_ANSIBLE_INVENTORY를 다음과 같이 수정한다.

```
REMOTE_BOOT_ANSIBLE_INVENTORY="/home/<본인계정>/ansible/inventory.ini"
```

이 개인 설정은 dry-run 검증과 수동 운영에 쓰인다. 실제 자동 부팅을 담당하는 `remote-boot.service` (systemd)는 별도 관리자 계정 소유로 이미 배포되어 있으며 이 개인 설정과 무관하게 동작한다.

MAC 주소

`REMOTE_BOOT_MAC_*` 는 물리 서버 하드웨어의 실제 MAC 주소로 장비통합관리문서에서 값을 가져온다.

Slack 알림 설정

부팅 실패 알림을 실제로 받아보려면 `remote_boot.local.env` 에서 두 값을 채운다.

```
REMOTE_BOOT_SLACK_ENABLED=true
REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM="https://hooks.slack.com/services/..."
```

- `REMOTE_BOOT_SLACK_WEBHOOK_URL_FARM` 도 MAC 주소와 마찬가지로 실제 운영 값이라 임의로 만들 수 없다. 기존 운영 담당자에게 확인하거나, 이미 배포된 운영 설정에서 값을 가져온다.
- LAB/FARM 구분 없이 모든 원격 부팅 알림은 이 FARM webhook 하나로 전송된다.
- `REMOTE_BOOT_SLACK_NOTIFY_API_URL` 은 webhook을 직접 호출하지 않고 백엔드 내부 notify API를 거친다.
- 설정 후 아래 명령으로 실제 Slack 채널에 메시지가 오는지 확인한다.

```
./test/test_slack_notification.sh --server-id FARM1
```

server-state

원본: md/system/server-state/index.md, md/system/server-state/design.md, md/system/server-state/operations.md

역할: FARM/LAB GPU 서버가 모두 같은 공통 설정을 유지하도록 기준을 정의하고, 운영 서버 점검과 신규 서버 구축에 같은 순서를 사용한다.

설계는 이 모듈이 확인·설정하는 범위, profile 구조, 모듈별 소유권과 현재 구현 수준을 설명하고, **운영**은 점검·구축 흐름에 실제로 사용하는 CLI 명령과 공통 설정 추가 절차를 설명한다.

문서 구성

문서	핵심 내용
현재 페이지	책임 범위와 문서 안내
설계	확인/설정 범위, profile 구조, 모듈별 소유권, 상태 표시, 디렉터리 구조, 현재 구현 수준
운영	운영 서버 점검·신규 서버 구축 흐름, 공용 서버 목록, 공통 설정 추가 방법, 테스트

처음 보는 사람은 **설계 문서**에서 이 모듈이 무엇을 소유하고 무엇을 다른 모듈에 위임하는지 먼저 확인하고, 실제로 서버를 점검하거나 새 서버를 구축해야 할 때 **운영 문서**를 확인한다.

server-state 설계

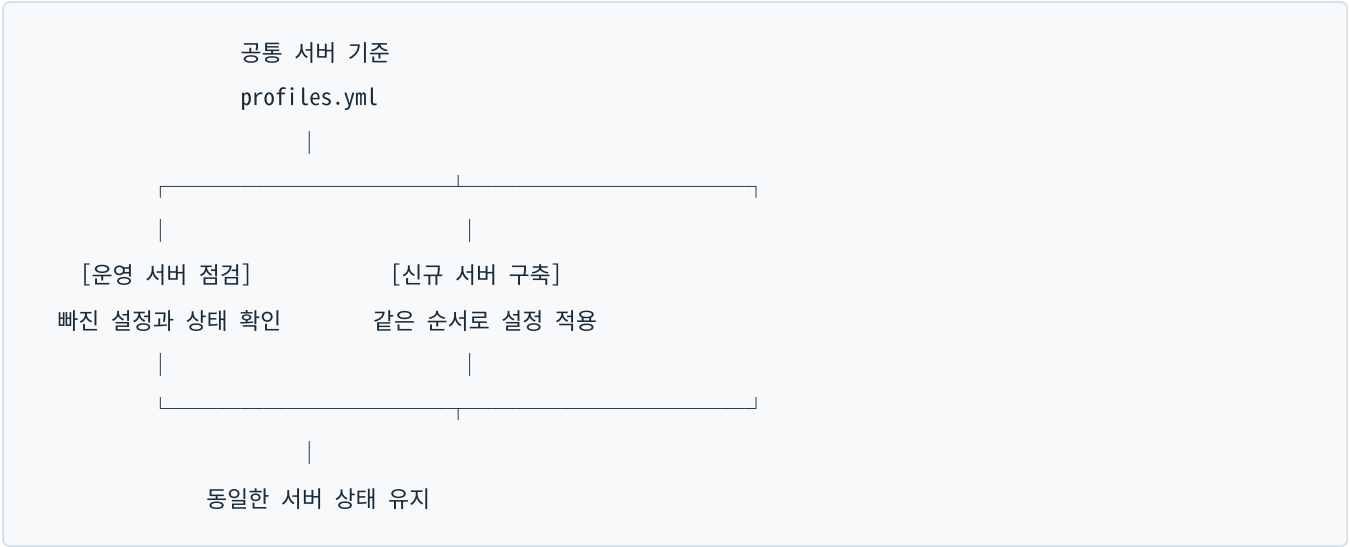
개요 · 운영

1. 이 모듈이 하는 일

server-state 는 FARM/LAB 서버가 모두 같은 공통 설정을 유지하도록 기준을 정의하는 모듈이다. 목적은 크게 두 가지다.

- 운영 서버 점검:** 각 서버에 Docker, NVIDIA driver/toolkit, Kubernetes, network tuning, Kerberos/NFS, monitoring 설정이 빠짐없이 적용되어 있는지 확인하고 차이가 있으면 복구할 방법을 제시한다.
- 신규 서버 구축:** 새 서버에 공통 설정을 같은 순서로 적용하여 운영 서버와 동일한 표준 상태로 만든다.

즉, 서버마다 설치 방법을 다시 기억해서 수동으로 설정하는 대신 하나의 공통 기준을 사용하려고 만든 코드다.



`server-state` 가 모든 기능을 직접 구현하는 것은 아니다. 공통 OS, Docker, NVIDIA와 network 설정은 직접 관리하고, Kerberos/NFS와 monitoring처럼 별도 모듈이 소유한 기능은 해당 모듈의 점검·복구 명령을 한 순서로 연결한다.

2. 디렉터리 지도

경로	핵심 기능
<code>README.md</code>	모듈 개요와 책임 범위 요약
<code>bin/server-state</code>	CLI 실행 파일
<code>script/cli.py</code>	<code>list-hosts</code> , <code>check</code> , <code>plan</code> , <code>apply</code> 출력
<code>script/inventory.py</code>	공용 JSONL inventory 로드와 host 선택
<code>script/profiles.py</code>	profile set 확장과 서버별 변수 치환
<code>script/__main__.py</code>	<code>python -m script</code> 로 CLI를 실행할 수 있게 하는 진입점
<code>config/profiles.yml</code>	공통 상태, 점검과 복구 명령의 기준
<code>ansible_playbook/bootstrap_gpu_server.yml</code>	신규/운영 서버의 공통 설정 task
<code>ansible_playbook/kerberos_nfs_client_recovery.yml</code>	Kerberos NFS 상태 점검과 제한된 복구
<code>docs/standard-gpu-server-pipeline.md</code>	신규/운영 서버 표준 단계의 개발 문서
<code>docs/module-boundaries.md</code>	모듈별 소유권 경계 정의 (§ 6 "모듈별 소유권"의 원본 문서)
<code>docs/repo-inventory-cleanup.md</code>	레포 내 일회성/legacy 파일 정리 감사 기록
<code>docs/reusable-operational-files.md</code>	재사용 가능한 운영 파일 목록
<code>requirements.txt</code>	Python 의존성 (<code>PyYAML</code>)
<code>tests/</code>	inventory와 profile 처리 unit test

3. 무엇을 확인하고 설정하는가

`new-host-bootstrap` 과 `existing-host-drift` 는 다음 항목을 같은 순서로 사용한다. 따라서 새로운 공통 설정을 추가할 때 두 흐름에 함께 넣을 수 있다.

순서	영역	운영 서버에서 확인하는 것	신규 서버에 설정하는 것
1	기본 접속 조건	inventory 등록, Ansible 접속, 비대화형 sudo, hostname	자동 설정 전 SSH, sudo, hostname, IP와 inventory를 확인
2	공통 OS	Ubuntu 계열 여부, 공통 package 설치 여부	apt repository, NFS, Kerberos와 network 도구 설치
3	Docker Engine	service 활성 상태, daemon 응답, systemd cgroup driver	Docker repository/package와 <code>daemon.json</code> 설정
4	NVIDIA driver	<code>nvidia-smi</code> 가 GPU와 driver version을 정상 출력하는지, package hold 여부	설정된 driver package 설치와 apt hold
5	NVIDIA Container Toolkit	<code>nvidia-ctk</code> , Docker NVIDIA runtime, containerd NVIDIA runtime	toolkit 설치 후 Docker/containerd runtime 설정
6	Kubernetes node	kubeadm/kubelet/kubectrl, kubelet, cluster join 파일과 node label	Kubernetes package 설치와 kubelet 활성화
7	network tuning	storage NIC 정보, RX queue 4096 이상, 영속화 service	storage NIC RX queue를 4096으로 유지하는 systemd service
8	Kerberos/NFS	realm 설정, machine keytab, service ticket, <code>rpc-gssd</code> , fstab과 mount 상태	client package/config와 GSS 준비 상태 구성
9	monitoring	두 exporter service와 metrics/health endpoint	monitoring 모듈의 exporter 배포 playbook 사용
10	사용자 container 전제조건	사용자 DB 환경, server inventory, Docker 접근	사용자 생성·삭제는 <code>user-lifecycle</code> 을 통해 처리

3.1 NVIDIA version 확인 범위

현재 점검 명령은 `nvidia-smi` 를 실행하여 GPU 이름과 실제 driver version이 출력되는지 확인한다. 따라서 driver가 GPU를 인식하지 못하거나 명령 자체가 실패하는 상태는 찾을 수 있다.

신규 설치 playbook의 기본 package는 현재 `nvidia-driver-580` 이며 설치 후 의도하지 않은 major version 변경을 막기 위해 apt hold한다. 다만 운영 서버의 driver version을 `580` 과 자동 비교하여 불일치 판정을 내리는 규칙은 아직 없다. 현재는 출력된 version을 관리자가 확인해야 한다.

3.2 network 설정 범위

현재 `network-tuning` 이 관리하는 대상은 스토리지 통신에 사용하는 NIC의 RX queue 크기다. inventory에서 서버별 storage interface를 읽고, RX queue가 4096 이상인지와 `decs-rx-queue.service` 가 활성화되어 있는지 확인한다.

IP 주소, gateway, DNS와 netplan 전체를 자동으로 설정하는 기능은 아직 없다. 신규 서버의 IP, hostname, SSH와 sudo는 bootstrap을 시작하기 전에 준비해야 하는 조건이다. 향후 모든 서버의 netplan까지 통일하려면 별도의 profile과 검증 규칙을 추가해야 한다.

4. 현재 구현 수준

이 모듈은 최종적으로 공통 기준과 실제 서버 상태를 자동 비교하고 필요한 복구까지 안전하게 실행하는 것을 목표로 한다. 현재 구현 수준은 다음과 같다.

기능	현재 상태
전체 서버가 따라야 할 공통 profile 정의	구현됨
신규/운영 서버에 같은 profile 순서 사용	구현됨
서버별 점검 명령 생성	구현됨
신규 서버 설정용 Ansible task	구현됨
check가 원격 서버를 순회하고 결과 판정	아직 구현되지 않음
apply --execute로 복구 자동 실행	아직 구현되지 않음. 명시적으로 거부됨
주기적인 전체 서버 drift 검사와 알림	아직 구현되지 않음
IP/DNS/netplan 전체 표준화	아직 구현되지 않음

따라서 “모든 서버가 같은 설정인지 확인하고 새 서버를 똑같이 설정한다”는 이해는 맞다. 다만 현재는 **기준, 점검 명령, 복구 playbook을 한곳에 정리한 단계**이고, 한 명령으로 전체 서버를 자동 검사·복구하는 controller까지 완성된 상태는 아니다.

5. profile 구조

config/profiles.yml에는 작은 단위의 profile과 이를 순서대로 묶은 profile set이 있다.

profile set	용도
new-host-bootstrap	신규 SSH-ready 서버의 표준 구축 순서
existing-host-drift	운영 서버의 공통 설정 점검·복구 순서
managed-host	운영 관리 서버용 기본 별칭
monitoring-host	monitoring 항목만 확인할 때 사용

새 공통 설정을 모든 서버에 적용하려면 작은 profile로 추가한 뒤 new-host-bootstrap과 existing-host-drift 양쪽에 넣는다. 이렇게 해야 새 서버 설치에는 들어갔지만 운영 서버 점검에서는 빠지거나, 그 반대가 되는 문제를 줄일 수 있다.

각 profile은 다음 정보를 가진다.

- 이 설정을 실제로 소유하는 모듈
- 부작용 없이 상태를 읽는 check
- 차이가 있을 때 사용할 remediation 명령
- 자동 실행할 수 없는 작업의 runbook과 safety level

6. 모듈별 소유권

영역	실제 소유자	server-state 의 역할
공통 OS, Docker, NVIDIA, Kubernetes package, network tuning	server-state	공통 기준과 bootstrap task 관리
NAS, AD, Kerberos와 NFS 정책	kerberos-nfs	점검 순서와 승인된 runbook 연결
exporter와 metrics endpoint	monitoring	배포·점검 playbook 연결
사용자와 container 생성·삭제	user-lifecycle	공용 inventory 사용과 전제조건 확인
서버 전원과 boot sequence	remote-operations	이 모듈에서 다루지 않음
사용자 container image	container-images	이 모듈에서 image 내부를 변경하지 않음

소유권을 나눈 이유는 `server-state` 에 모든 운영 코드를 복사하지 않고, 실제 담당 모듈의 rollback과 안전 절차를 그대로 사용하기 위해서다.

7. 상태 표시

상태	의미
OK	inventory나 로컬 파일처럼 즉시 확인 가능한 조건이 충족됨
MISSING	필요한 로컬 파일이나 값이 없음
DRY-RUN	실행할 원격 점검 또는 복구 명령만 표시함
MANUAL	도메인 가입, join, mount처럼 담당자 확인이 필요한 작업
UNKNOWN	아직 지원하지 않는 check 또는 remediation 형식

server-state 운영

개요 · 설계

1. 운영 서버 점검 흐름

1.1 existing-host-drift 란?

`existing-host-drift` 는 `config/profiles.yml` 에 정의된 **점검 항목 묶음의 이름**이다. 그 안에는 접속 확인(baseline-host)부터 `os-common`, `docker-engine`, `nvidia-driver`, `nvidia-container-runtime`, `kubernetes-node`, `network-tuning`, `kerberos-nfs-client`, `monitoring-exporters`, `user-container-host`까지 10개 영역이 순서대로 들어 있다. "운영 중인 FARM/LAB 서버가 이 10개 영역의 공통 기준을 다 지키고 있는지 확인하는 점검표"라고 보면 된다.

실제로 실행하는 스크립트는 `./server-state/bin/server-state` 이고, 내부 동작은 `script/cli.py` 가 맡는다. `cli.py` 가 `config/profiles.yml` 을 읽어서 `existing-host-drift` 에 묶인 10개 영역의 점검·복구 항목을 순서대로 꺼내 쓰는 구조다.

1.2 "명령어를 만들어서 보여준다"는 게 무슨 뜻인가

`profiles.yml` 에는 완성된 명령이 아니라, `{host}` 나 `{repo}` 같은 자리표시자가 들어간 **명령 틀**만 적혀 있다. 예를 들어 `os-common` 영역의 점검 항목 하나는 실제로 다음과 같이 정의되어 있다.

```
# config/profiles.yml (os-common 영역의 점검 항목 중 하나)
command: "ANSIBLE_CONFIG={repo}/monitoring/ansible_playbook/ansible.cfg \
  ansible {host} -b -m shell -a '. /etc/os-release && test \"$ID\" = ubuntu ...'"
```

`check` 명령을 실행하면 `script/cli.py` 가 이 틀의 `{host}` 를 실제 서버 이름(`farm8`)으로, `{repo}` 를 실제 저장소 경로로 바꿔 채운 뒤, 그 결과 문자열을 화면에 출력만 한다. 예시:

```
$ ./server-state/bin/server-state check --hosts farm8 --profile os-common
farm8 [os-common] ubuntu-release: DRY-RUN
  detail: ANSIBLE_CONFIG=/home/jy/server_manage/monitoring/ansible_playbook/ansible.cfg ansible farm8 -b -m
  shell -a '. /etc/os-release && test "$ID" = ubuntu && printf "%s\n" "$VERSION_ID"
```

`detail`: 뒤에 나온 문자열이 "만들어서 보여준 명령"이다. `server-state` 는 자리표시자를 채운 문자열을 Python에서 만들어 출력하기만 할 뿐, 이 명령을 실제로 실행(예: `subprocess` 호출)하지 않는다. 서버 상태를 실제로 확인하려면 관리자가 이 문자열을 복사해서 직접 실행해야 한다.

1.3 전체 점검 흐름

아래 5단계 중 실제로 서버 상태가 바뀌는 건 5번뿐이고, 그마저도 관리자가 직접 실행해야만 일어난다.

1. **점검 대상 서버 선택** — 공용 inventory(3절 "공용 inventory와 서버 목록" 참고)에서 `--hosts` 조건(`all`, `farm8` 등)에 맞는 서버를 로컬에서 골라낸다. 서버에 접속하지 않는다.
2. **점검 항목 생성** — `config/profiles.yml`의 `existing-host-drift` profile을 서버별 변수(hostname, IP 등)로 채워 로컬에서 명령 목록을 만든다. 서버에 접속하지 않는다.
3. **점검 명령을 DRY-RUN으로 출력** — 서버별로 실행하면 될 읽기 전용 Ansible 명령을 화면에 보여줄 뿐, 실행하거나 서버에 접속하지 않는다.
4. **복구 명령을 DRY-RUN으로 출력** — 3번에서 차이가 발견된 항목에 한해 복구용 Ansible 명령(`--check --diff` 포함)을 보여준다. 역시 실행하지 않는다.
5. **관리자가 검토 후 직접 실행** — 3, 4번에서 출력된 명령을 관리자가 그대로 복사해 실행해야 서버 상태가 바뀐다. `server-state` 는 이 단계를 대신 실행하지 않는다.

실제 명령은 다음과 같다. 1~3번 단계가 `check` 한 번으로 함께 실행된다.

```
cd /home/jy/server_manage

# 전체 서버의 점검 항목 확인
./server-state/bin/server-state check \
  --hosts all \
  --profile existing-host-drift

# 특정 서버(farm8)만 확인
./server-state/bin/server-state check \
  --hosts farm8 \
  --profile existing-host-drift
```

두 명령 모두 서버에 접속하거나 서버 상태를 바꾸지 않는다. 화면에 출력된 `DRY-RUN` 명령을 관리자가 직접 실행하거나, 별도 자동화가 그 명령을 실행하도록 연결해야 실제 점검(3번 단계 이후)이 이루어진다.

4번 단계, 즉 차이가 있을 때 사용할 복구 명령만 먼저 볼 수도 있다.

```
./server-state/bin/server-state plan \
  --hosts farm8 \
  --profile existing-host-drift
```

2. 신규 서버 구축 흐름

새 서버를 운영 서버와 같은 상태로 만드는 것이 `new-host-bootstrap` 의 목적이다. 다만 아무것도 설치되지 않은 장비의 전원 투입부터 전부 처리하는 형태는 아니다. 다음 조건은 먼저 준비되어 있어야 한다.

- Ubuntu가 설치되어 있다.

- IP, hostname과 SSH 접속이 준비되어 있다.
- 관리 계정이 비대화형 sudo를 사용할 수 있다.
- 서버가 공용 inventory에 등록되어 있다.

이후 표준 설치 순서를 확인한다.

```
./server-state/bin/server-state plan \
--hosts farm8 \
--profile new-host-bootstrap
```

`ansible_playbook/bootstrap_gpu_server.yml`에는 package 설치와 설정 파일 변경을 실제로 수행하는 idempotent task가 들어 있다. 각 단계는 tag로 나뉘며, 먼저 생성된 명령의 host, tag와 변수를 확인한 뒤 `--check --diff`로 예상 변경을 검토한다. 실제 적용은 검토가 끝난 Ansible 명령에서 `--check`를 제거하여 관리자가 실행한다.

다음 항목은 공통 설정이라도 자동으로 밀어 넣지 않고 별도 승인을 요구한다.

- NVIDIA driver 변경: reboot와 실행 중 GPU workload 조정이 필요하다.
- Kubernetes join: cluster별 token과 controller 승인이 필요하다.
- Kerberos machine keytab: FARM과 LAB의 도메인 가입 방식이 다르고 secret을 안전하게 전달해야 한다.
- Kerberos NFS mount: 실행 중 container와 사용자 session에 영향을 줄 수 있다.

Kerberos/NFS의 상세 점검과 복구 순서는 [Kerberos/NFS 매뉴얼](#)에서 관리한다.

현재 `apply`도 실제 변경 없이 복구 계획만 출력한다.

```
./server-state/bin/server-state apply \
--hosts farm8 \
--profile existing-host-drift
```

`apply --execute`는 아직 구현되지 않았으며 실행하면 오류로 중단된다.

3. 공용 inventory와 서버 목록

기본 inventory는 `user-lifecycle/server_info/servers.jsonl`이다. 서버 한 대당 한 줄(JSON)로 host 이름, `server_id`, domain(FARM/LAB), SSH 접속 정보 (`ansible_host`, `ansible_port`, `ansible_user`), 네트워크 인터페이스, OS 버전 등을 담는다. 이 파일은 사람이 손으로 쓰지 않는다. `user-lifecycle/server_info/generate_servers_jsonl.py`가 Ansible `setup` facts(각 서버에서 실제로 수집한 정보)와 고정 topology 정보를 합쳐서 만든 결과물이다.

이 파일은 원래 `user-lifecycle`(사용자·container 생성)이 관리하던 서버 목록이다. `server-state`는 점검할 서버 목록이 필요할 때 이 파일을 그대로 읽어서 쓰고("공용" inventory), 자신만의 목록을 별도로 만들지 않는다. IP, SSH port와 논리 server ID를 두 군데에서 따로 관리하면 서로 달라지는 문제가 생기기 때문이다. `server-state`의 `script/inventory.py`가 이 파일을 읽어서 `--hosts` selector에 맞는 서버를 골라낸다.

```
./server-state/bin/server-state list-hosts --hosts all
./server-state/bin/server-state list-hosts --hosts farm
./server-state/bin/server-state list-hosts --hosts farm8,lab10
```

selector는 `all`, `farm`, `lab`, 개별 host 이름과 여러 host 조합을 지원한다.

4. 공통 설정 추가 방법

1. 설정을 실제로 관리할 모듈을 정한다.
2. `config/profiles.yml` 에 작은 profile을 추가한다.
3. 부작용 없이 읽을 수 있는 항목만 `checks` 에 둔다.
4. 복구는 가능하면 Ansible `--check --diff` 명령부터 제공한다.
5. driver, cluster join, keytab과 mount처럼 위험한 작업은 `manual` 또는 `gated` 로 둔다.
6. 모든 서버에 필요한 설정이면 `new-host-bootstrap` 과 `existing-host-drift` 에 모두 추가한다.
7. profile 순서와 서버별 변수 치환 test를 추가한다.

5. 테스트

```
cd /home/jy/server_manage/server-state
python3 -m unittest discover -s tests -v

./bin/server-state --format json check \
  --hosts farm8 \
  --profile existing-host-drift

./bin/server-state plan \
  --hosts lab10 \
  --profile new-host-bootstrap
```

테스트와 dry-run 출력에서는 실제 운영 서버를 변경하지 않는다.

Infra

원본: md/infra/index.md, md/infra/개요.md, md/infra/design/시스템-아키텍처.md, md/infra/design/기초-개념.md, md/infra/design/데이터베이스.md, md/infra/operations/시작.md, md/infra/operations/운영-매뉴얼.md, md/infra/operations/API-레퍼런스.md, md/infra/operations/Helm-차트-레퍼런스.md

이 묶음은 `admin_infra`의 config-server가 계정, 홈 디렉터리, Pod, NodePort, Kerberos를 어떻게 다루는지 설명한다. 처음 보는 사람도 어디서 시작해야 할지 바로 잡을 수 있게, 읽는 목적별 경로와 문서 지도를 함께 둔다.

먼저 읽을 길

지금 하려는 일	추천 순서
처음 합류해서 전체를 훑는다	개요 → 시스템 아키텍처 → 운영 매뉴얼
승인 뒤 무슨 작업이 일어나는지 본다	개요 → 시스템 아키텍처 → API 레퍼런스
배포나 장애 대응을 한다	운영 매뉴얼 → 데이터베이스 → Helm 차트 레퍼런스
용어가 낯설다	기초 개념 → 필요한 본문 문서
Kerberos, AD, keytab 흐름을 본다	kdc-setup → 설계 → 운영

문서 지도

묶음	문서	이 문서에서 보는 것	다음에 읽을 문서
입문	개요	config-server가 시스템에서 맡는 역할과 큰 흐름	시스템 아키텍처
설계	시스템 아키텍처	계정, 홈 디렉터리, Pod, NodePort가 만들어지는 순서	데이터베이스, API 레퍼런스
설계	기초 개념	UID/GID, Kubernetes, NFS, NodePort 같은 배경 용어	필요한 본문 문서
설계	데이터베이스	infra-mysql과 NodePort 배정 기록 구조	운영 매뉴얼
작업 시작	처음 작업할 때	브랜치, 로컬 실행, 배포 흐름, PR 규칙	운영 매뉴얼
운영	운영 매뉴얼	배포, 상태 점검, 장애 확인, 복구 순서	데이터베이스, Helm 차트 레퍼런스
운영	API 레퍼런스	config-server 엔드포인트, 입력, 응답, 호출 주체	시스템 아키텍처
운영	Helm 차트 레퍼런스	Helm values와 차트 템플릿이 어디에 쓰이는지	운영 매뉴얼
Kerberos	kdc-setup	AD, Secret, 노드, Pod 사이의 Kerberos 흐름 전체	설계, 운영

이 묶음의 범위

- `infra`는 config-server가 직접 만드는 계정, 홈 디렉터리, Pod, NodePort, Kerberos 준비를 다룬다.
- NAS service keytab, NFS 서버 자체 설정, 사용자 이미지 내부 동작처럼 다른 묶음이 더 적합한 내용은 `system` 문서로 보낸다.
- 실제 값, 비밀번호, keytab, SSH 개인키는 여기서 설명하지 않고 운영 확인 절차와 위치만 적는다.

개요

이 문서에서 다루는 내용

- config-server가 전체 시스템에서 맡는 역할
- 승인 뒤 계정, 홈 디렉터리, Pod, NodePort가 만들어지는 큰 흐름
- 관련 저장소, 운영 주소, 다음에 읽을 문서

빠른 이동

궁금한 내용	바로 볼 문서
승인 뒤 세부 순서가 궁금하다	시스템 아키텍처
Kubernetes, UID/GID, NFS 용어가 낯설다	기초 개념
운영 점검과 장애 대응을 해야 한다	운영 매뉴얼
API 입력과 응답을 확인해야 한다	API 레퍼런스
Kerberos와 AD 설정이 궁금하다	kdc-setup

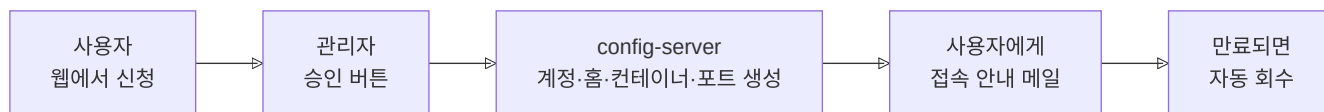
1. config-server가 하는 일

DECS는 연구실 GPU 서버를 웹에서 신청하고 승인받아 사용하는 시스템이다. 세 서버가 다음 일을 나눠 한다.

서버	하는 일
admin_fe	사용자와 관리자가 신청과 승인 결과를 확인하는 화면을 제공한다.
admin_be	신청 내용과 승인 상태를 저장하고 config-server에 작업을 요청한다.
config-server (admin_infra)	계정, 홈 디렉터리, Pod, 접속 포트를 만들거나 지운다.

admin_be가 승인 여부를 정하고, config-server가 Kubernetes와 NAS에 실제 작업을 수행한다. config-server는 스스로 사용자를 승인하지 않는다.

2. 승인 뒤에 일어나는 일



관리자가 승인하면 config-server는 다음 순서로 작업을 한다.

1. **계정 만들기** — 리눅스 계정(UID)을 공용 계정 파일에 등록
2. **홈 폴더 만들기** — NAS(공용 저장 장비)에 그 사용자의 홈 디렉터리를 생성
3. **GPU Pod 만들기** — GPU 사용량이 낮은 노드를 골라 Pod를 만든다.
4. **접속 포트 열기** — SSH·Jupyter로 접속할 수 있는 포트를 배정한다.
5. **결과 전달** — Pod 이름과 포트를 admin_be에 보내고, admin_be가 사용자에게 안내 메일을 보낸다.

만료되면 Pod와 포트를 지운 뒤 계정과 홈 디렉터리를 정리한다.

3. config-server가 없을 때 하던 일

이 시스템이 없던 때에는 관리자가 서버에 접속해 계정·컨테이너·포트를 모두 명령어로 처리했다.

- 신청이 몰리면 **관리자 한 사람이 병목**이 된다.
- UID 중복·포트 충돌 같은 **수작업 실수**가 발생한다.
- 만료된 자원을 사람이 기억해 삭제해야 하므로 **GPU가 방치**될 수 있다.

config-server는 이 작업을 API로 처리한다. 계정, GPU 노드, 포트가 정해지는 방법은 **시스템 아키텍처**에 있다.

4. 원본 인프라 저장소의 구성

아래 디렉터리는 이 위키가 아닌 `CSID-DGU/admin_infra` 저장소에 있다.

디렉터리	내용
<code>config-server/</code>	Flask API 서버와 Helm 차트
<code>infra-sql/</code>	NodePort 기록을 저장하는 MySQL 배포 파일
<code>pvc-image-chart/</code>	사용자 이미지 파일을 저장하는 PVC 차트

5. 서버 환경 요약

항목	값
배포 위치	FARM Kubernetes, namespace <code>ailab-infra</code>
외부 접근	NodePort 30082 (Service <code>containerssh-config-service</code>)
Swagger UI	<code>http://210.94.179.18:30082/apidocs/</code>
전용 DB (config-server가 관리)	infra-mysql <code>pod_port_db</code> — 데이터베이스

6. 필요한 문서 찾기

상황	추천 페이지
처음 작업한다	이 페이지 → 처음 작업할 때
계정, Pod, GPU 노드, 포트가 어떻게 정해지는지 본다	시스템 아키텍처
API 요청과 응답이 필요하다	API 레퍼런스
점검하거나 장애를 처리한다	운영 매뉴얼
Helm 값이나 차트를 바꾼다	Helm 차트 레퍼런스

시스템 아키텍처

이 문서에서 다루는 내용

- `admin_infra`의 config-server가 계정, 홈 디렉터리, Pod, NodePort를 어떤 순서로 만드는지
- 어떤 정보가 `admin_be`, infra-mysql, `/kube_share`, NAS에 나뉘어 저장되는지
- 계정 생성, Pod 생성, 삭제 흐름에서 어디서 실패할 수 있는지

빠른 이동

필요할 때	문서
실제 운영 명령과 점검 순서를 본다	운영 매뉴얼
요청 형식과 응답을 확인한다	API 레퍼런스

필요할 때	문서
Kerberos 준비 단계를 자세히 본다	kdc-setup

1. 서버마다 하는 일

신청을 받고 승인하는 일, Pod와 계정을 만드는 일, 파일과 인증을 제공하는 일은 서로 다른 서버가 맡는다.

하는 일	서버 또는 서비스	설명
신청과 승인	<code>admin_fe</code> , <code>admin_be</code>	신청을 받고 승인 결과와 사용자 설정을 저장한다.
Pod와 계정 만들기	<code>config-server</code>	계정, 홈 디렉터리, Pod, NodePort Service를 만들거나 지운다.
Pod를 만드는 데 필요한 서비스	Kubernetes, NAS, KDC, Prometheus	컨테이너를 실행하고, 파일을 저장하고, Kerberos 인증과 GPU 노드 정보를 제공한다.

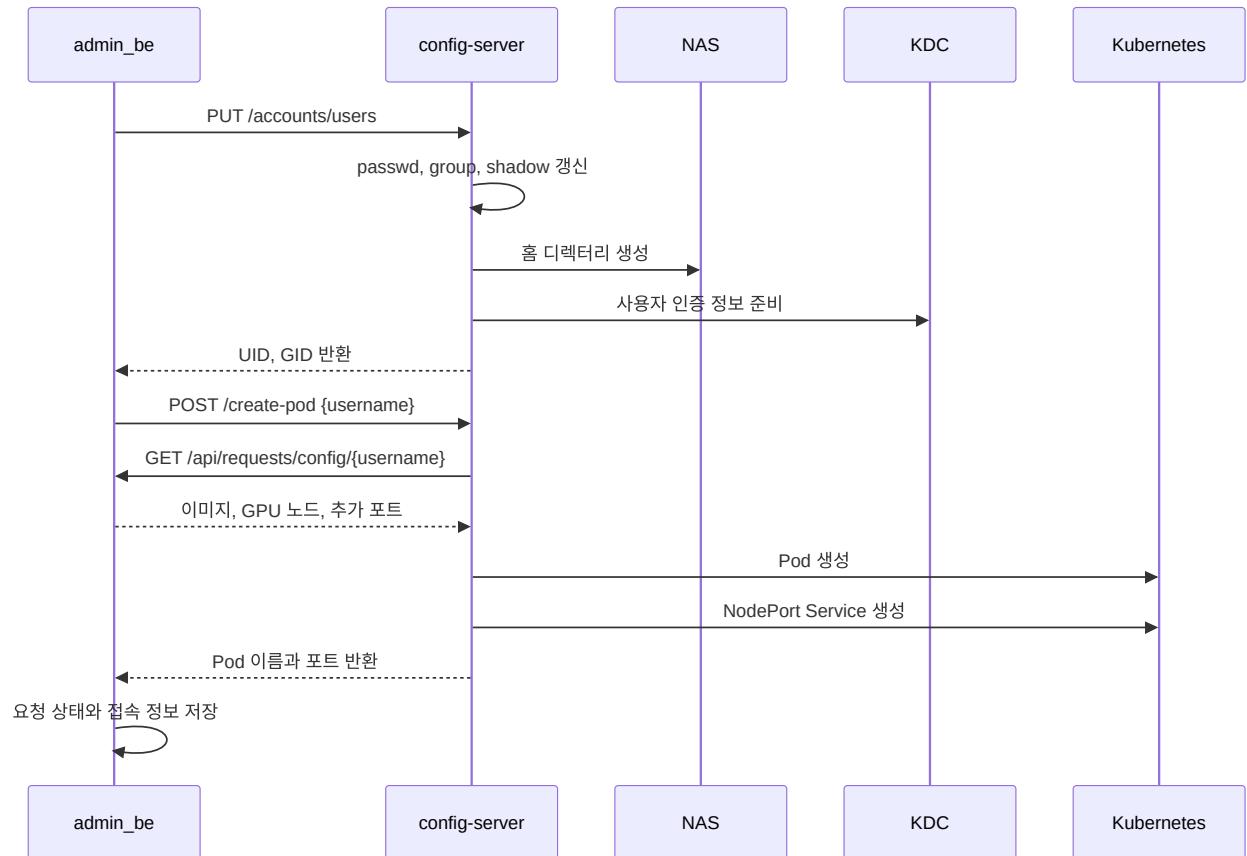
중요한 정보는 다음 위치에 저장한다.

정보	저장 위치	사용하는 곳
신청 상태와 사용자 설정	<code>admin_be</code> DB	<code>admin_be</code> , <code>config-server</code>
NodePort 배정	<code>infra-mysql</code> 의 <code>nodeport_allocations</code>	<code>config-server</code>
UID, GID, 그룹	<code>/kube_share</code> 의 계정 파일	<code>config-server</code> , 사용자 Pod
사용자 홈과 이미지 저장소	NAS	Kubernetes 노드, <code>config-server</code>

`admin_be`는 어떤 사용자에게 Pod를 만들지 정한다. `config-server`는 그 요청을 받아 Pod, 홈 디렉터리, 접속 포트를 만든다. Pod를 만들 때 필요한 이미지, GPU 노드, 추가 포트는 `admin_be`에서 다시 받는다.

2. 승인 후 사용자 환경을 만드는 순서

승인이 끝나면 `admin_be`는 먼저 계정을 만들고, 그다음 Pod를 만든다.



2.1 계정과 홈 디렉터리

`PUT /accounts/users` 는 `/kube_share` 의 `passwd` , `group` , `shadow` 파일에 새 사용자 정보를 쓴다. 새 UID와 GID를 정하는 방법은 [계정 정보](#)에서 확인한다.

홈 디렉터리는 Pod보다 먼저 NAS에 만든다. NFS의 `root_squash` 설정 때문에 Pod 안의 root 사용자는 NAS에 홈 디렉터리를 만들 수 없다. config-server가 NAS에 SSH로 접속해 디렉터리를 만들고, 새 UID와 GID로 소유자를 지정한 뒤 권한을 `700` 으로 설정한다. 홈 생성이 실패하면 계정 파일에 쓴 내용도 되돌리고 계정 생성 요청을 실패로 끝낸다.

2.2 사용자 설정과 Pod 생성

`POST /create-pod` 의 입력은 사용자 이름이다. config-server는 admin_be 의 `GET /api/requests/config/{username}` 을 호출해 다음 값을 가져온다.

- 사용할 이미지
- 사용할 GPU 수와 후보 노드
- 기본 포트 외에 열 포트

config-server는 받은 값으로 Pod를 만든다. 추가 포트에 내부 포트 `6080` 이 있거나 용도가 `novnc` , `vnc` 이면 Pod 환경 변수에 `ENABLE_VNC=true` 도 넣는다. Pod가 접속을 받을 준비가 된 뒤에 NodePort Service를 만들고, Pod 이름과 NodePort 목록을 admin_be 에 돌려준다.

2.3 Pod와 계정 삭제

사용자 환경을 지울 때는 Pod 관련 작업과 계정 관련 작업을 나누어 처리한다.

구분	API	정리 대상
Pod 관련 작업	<code>POST /delete-pod</code>	Pod, NodePort Service, NodePort 배정 기록
계정 관련 작업	<code>DELETE /accounts/users/{username}</code>	계정 파일, NAS 홈, Kerberos 관련 파일

두 요청을 모두 처리해야 계정과 Pod가 함께 삭제된다. 요청 형식과 응답은 [API 레퍼런스](#)에서 확인한다.

3. GPU 노드와 포트 고르는 방식

3.1 노드 선택

config-server는 `admin_be`가 전달한 `gpu_nodes` 목록에서 Pod를 올릴 GPU 노드를 고른다. 노드 이름은 소문자로 바꾸고, 목록에 들어 있던 순서대로 비교한다. `gpu_nodes`가 비어 있으면 Kubernetes에서 준비가 끝난 워커 노드를 가져온다. control-plane 노드에 붙은 `NoSchedule` 표시는 관리용 노드라는 뜻이므로 후보에서 뺀다.

지표	사용 방식
<code>DCGM_FI_DEV_GPU_UTIL</code>	노드 GPU 사용률 평균을 그대로 사용한다.
<code>DCGM_FI_DEV_FB_USED</code>	GPU 메모리 사용량 평균을 GiB 단위로 환산한다.
<code>DCGM_FI_DEV_GPU_TEMP</code>	GPU 온도 평균을 보조 값으로 사용한다.

$$\text{score} = \text{avg}(\text{GPU_UTIL}) + \text{avg}(\text{FB_USED}) / 1024 + \text{avg}(\text{GPU_TEMP}) / 100$$

점수가 가장 낮은 노드를 고른다. 점수가 같으면 후보 목록에서 앞에 있던 노드를 고른다.

각 지표 조회에는 `or vector(0)`이 들어 있다. 어떤 지표가 없으면 그 값은 0으로 계산한다. Prometheus에 연결하지 못했거나 결과가 비어 있으면 그 노드는 고르지 않는다. 모든 후보를 조회하지 못하면 Pod를 만들지 않는다.

고른 노드가 `gpu_nodes` 목록에 있으면 그 항목의 CPU, 메모리, GPU 개수를 Pod 설정에 넣는다. `gpu_nodes` 없이 워커 노드를 찾은 경우에는 config-server에 설정된 기본값을 사용한다.

3.2 NodePort 배정

사용자 Pod에는 SSH(22), Jupyter(8888), `admin_be`가 전달한 추가 포트마다 NodePort Service를 하나씩 만든다. 추가 포트는 받은 순서대로 SSH와 Jupyter 뒤에 붙인다. 어떤 Pod에 어떤 NodePort를 줬는지는 `infra-mysql`의 `nodeport_allocations` 테이블에 적는다.

포트를 고르기 전에는 MySQL 기록과 Kubernetes Service를 비교한다. 이 비교는 마지막 시도 뒤 5분이 지나야 다시 실행되며, 성공하지 못한 시도도 5분 간격에 포함한다. `app=ai lab-nodeport` 라벨과 같은 `pod_name`을 가진 Service가 Kubernetes에 하나도 없고 MySQL에만 남아 있으면 그 Pod의 포트 기록을 지운다. 이 비교는 포트별이 아니라 Pod별로 한다.

포트를 고르고 MySQL에 기록하는 일은 한 번의 DB 작업으로 처리한다. 중간에 실패하면 새 기록은 저장하지 않는다.

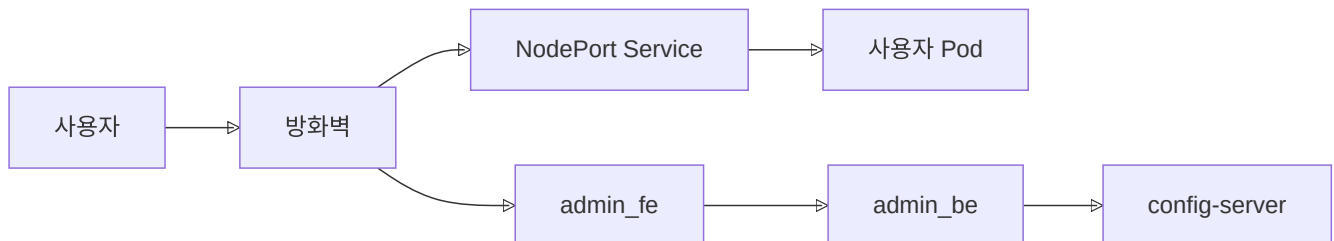
1. `nodeport_allocations`의 포트 기록을 잠가, 동시에 들어온 요청이 같은 번호를 고르지 못하게 한다.
2. MySQL에 적힌 포트와 클러스터 전체 Service가 이미 쓰는 NodePort를 모은다. 클러스터 전체 Service 조회가 실패하면 오류를 로그에 남기고 MySQL 기록만으로 계속 진행한다. 이 경우 다른 Service가 이미 쓰는 포트를 고를 수 있고, 뒤의 Service 생성 단계가 실패할 수 있다.
3. 30000부터 32767까지 오름차순으로 훑어 빈 포트를 필요한 개수만큼 고른다.
4. 고른 포트를 `username`, `pod_name`, `node_name`, 내부 포트, 용도와 함께 MySQL에 저장한다.
5. Pod가 Ready 상태가 되면 기록된 포트마다 NodePort Service를 만든다.

Pod 또는 Service 생성에 실패하면 해당 Pod의 Service, NodePort 배정 행, 생성된 Pod를 정리한다.

구분	저장 위치	의미
요청 포트	<code>admin_be</code> DB의 <code>port_requests</code>	사용자가 요청한 컨테이너 내부 포트
확정 포트	<code>admin_be</code> DB의 <code>pod_external_ports</code>	실제 NodePort와 내부 포트의 연결 정보
배정 기록	<code>infra-mysql</code> 의 <code>nodeport_allocations</code>	Pod와 NodePort Service를 정리할 때 사용하는 기록

3.3 외부 접근 경로

외부 사용자는 방화벽을 거쳐 `admin_fe` 또는 사용자 Pod의 NodePort Service에 접속한다. 방화벽은 외부 포트를 NodePort로 매핑하므로 외부 포트와 NodePort 번호가 같을 필요는 없다. 외부 접속이 되지 않으면 해당 Service의 NodePort와 pfSense 매핑 대상이 같은지 확인한다.



4. 계정, 홈 디렉터리, 인증

4.1 계정 정보 {#account-information}

UID와 GID는 `/kube_share`의 계정 파일에 적힌 값을 사용한다. NAS 파일의 소유자도 이 숫자로 표시되므로, UID와 GID를 다른 데이터베이스에 따로 저장하지 않는다.

계정을 추가할 때는 `passwd` 파일을 잠가, 동시에 들어온 요청이 같은 UID를 고르지 못하게 한다. `/home/`을 쓰는 사용자 중 UID가 20000 이상인 값 가운데 가장 큰 값에 1을 더해 새 UID 후보를 만든다. 이 번호가 시스템 계정을 포함해 이미 쓰이고 있으면, 비어 있는 번호가 나올 때까지 1씩 올린다. 요청에 UID가 들어 있어도 config-server는 사용하지 않고 이 방식으로 정한 값을 반환한다.

새 사용자의 기본 GID는 새 UID와 같은 번호로 정한다. 기본 그룹 이름은 요청의 `primary_group_name`을 쓰고, 이 값이 없으면 사용자 이름을 쓴다. 추가 그룹은 요청에 들어 있는 `{name, gid}` 목록으로 처리한다. 같은 GID의 그룹이 이미 있으면 사용자를 그 그룹에 넣고, 없으면 요청에 적힌 GID로 그룹을 새로 만든다. 추가 그룹의 GID는 config-server가 새

로 정하지 않는다.

`passwd`에 사용자 항목을 쓴 뒤 기본 그룹과 추가 그룹을 `group`에 기록하고, SHA-512 암호 해시를 `shadow`에 기록한다. `sudo` 허용 명령이 설정된 경우에만 사용자별 `sudoers` 파일도 만든다. 이 단계에서 실패하면 앞에서 쓴 계정·그룹 정보를 지운다. NAS 홈 디렉터리의 소유자도 같은 UID와 GID로 바꾼다. Pod를 만들 때 `config-server`는 UID, 기본 GID, 그룹 목록을 환경 변수로 넣는다. 컨테이너가 시작할 때 실행하는 스크립트는 이 값으로 컨테이너 안의 사용자를 만든다. 사용자 Pod는 `/kube_share`를 직접 마운트하지 않는다.

4.2 홈 디렉터리와 이미지 저장소

사용자 홈은 NAS의 NFS 공유를 노드에 마운트한 뒤 Pod의 `/home`에 연결한다. 사용자마다 PVC를 만들지는 않는다. 홈 디렉터리의 소유자와 권한으로 사용자별 접근을 나눈다.

사용자 이미지 tar 파일은 별도의 `pvc-image-store`에 저장한다. `config-server`가 이미지 저장과 로드를 담당하고, Redis에는 이미지 작업의 진행 상태를 기록한다.

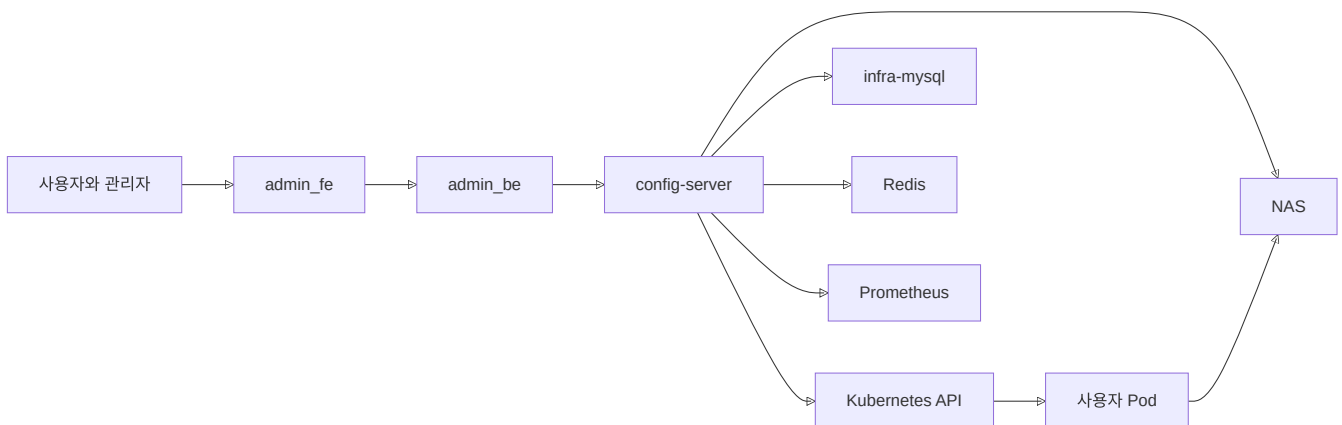
4.3 Kerberos를 사용하는 방식

FARM 환경에서는 NFS 홈을 쓰기 전에 Kerberos 인증을 준비한다. 계정을 만들 때 `config-server`는 사용자 principal을 만들고 keytab을 Kubernetes Secret에 저장한다. Pod를 만들 때는 선택된 노드에 keytab과 갱신용 설정을 두고 ccache를 만든다. Pod에는 keytab을 넣지 않고 ccache만 연결한다.

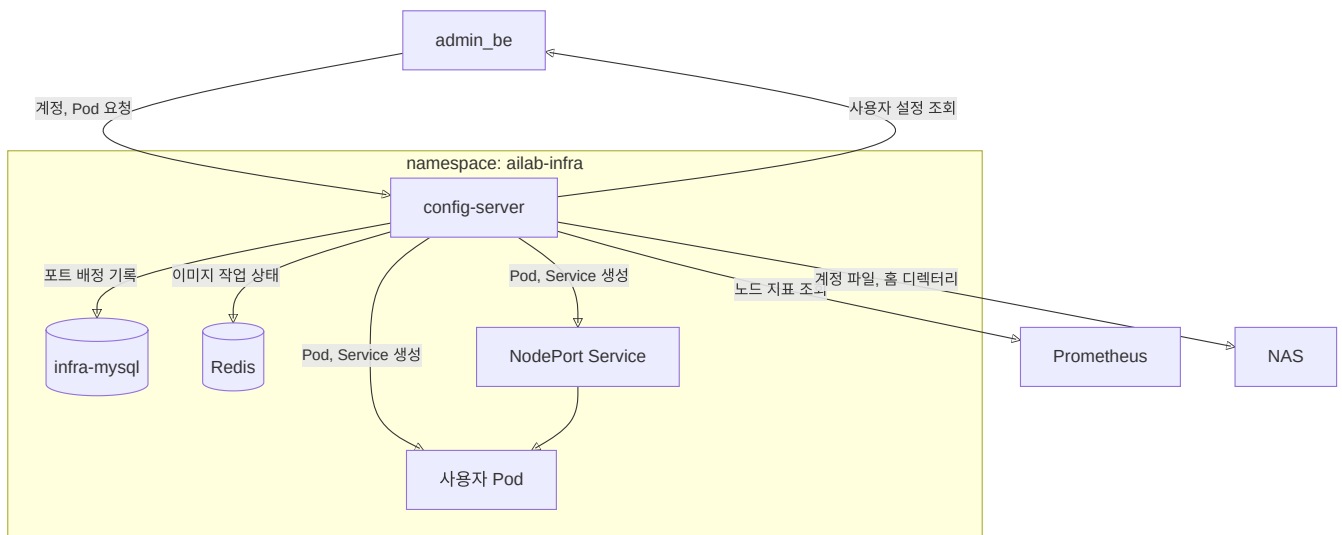
Kerberos 서버와 설정 값은 `kdc-setup`에서 확인한다.

5. 서버와 서비스가 연결되는 모습

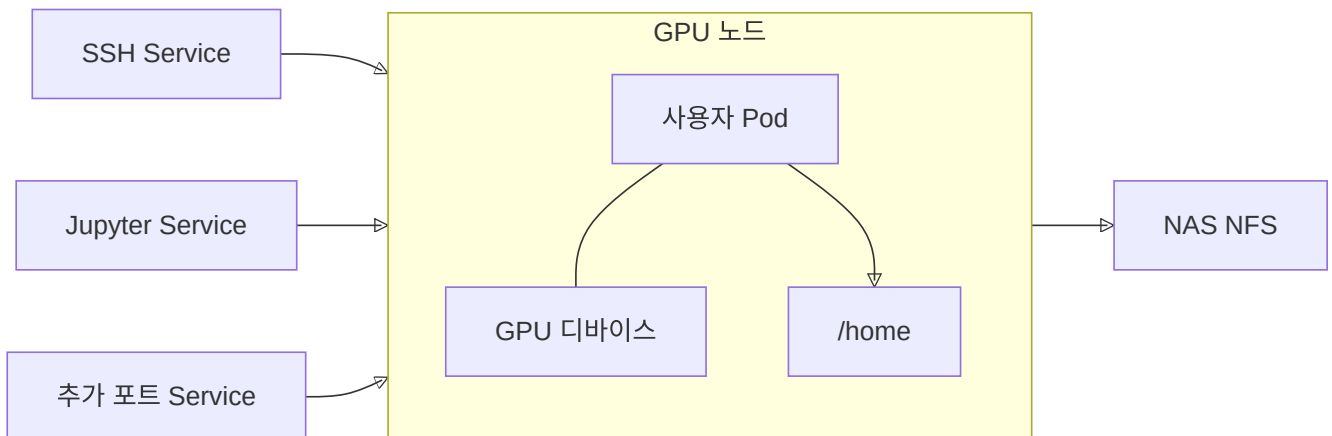
5.1 전체 연결



5.2 config-server가 연결하는 서비스



5.3 사용자 Pod



사용자 Pod는 접속용 Service, GPU 디바이스, 홈 디렉터리를 사용한다. UID와 GID, 그룹 정보는 Pod를 만들 때 넣고, 홈 디렉터리는 노드의 NFS 마운트를 통해 연결한다.

기초 개념

이 문서에서 다루는 내용

- 시스템 아키텍처에 나오는 UID/GID, 계정 파일, 권한, Kubernetes, NFS, NodePort 기본 개념
- config-server 문서를 읽을 때 자주 헷갈리는 배경 용어
- 운영 절차가 아니라 개념 설명만 먼저 보고 싶을 때 필요한 내용

빠른 이동

필요할 때	문서
실제 생성 순서를 본다	시스템 아키텍처
배포와 장애 대응을 본다	운영 매뉴얼
포트 배정 기록과 DB 구조를 본다	데이터베이스

1. 리눅스 계정과 파일 권한

1.1 UID와 GID

리눅스는 사용자와 그룹을 이름뿐 아니라 숫자로도 구분한다. 사용자 번호는 UID, 그룹 번호는 GID이다.

파일에도 사용자 이름이 아닌 UID와 GID가 소유권 정보로 기록된다. 예를 들어 파일의 소유자가 `dongmin`으로 보이더라도, 실제 디스크에는 `UID 2041`과 같은 숫자가 저장된다.

이름과 숫자의 대응 관계는 다음 파일에서 관리한다.

파일	역할
<code>passwd</code>	사용자 이름, UID, 기본 GID를 기록한다.
<code>group</code>	그룹 이름, GID, 그룹 구성원을 기록한다.

UID를 정하는 곳

NFS에 저장된 사용자 홈 디렉터리 역시 UID 숫자를 기준으로 소유권을 판단한다. 따라서 동일한 NAS를 사용하는 FARM 서버와 Kubernetes Pod는 반드시 동일한 UID 대응 관계를 사용해야 한다.

서버마다 같은 UID를 다른 사용자에게 배정하면, 특정 사용자가 다른 사용자의 홈 디렉터리를 자신의 파일처럼 읽거나 수정할 수 있다.

이를 막기 위해 config-server가 `/kube_share/passwd`를 계정 정보의 기준 파일로 사용하고 UID도 여기서 정한다.

1.2 passwd, group, shadow, sudoers

파일	저장 내용	주의사항
<code>passwd</code>	사용자 이름, UID/GID, 홈 경로, 로그인 셸	암호 정보는 없으며 누구나 읽을 수 있다.
<code>group</code>	그룹 이름, GID, 그룹 멤버	그룹 권한 관리에 사용한다.

파일	저장 내용	주의사항
shadow	사용자 암호 해시	root만 읽을 수 있다.
sudoers	sudo 사용 권한	문법 오류가 발생하면 sudo 전체가 동작하지 않을 수 있다.

일반적인 리눅스 환경에서는 `/etc/passwd`, `/etc/group` 등을 `useradd` 와 같은 명령으로 관리한다.

계정 파일을 쓰는 방법

여기서는 config-server가 NFS의 `/kube_share` 에 저장된 계정 파일을 직접 수정해 UID/GID 배정 기록으로 사용한다.

사용자 Pod에는 이 파일들을 직접 마운트하지 않는다. 대신 다음 순서로 사용자를 생성한다.

1. config-server가 `/kube_share/passwd` 에서 UID/GID를 조회한다.
2. 조회한 값을 Pod 환경변수로 주입한다.
3. 컨테이너 entrypoint가 전달받은 UID/GID로 내부 사용자를 생성한다.

계정 파일을 컨테이너의 `/etc` 에 직접 마운트하던 방식은 더 이상 쓰지 않는다.

1.3 chmod 700

`chmod` 의 세 숫자는 차례대로 다음 권한을 의미한다.

1. 파일 소유자
2. 소유 그룹
3. 기타 사용자

각 권한 값은 다음 숫자의 합으로 표현한다.

| 권한 | 값 | | ----- | -: | | 읽기 | 4 | | 쓰기 | 2 | | 실행 또는 디렉터리 진입 | 1 |

따라서 `chmod 700` 은 다음과 같은 권한 설정이다.

대상	권한
소유자	읽기, 쓰기, 진입
그룹	권한 없음
기타 사용자	권한 없음

홈 디렉터리 권한

모든 사용자 홈 디렉터리는 `chmod 700` 으로 생성한다.

현재는 사용자별 PVC를 만들지 않는다. 사용자 홈 사이의 접근은 UID 소유권과 `chmod 700` 권한으로 막는다.

2. NFS와 NAS

2.1 NFS와 NAS

NFS(Network File System)는 다른 서버의 디렉터리를 내 서버의 디렉터리처럼 쓰게 하는 방식이다.

서버가 특정 디렉터리를 외부에 공개하면(`export`), 클라이언트는 해당 경로를 자신의 파일시스템에 마운트한다. 이후 `ls`, `cat`, 파일 저장 등의 작업은 로컬 파일처럼 보이지만 실제로는 네트워크 통신을 통해 처리된다.

NAS(Network Attached Storage)는 이 NFS 저장소를 제공하는 장비다.

NAS에 저장하는 데이터

이 시스템의 NAS에는 다음 데이터가 저장된다.

- 사용자 홈 디렉터리
- `/kube_share` 계정 파일
- `infra-mysql` 데이터

따라서 NAS에 문제가 생기면 사용자 파일, 계정 관리, 데이터베이스가 함께 영향을 받는다. 이 저장소를 대신할 장비가 현재 구성되어 있지 않으므로 NAS 장애는 전체 서비스에 영향을 준다.

2.2 root_squash

`root_squash`는 NFS의 기본 보안 기능이다.

NFS 클라이언트의 `root` 사용자가 공유 디렉터리에 접근하면, NFS 서버는 그 요청을 `nobody` 같은 익명 사용자 권한으로 처리한다.

그래서 한 클라이언트 서버에서 `root` 권한을 얻더라도 NAS 파일 전체를 마음대로 바꾸기 어렵다.

홈 디렉터리를 만드는 방법

Pod 안의 프로세스가 `root`로 실행되더라도 NFS 위에서는 `root` 권한을 사용할 수 없다. 따라서 Pod가 NFS에 사용자 홈 디렉터리를 직접 생성하거나 소유권을 변경할 수 없다.

`config-server`는 Pod를 생성하기 전에 NAS에 SSH로 접속하여 다음 작업을 수행한다.

```
mkdir <사용자 홈>
chown <UID>:<GID> <사용자 홈>
chmod 700 <사용자 홈>
```

이 과정이 실패하면 Pod 생성도 실패하도록 처리한다.

2.3 NFS 파일 락

파일 잠금은 여러 프로세스가 같은 파일을 동시에 바꾸지 못하게 하는 장치다.

NFS 위의 파일 잠금은 네트워크와 별도 잠금 서비스에 의존하므로, 이 시스템에서는 계정 파일에 직접 적용하지 않는다.

현재 잠금 방식

config-server의 `LockedFile`은 NFS 파일을 수정하기 전에 config-server Pod 안의 `/tmp`에 잠금 파일을 만든다.

```
/tmp/cssh_lock*
```

현재 config-server는 한 개만 실행한다. 이 전제에서 같은 서버 안의 요청은 이 잠금으로 순서대로 처리된다.

config-server를 여러 개로 늘리면

config-server를 여러 개로 늘리면 이 방식만으로는 부족하다. 각 인스턴스의 `/tmp` 잠금 파일은 서로 보이지 않기 때문이다.

3. Kubernetes

3.1 클러스터와 노드

Kubernetes는 여러 서버를 하나의 컨테이너 실행 환경으로 묶어 관리하는 오케스트레이션 시스템이다.

- 클러스터는 여러 노드로 구성된 전체 실행 환경이다.
- 노드는 Pod가 실제로 실행되는 개별 서버이다.

Kubernetes에는 필요한 Pod와 Service의 내용을 API로 등록한다. Kubernetes는 등록된 내용과 실제 상태가 다르면 다시 맞추려고 한다.

Pod와 Service 생성

현재는 config-server가 Pod와 Service 정보를 Kubernetes API에 보내고, Kubernetes가 이를 실행한다.

3.2 Pod

Pod는 Kubernetes가 컨테이너를 실행하는 최소 단위이다.

Pod 스펙에는 다음과 같은 정보가 포함된다.

- 컨테이너 이미지
- 환경변수
- 볼륨 마운트

- GPU 개수
- 실행할 노드
- 포트 설정

사용자 Pod 규칙

여기서는 사용자 한 명마다 Pod 하나를 만든다.

config-server는 Kubernetes 클라이언트 라이브러리를 이용해 사용자 Pod 스펙을 생성한다.

보통은 Kubernetes가 실행 노드를 고르지만, 이 시스템은 config-server가 Prometheus GPU 지표로 먼저 노드를 고른 뒤 Pod에 그 노드를 지정한다.

3.3 Service와 NodePort

Pod의 IP는 Pod가 재생성될 때 변경될 수 있다. Service는 이러한 Pod에 고정된 네트워크 접근 지점을 제공한다.

NodePort 타입 Service는 모든 Kubernetes 노드의 특정 포트를 열고, 해당 포트로 들어온 요청을 대상 Pod로 전달한다.

기본 NodePort 범위는 다음과 같다.

```
30000~32767
```

사용자 Pod에 여는 포트

사용자 Pod의 각 포트마다 별도의 NodePort Service를 생성한다.

- SSH는 컨테이너 포트 22를 사용한다.
- Jupyter는 컨테이너 포트 8888을 사용한다.
- 사용자가 추가로 신청한 포트도 별도 Service로 생성한다.

config-server가 배정한 포트는 infra-mysql의 `nodeport_allocations` 테이블에 기록한다. 포트를 고를 때는 이 테이블뿐 아니라 클러스터 전체 Service의 NodePort도 함께 확인한다.

NodePort와 방화벽 매핑

Kubernetes의 NodePort는 클러스터 안에서 쓰는 포트이고, pfSense는 외부 포트를 이 NodePort로 전달한다. 외부 포트와 NodePort 번호는 같을 필요가 없다. 외부에서 접속하지 못하면 Service의 NodePort와 pfSense의 전달 대상이 같은지 확인한다.

3.4 Namespace

Namespace는 클러스터 리소스를 용도나 팀별로 구분하는 논리적 공간이다.

이 시스템의 주요 Namespace는 다음과 같다.

Namespace	용도
ailab-frontend	사용자 화면
default	admin_be
ailab-infra	config-server와 사용자 Pod
monitoring	Prometheus와 Grafana

리소스를 조회할 때는 Namespace를 정확히 지정해야 한다.

```
kubectl get pods -n ailab-infra
```

Namespace를 생략하면 다른 공간의 리소스를 조회하여 장애 원인을 놓칠 수 있다.

3.5 볼륨, hostPath, subPath

hostPath

hostPath는 노드 호스트의 특정 파일이나 디렉터리를 컨테이너에 직접 마운트하는 방식이다.

이 시스템에서는 다음 항목에 사용한다.

- 노드에 마운트된 NFS 홈 경로
- GPU 디바이스 `/dev/nvidia*`

subPath

subPath는 볼륨 전체가 아니라 볼륨 안의 특정 파일이나 디렉터리만 골라 마운트하는 옵션이다.

과거에는 `/kube_share/passwd`와 `shadow`를 컨테이너의 `/etc/passwd`, `/etc/shadow`에 직접 마운트했다. 현재 Pod 스펙에는 이 마운트가 없고, config-server가 UID/GID를 환경변수로 전달하면 컨테이너 시작 스크립트가 내부 사용자를 만든다.

코드 주석과 예전 배포 기록에는 **subPath** 방식이 남아 있을 수 있다. 현재 Pod 스펙에는 이 마운트가 없다.

3.6 imagePullPolicy와 Ready

imagePullPolicy

Pod를 생성할 때 컨테이너 이미지를 어떻게 가져올지 결정하는 정책이다.

값	동작
Always	Pod 생성 시마다 이미지를 내려받는다.

값	동작
<code>IfExists</code>	노드에 이미지가 없을 때만 내려받는다.

`Always` 는 최신 이미지를 보장하지만 시작 시간이 길어질 수 있다.

Ready

컨테이너 프로세스가 시작된 상태와 실제 요청을 받을 준비가 된 상태는 다르다.

`config-server`는 다음 순서로 Pod를 생성한다.

1. Pod 생성 요청을 보낸다.
2. Pod가 Ready가 될 때까지 상태를 확인한다.
3. Ready가 되면 NodePort Service를 만든다.
4. 생성 결과를 응답한다.

NFS 마운트가 실패하여 `FailedMount` 상태가 되면 Pod는 Ready가 되지 않으며, `config-server`의 대기는 타임아웃으로 종료된다.

3.7 RBAC와 ServiceAccount

RBAC(Role-Based Access Control)는 Kubernetes 리소스를 누가 다룰 수 있는지 정하는 권한 설정이다.

ServiceAccount는 사람이 아닌 애플리케이션이나 Pod가 사용하는 Kubernetes 내부 신원이다.

config-server의 Kubernetes 권한

`config-server`는 자신의 ServiceAccount 권한으로 Kubernetes API를 호출하여 다음 작업을 수행한다.

- Pod 생성과 삭제
- Service 생성과 삭제
- Pod 상태 조회

관련 ServiceAccount, Role, RoleBinding은 Helm 차트 배포 과정에서 함께 생성한다.

3.8 Helm

Helm은 여러 Kubernetes YAML 파일과 설정값을 한 묶음으로 관리하는 도구다.

Deployment, Service, RBAC 등의 정의를 템플릿으로 구성하고, 환경별 값만 변경해 배포할 수 있다.

`config-server`는 `Chart/` 디렉터리의 Helm 차트로 배포하며, 다음 명령이 배포 절차를 감싼다.

```
make deploy
```

4. 모니터링

4.1 Prometheus와 Pull 모델

Prometheus는 시스템 지표를 수집하고 저장하는 모니터링 시스템이다.

Prometheus는 각 서버가 데이터를 보내는 push 방식이 아니라, Prometheus가 각 대상의 HTTP 엔드포인트를 주기적으로 조회하는 **pull 방식**을 사용한다.

Prometheus 형식으로 지표를 제공하는 프로그램을 exporter라고 한다.

4.2 DCGM Exporter

DCGM은 NVIDIA GPU의 상태를 수집하는 도구이며, DCGM exporter는 GPU 지표를 Prometheus 형식으로 제공한다.

config-server는 다음 지표를 노드 선택 점수에 사용한다.

```
DCGM_FI_DEV_GPU_UTIL
DCGM_FI_DEV_FB_USED
DCGM_FI_DEV_GPU_TEMP
```

GPU 지표가 없을 때

DCGM exporter가 중단되면 해당 노드의 GPU 지표가 Prometheus에서 사라진다.

현재 쿼리는 값이 없을 때 다음 표현으로 0을 반환한다.

```
... or vector(0)
```

이 때문에 지표가 없는 노드를 사용률이 0인 노드, 즉 가장 한가한 노드로 잘못 판단할 수 있다.

지표 없음과 실제 사용률 0은 구분해야 한다.

4.3 PromQL 기본 문법

PromQL은 Prometheus에 저장된 지표를 조회하는 질의 언어이다.

예를 들어 farm5의 GPU 평균 사용률은 다음과 같이 조회한다.

```
avg(DCGM_FI_DEV_GPU_UTIL{Hostname="farm5"})
```

주요 문법은 다음과 같다.

표현	의미
<code>metric{label="value"}</code>	특정 레이블을 가진 지표를 조회한다.
<code>avg(...)</code>	여러 GPU 값의 평균을 계산한다.
<code>or vector(0)</code>	조회 결과가 없을 경우 0을 반환한다.

`or vector(0)` 은 계산을 단순화하지만, 장애로 인한 지표 부재까지 정상적인 0으로 처리한다는 문제가 있다.

4.4 Grafana

Grafana는 Prometheus에 저장된 지표를 대시보드 형태로 보여주는 시각화 도구이다.

관리자는 다음 주소에서 노드와 GPU 상태를 확인할 수 있다.

```
http://210.94.179.18:30080
```

PromQL을 직접 실행하려면 Prometheus에 port-forward로 접속한다.

5. 네트워크와 방화벽

5.1 공인 IP, 사설 IP, NAT

- **사설 IP**는 내부 네트워크에서만 사용하는 주소이다.
- **공인 IP**는 인터넷에서 직접 접근할 수 있는 주소이다.
- **NAT**는 사설 IP와 공인 IP 사이에서 주소나 포트를 변환하는 기술이다.
- **포트포워딩**은 특정 공인 IP와 포트로 들어온 요청을 내부 서버의 특정 포트로 전달하는 규칙이다.

예시는 다음과 같다.

```
공인 IP:9300 → farm6:30000
```

5.2 pfSense

pfSense는 외부 네트워크와 내부 서버 사이에서 방화벽과 라우터 역할을 한다.

공인 IP	정책
210.94.179.18	사전에 허용한 포트만 통과시킨다.
210.94.179.19	외부 포트를 사용자 Pod의 NodePort로 전달한다.

외부 접속이 안 될 때

NodePort 번호만 보고 외부 접속 주소를 추정하지 않는다. Service의 NodePort, pfSense가 전달하는 대상, 사용자에게 안내한 외부 주소를 함께 확인한다.

6. 저장소와 동시성

6.1 트랜잭션과 SELECT ... FOR UPDATE

트랜잭션은 여러 SQL 작업을 하나의 작업 단위로 묶는 기능이다.

트랜잭션 내부의 작업은 모두 성공하거나 모두 취소된다.

`SELECT ... FOR UPDATE` 는 조회한 행에 잠금을 걸어, 현재 트랜잭션이 끝날 때까지 다른 트랜잭션이 해당 행을 수정하지 못하게 한다.

NodePort를 배정하는 순서

NodePort 배정은 다음 순서로 처리한다.

1. 트랜잭션을 시작한다.
2. 사용 중인 포트 목록을 `FOR UPDATE` 로 조회한다.
3. 빈 포트를 선택한다.
4. 배정 정보를 insert한다.
5. 트랜잭션을 commit한다.

두 개의 Pod 생성 요청이 동시에 들어오더라도 같은 포트가 중복 배정되는 것을 방지한다.

6.2 Redis

Redis는 메모리 기반 키-값 저장소이다.

이 시스템에서는 사용자 이미지 저장과 로드 작업의 진행 상태만 기록한다.

```
img:<username>
```

이미지 commit 또는 save 작업이 오래 걸리므로, Redis는 현재 작업 단계를 API에서 조회할 수 있도록 상태판 역할을 한다.

6.3 정보별 기준 저장 위치

같은 정보가 여러 곳에 있으면 값이 달라질 수 있다. 그래서 정보마다 어느 곳의 값을 기준으로 사용할지 정해 둔다.

이 시스템에서는 다음 위치의 값을 기준으로 사용한다.

정보	기준으로 보는 곳
사용자 UID/GID	/kube_share/passwd
요청 상태	admin_be DB의 requests 테이블
NodePort 배정	infra-mysql의 nodeport_allocations 테이블

admin_be가 config-server에 UID를 전달하더라도 config-server는 전달값을 신뢰하지 않고 /kube_share/passwd 를 기준으로 동작한다.

반대로 요청의 PENDING, FULFILLED, DELETED 상태는 admin_be가 관리하며 config-server는 해당 상태를 저장하지 않는다.

6.4 삭제 표시(soft delete)

삭제 표시는 데이터베이스 행을 지우지 않고 삭제되었다는 상태만 기록하는 방식이다.

admin_be의 요청 상태 DELETED 가 이에 해당한다.

주요 장점은 다음과 같다.

- 삭제 이력을 보존한다.
- 운영 감사와 장애 추적이 가능하다.
- 필요할 경우 데이터를 복구할 수 있다.

7. 실행 환경

7.1 Flask와 Gunicorn

Flask는 config-server에 사용한 Python 웹 프레임워크이다.

Flask의 내장 서버는 개발용이므로 운영 환경에서는 Gunicorn을 사용한다. Gunicorn은 여러 워커 프로세스로 Flask 애플리케이션을 실행하여 동시 요청을 처리한다.

config-server의 로컬 파일 락은 Gunicorn의 여러 워커가 동시에 계정 파일을 수정하는 상황을 제어하는 역할도 한다.

7.2 WAS와 Spring Boot

WAS(Web Application Server)는 웹 요청을 받아 비즈니스 로직을 실행하는 서버를 의미한다.

이 시스템에서는 Spring Boot로 작성한 `admin_be`가 WAS 역할을 수행한다.

운영 문서나 대화에서 “WAS”라고 표현하면 일반적으로 `admin_be`를 의미한다.

7.3 Swagger

Swagger는 API 목록과 요청 형식을 웹 페이지로 제공하고, 브라우저에서 직접 API를 호출할 수 있게 하는 도구이다.

config-server Swagger 주소는 다음과 같다.

```
http://210.94.179.18:30082/apidocs/
```

Swagger를 사용할 때

config-server API에는 별도의 인증 기능이 없다.

따라서 Swagger의 `Try it out` 기능은 단순한 테스트가 아니라 실제 Kubernetes와 NAS 리소스를 생성·수정·삭제할 수 있는 운영 명령이다.

Swagger 호출은 실제 인프라를 변경하는 작업이므로 운영 권한과 동일하게 취급해야 한다.

핵심 구조 요약

여기서는 다음 방식으로 동작한다.

1. 사용자 식별과 파일 권한의 기준은 UID/GID이다.
2. UID/GID는 `/kube_share/passwd`에 적힌 값을 사용한다.
3. 사용자 홈은 NFS에 저장하며 `chmod 700`으로 다른 사용자의 접근을 막는다.
4. `root_squash` 때문에 사용자 홈은 Pod 생성 전에 NAS에서 미리 생성해야 한다.
5. 사용자 한 명당 하나의 Pod와 여러 NodePort Service를 생성한다.
6. 실행 노드는 config-server가 Prometheus GPU 지표를 바탕으로 선택한다.
7. NodePort는 infra-mysql에서 트랜잭션과 행 잠금으로 배정한다.
8. 외부 접속은 pfSense가 외부 포트를 해당 NodePort로 전달해야 가능하다.

9. NAS, 계정 파일, NodePort DB 등 주요 기준 정보가 손상되면 전체 시스템에 영향을 줄 수 있다.
10. Swagger와 config-server API는 인증이 없으므로 실제 인프라 관리 인터페이스로 취급해야 한다.

데이터베이스

이 문서에서 다루는 내용

- infra-mysql이 어떤 테이블을 가지고 있고 config-server가 어디에 쓰는지
- NodePort를 동시에 안전하게 배정하기 위해 어떤 기록과 잠금을 쓰는지
- DB를 확인하거나 복구할 때 Kubernetes 상태와 함께 무엇을 봐야 하는지

빠른 이동

필요할 때	문서
승인부터 Pod 생성까지 전체 흐름을 본다	시스템 아키텍처
운영 중 포트 충돌이나 잔여 기록을 확인한다	운영 매뉴얼
API 응답과 오류 형식을 확인한다	API 레퍼런스

1. 사용하는 데이터베이스

DB	관리 주체	용도	관련 저장소
infra-mysql pod_port_db	config-server 전용	NodePort 배정 기록과 FARM 노드 Kerberos 정리 재 시도 목록을 저장한다.	이 저장소(admin_infra)
admin_be MySQL	admin_be(W AS)	신청·사용자 정보를 관리한다.	별도 저장소이며, 이 문서에서는 이름만 언급한다.

NodePort는 동시에 두 요청이 들어와도 같은 번호를 두 번 배정하면 안 된다. config-server는 포트를 배정할 때 infra-mysql의 행 잠금을 사용해 이 문제를 막는다.

신청과 사용자 정보는 admin_be MySQL에서 관리한다. 두 시스템은 서로의 DB를 직접 조회하지 않고 API로만 통신한다.

포트 배정 기록은 infra-mysql에 남는다. 다만 실제 Service가 남아 있는지도 Kubernetes에서 함께 확인해야 한다.

2. infra-mysql 배포 형태

infra-mysql은 `infra-sql/` 디렉터리의 매니페스트로 배포한다.

리소스	내용
StatefulSet <code>infra-mysql</code>	<code>mysql:8.0.36</code> 이미지, replicas는 1이고 Namespace는 <code>ailab-infra</code> 이다.
Service <code>infra-mysql</code>	ClusterIP 타입이며 3306 포트를 제공. config-server는 <code>db.host: "infra-mysql"</code> 로 접속한다.
StorageClass <code>sc-mysql</code>	NFS CSI인 <code>nfs.csi.k8s.io</code> 를 사용. NAS export를 백엔드로 사용하며 <code>reclaimPolicy: Retain</code> , <code>nfsvers=3</code> 으로 설정.
PVC <code>mysql-data</code>	StatefulSet의 <code>volumeClaimTemplate</code> 으로 생성되며 용량은 10Gi. <code>/var/lib/mysql</code> 에 마운트.

StatefulSet은 Pod 이름과 스토리지를 고정하는 Kubernetes 워크로드이다. 따라서 infra-mysql Pod의 이름은 항상 `infra-mysql-0` 이다.

계정과 DB 생성 주체

계정과 데이터베이스는 MySQL 공식 이미지의 초기화 동작이 생성한다.

공식 MySQL 이미지는 데이터 디렉터리가 비어 있는 최초 기동 시 다음 환경변수를 읽어 초기화를 수행한다.

환경변수	역할
<code>MYSQL_DATABASE</code>	지정한 데이터베이스를 생성. 이 시스템에서는 <code>pod_port_db</code> .
<code>MYSQL_USER</code>	일반 사용자 계정을 만든다. 이 시스템에서는 <code>pod_port_user</code> 다.
<code>MYSQL_PASSWORD</code>	생성한 사용자 계정의 비밀번호를 설정.

이미지는 생성한 사용자에게 `MYSQL_DATABASE` 로 지정한 데이터베이스의 전체 권한을 부여한다.

즉 `CREATE USER`, `GRANT` 같은 계정 생성과 권한 부여는 MySQL 이미지가 처음 시작할 때 처리한다.

현재 상태는 다음과 같다.

- 배포 매니페스트와 코드에 수동 `GRANT` 구문이 없다.
- 계정과 권한 생성은 MySQL 이미지의 초기화 동작에 위임한다.
- 별도의 init SQL 파일이나 초기 스키마 스크립트가 없다.

config-server는 Kubernetes Secret인 `config-server-db-secret` 에서 비밀번호를 `DB_PASSWORD` 환경변수로 주입받는다. infra-mysql 측 비밀번호는 배포 manifest나 Secret을 `kubectrl` 로 조회한다.

3. 스키마와 DDL 관리

infra-mysql에서 현재 코드가 사용하는 테이블은 두 개다.

테이블	쓰는 곳	용도
<code>nodeport_allocations</code>	<code>main.py</code>	사용자 Pod의 NodePort 배정 기록
<code>krb5_cleanup_pending</code>	<code>main.py</code> , <code>reconcile_krb5.py</code>	FARM 노드에서 실패한 Kerberos 파일 정리를 다음 CronJob 실행 때 다시 시도할 목록

`server_nodes` 와 같은 다른 테이블은 현재 코드에서 사용하지 않는다.

3.1 `nodeport_allocations`

컬럼	역할
<code>username</code>	포트를 소유한 사용자를 기록한다.
<code>pod_name</code>	포트를 사용하는 사용자 Pod 이름이다. 포트 해제와 동기화 작업의 키로 사용한다.
<code>node_name</code>	Pod가 배치된 Kubernetes 노드를 기록한다.
<code>internal_port</code>	컨테이너 내부 포트이다. 예시는 22, 8888 등이다.
<code>node_port</code>	외부 접근을 위해 배정한 NodePort이다. 기본 Kubernetes 범위는 30000~32767이다.
<code>purpose</code>	포트의 용도를 기록한다. 예시는 <code>ssh</code> , <code>jupyter</code> , <code>custom</code> 등이다.

3.2 `krb5_cleanup_pending`

Kerberos를 쓰는 배포에서는 Pod 또는 계정을 지울 때 FARM 노드의 keytab, timer, ccache도 지운다. 이 작업이 실패하면 config-server는 아래 정보를 `krb5_cleanup_pending` 에 기록한다. 30분마다 실행되는 `reconcile_krb5.py` 는 이 기록을 다시 처리하고, 성공하면 행을 지운다.

컬럼	역할
<code>username</code>	정리할 사용자 이름
<code>node_name</code>	정리 작업이 실패한 FARM 노드
<code>failed_at</code>	마지막으로 정리에 실패한 시각

코드의 `ON DUPLICATE KEY UPDATE` 구문상 `username`, `node_name` 조합에는 중복을 막는 키가 있어야 한다. 정확한 기본 키·인덱스·컬럼 타입은 코드만으로 확인할 수 없으므로, 재구축할 때는 기존 DB의 DDL을 사용한다.

3.3 DDL 생성 방식

DDL(Data Definition Language)은 테이블 구조를 생성하거나 변경하는 SQL이다. 대표적인 구문은 `CREATE TABLE`, `ALTER TABLE` 이다.

현재 코드에는 `CREATE TABLE` 구문이 없다.

따라서 DB를 새로 구축하면 `nodeport_allocations` 와 `krb5_cleanup_pending` 을 직접 만들어야 한다. 자동으로 테이블을 만드는 기능은 없다.

위 컬럼 목록은 코드의 `INSERT` 와 `SELECT` 에서 실제로 쓰는 컬럼을 기준으로 작성했다.

자동 증가 ID, 기본 키, 인덱스, 컬럼 타입은 코드만으로 확인할 수 없다. DB를 다시 만들 때는 기존 DB에서 아래 결과를 먼저 가져와 사용한다.

```
SHOW CREATE TABLE nodeport_allocations;
SHOW CREATE TABLE krb5_cleanup_pending;
```

기존 스키마 백업이 없을 때만 위 컬럼 목록을 바탕으로 새 DDL을 작성한다.

재구축용 DDL 실행 절차

```
kubectl exec -it infra-mysql-0 -n ailab-infra -- \
mysql -u pod_port_user -p pod_port_db
```

MySQL 접속 후 두 테이블의 생성 DDL을 실행한다.

```
CREATE TABLE nodeport_allocations (...);
CREATE TABLE krb5_cleanup_pending (...);
```

실제 운영 환경에서는 두 테이블의 `SHOW CREATE TABLE` 결과를 사용해야 한다.

4. Flask에서의 사용 패턴

4.1 접속 방식

config-server의 MySQL 라이브러리는 PyMySQL이다.

`utils.py` 의 `get_db_connection()` 함수가 다음 환경변수를 읽어 DB에 접속한다.

환경변수	역할
<code>DB_HOST</code>	MySQL 호스트이다.
<code>DB_USER</code>	MySQL 사용자이다.

환경변수	역할
DB_PASSWORD	MySQL 비밀번호이다.
DB_NAME	사용할 데이터베이스 이름이다.

포트 배정, 해제, 동기화와 같은 작업 단위마다 새로운 DB 연결을 생성하고, 작업이 끝나면 `finally` 구문에서 연결을 닫는다.

`autocommit=False` 로 동작하므로 SQL 실행 후 명시적으로 `commit()` 을 호출해야 변경 사항이 반영된다.

4.2 포트 배정 트랜잭션

NodePort 배정 트랜잭션은 `main.py` 의 `allocate_nodeports()` 함수가 담당한다.

처리 순서는 다음과 같다.

1. 사용 중인 포트 목록을 `FOR UPDATE` 로 조회한다.

```
SELECT node_port
FROM nodeport_allocations
FOR UPDATE;
```

1. 클러스터 모든 namespace의 Service가 쓰는 NodePort도 조회해 사용 중인 목록에 합친다. 이 조회가 실패하면 오류를 로그에 남기고 MySQL 목록만으로 계속 진행한다. 이때 다른 Service가 이미 쓰는 포트를 고르면 Service 생성 단계에서 실패할 수 있다.
2. NodePort 범위인 30000~32767에서 사용하지 않는 포트를 찾는다.
3. 요청한 포트 개수만큼 `nodeport_allocations` 에 행을 삽입한다.
4. 모든 삽입이 성공하면 `conn.commit()` 으로 트랜잭션을 확정한다.
5. 예외가 발생하면 `conn.rollback()` 으로 전체 작업을 취소한다.

트랜잭션은 여러 SQL 작업을 묶어서 모두 반영하거나, 하나라도 실패하면 모두 취소하는 기능이다.

4.3 SELECT ... FOR UPDATE를 사용하는 이유

두 개의 승인 요청이 거의 동시에 들어오면 두 트랜잭션이 같은 시점에 동일한 빈 포트 목록을 읽을 수 있다.

잠금이 없다면 다음과 같은 경쟁 상태가 발생할 수 있다.

1. 트랜잭션 A가 30042번 포트가 비어 있다고 판단한다.
2. 트랜잭션 B도 30042번 포트가 비어 있다고 판단한다.
3. A와 B가 모두 30042번 포트를 배정하려고 한다.

4. 같은 NodePort가 두 사용자에게 중복 배정된다.

`SELECT ... FOR UPDATE` 는 현재 작업이 끝날 때까지 조회한 행을 다른 작업이 바꾸지 못하게 한다.

두 번째 요청은 첫 번째 요청이 `commit` 하거나 `rollback` 할 때까지 기다린다. 그래서 동시에 들어온 포트 배정 요청도 차례대로 처리된다.

이 구조를 통해 NodePort 중복 배정을 방지한다.

4.4 포트 해제와 동기화

함수	동작
<code>release_nodeports(pod_name)</code>	해당 <code>pod_name</code> 과 연결된 행을 삭제한 후 commit한다.
<code>reconcile_nodeport_allocations(namespace)</code>	MySQL 기록과 Kubernetes Service를 비교해 남은 기록을 정리한다.

release_nodeports

`release_nodeports(pod_name)` 은 특정 Pod가 사용하던 NodePort 배정 정보를 삭제한다.

처리 방식은 다음과 같다.

```
DELETE FROM nodeport_allocations
WHERE pod_name = <pod_name>;
```

삭제 후 `commit()` 을 호출한다.

reconcile_nodeport_allocations

`reconcile_nodeport_allocations(namespace)` 는 infra-mysql 기록과 Kubernetes의 실제 NodePort Service 목록을 비교한다.

이 동기화 작업은 Kubernetes에 실제로 남아 있는 Service를 기준으로 한다.

`app=ai lab-nodeport` 라벨과 같은 `pod_name` 을 가진 Service가 Kubernetes에 하나도 없는데 infra-mysql에 그 Pod의 배정 기록이 남아 있으면 해당 Pod의 행을 삭제한다. 포트별 Service를 하나씩 비교하는 방식은 아니다. 따라서 같은 Pod의 Service가 하나라도 남아 있으면 그 Pod의 DB 행은 모두 남는다.

포트 배정 직전에 이 함수를 호출하지만, Kubernetes API와 DB를 매 요청마다 전체 조회하지 않도록 마지막 시도부터 5분(300초)이 지나야 다시 실행한다. 실패한 시도도 이 5분 간격에 포함한다.

동기화에 실패하면 트랜잭션을 rollback한다. 다만 동기화 실패는 치명적 오류로 처리하지 않으며, 포트 배정 작업은 계속 진행한다.

5. 운영 절차

5.1 상태 확인

접속 방법은 [운영 매뉴얼](#)의 관리 주소와 명령 표를 따른다.

`kubectl exec` 로 `infra-mysql-0` Pod 내부에서 MySQL 클라이언트를 실행한다. 비밀번호는 Kubernetes Secret에서 조회한다.

```
kubectl exec -it infra-mysql-0 -n ailab-infra -- \
mysql -u pod_port_user -p pod_port_db
```

현재 점유 중인 포트 전체는 다음 쿼리로 조회한다.

```
SELECT
    username,
    pod_name,
    node_port,
    purpose
FROM nodeport_allocations
ORDER BY node_port;
```

특정 사용자의 포트 배정 현황은 다음 쿼리로 조회한다.

```
SELECT *
FROM nodeport_allocations
WHERE username = '<username>';
```

FARM 노드에서 다시 정리해야 할 Kerberos 파일 목록은 다음 쿼리로 확인한다.

```
SELECT username, node_name, failed_at
FROM krb5_cleanup_pending
ORDER BY failed_at;
```

5.2 백업

현재 자동 백업은 구성되어 있지 않다.

배포 매니페스트와 코드 어디에도 다음 구성은 존재하지 않는다.

- `mysqldump` 자동 실행

- Kubernetes CronJob
- 외부 백업 스크립트
- 정기 스냅샷 설정

현재 데이터 보호 장치는 StorageClass의 다음 설정뿐이다.

```
reclaimPolicy: Retain
```

Retain 정책은 PVC나 StatefulSet을 삭제하더라도 NFS 위의 PV 데이터를 자동으로 제거하지 않도록 한다.

다만 이는 백업이 아니라 삭제 방지 장치이다. 파일 손상, 잘못된 SQL 실행, NAS 장애에는 대응하지 못한다.

infra-mysql에는 NodePort 배정 기록과 Kerberos 정리 재시도 기록이 있다. 데이터가 사라지면 Kubernetes의 실제 Service를 보고 포트 기록을 다시 만들 수 있다. Kerberos 정리 재시도 기록은 삭제 실패 로그와 FARM 노드의 파일을 대조해 다시 만들어야 한다.

스키마 정의는 코드에서 자동으로 생성되지 않으므로 두 테이블의 DDL을 함께 백업해야 한다.

```
SHOW CREATE TABLE nodeport_allocations;
SHOW CREATE TABLE krb5_cleanup_pending;
```

5.3 재구축 순서

infra-mysql 재구축은 다음 순서로 진행한다.

1. `nfs-mysql.yaml`을 적용하여 StorageClass `sc-mysql`을 생성한다.
2. `infra-mysql.yaml`을 적용하여 StatefulSet과 Service를 생성한다.
3. MySQL 최초 기동 과정에서 `pod_port_db` 데이터베이스와 `pod_port_user` 계정이 자동 생성되는지 확인한다.
4. `nodeport_allocations`, `krb5_cleanup_pending` 테이블의 DDL을 수동으로 실행한다.
5. config-server를 재시작한다.
6. 사용자 Pod 생성 요청을 한 건 실행한다.
7. infra-mysql에 NodePort 배정 행이 정상적으로 삽입되는지 확인한다.
8. Kubernetes에 NodePort Service가 정상적으로 생성되는지 확인한다.

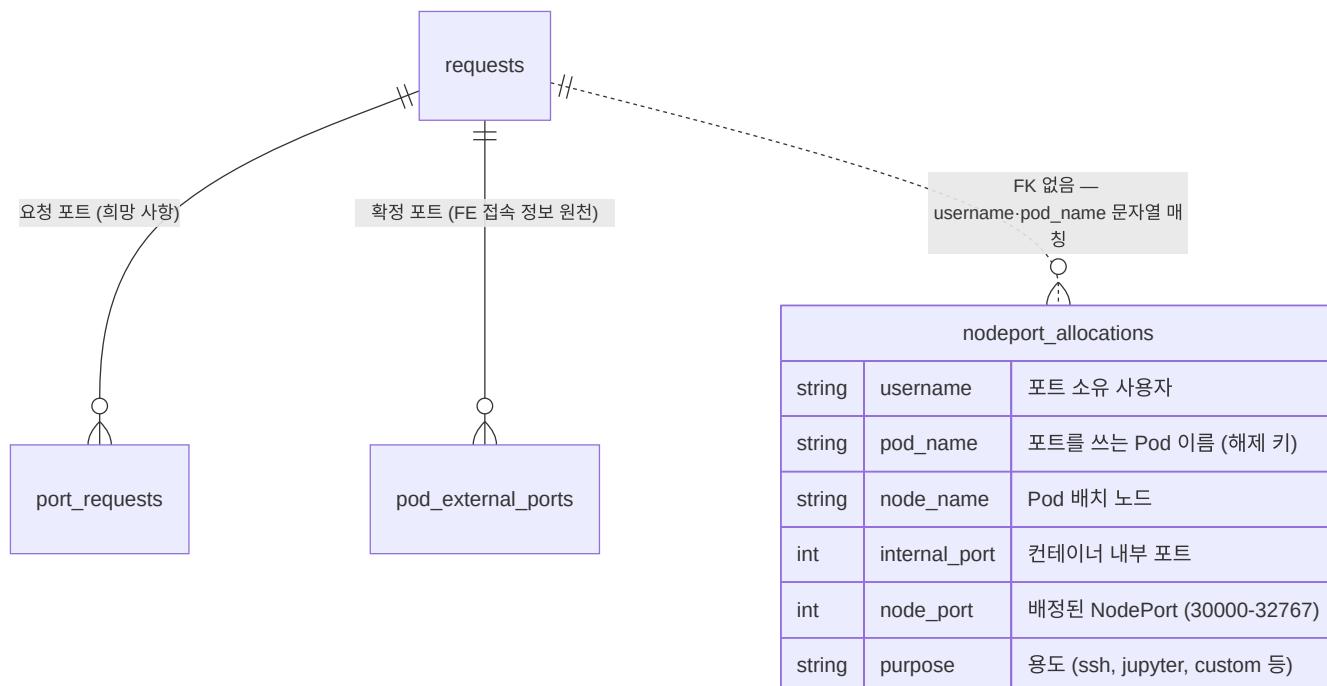
자동 스키마 bootstrap이 없으므로 4단계의 수동 DDL 실행은 필수이다.

6. admin_be DB와 포트 정보를 비교하는 방법

다음 ERD는 포트 요청과 배정 정보를 두 데이터베이스에서 어떻게 비교하는지 보여 준다.

`nodeport_allocations`의 컬럼은 코드에서 실제 사용하는 컬럼을 기준으로 작성한다. 타입과 기본 키 구성은 미확인 상태이므로 개념 수준으로 표기한다.

`admin_be`의 테이블은 별도 저장소에서 관리하므로 테이블 이름과 관계만 표현하고 상세 컬럼은 생략한다.



`infra-mysql`과 `admin_be` MySQL은 물리적으로 분리된 데이터베이스이다.

두 데이터베이스 사이에는 외래 키가 존재하지 않는다.

`nodeport_allocations`와 `admin_be` 테이블은 `username`, `pod_name` 문자열로만 비교할 수 있다. 외래 키가 없으므로 두 DB를 SQL 조인으로 조회할 수 없다.

대조 작업은 다음 방식으로 수행한다.

1. `admin_be` MySQL에서 요청과 확정 포트 정보를 조회한다.
2. `infra-mysql`에서 `nodeport_allocations`를 조회한다.
3. `username`과 `pod_name`을 기준으로 결과를 비교한다.
4. 불일치가 있으면 Kubernetes의 실제 Service 상태까지 함께 확인한다.

ERD의 점선은 외래 키 없이 `username`, `pod_name`을 비교한다는 표시다.

핵심 구조 요약

이 시스템의 데이터베이스 구조는 다음과 같다.

1. MySQL은 `infra-mysql`과 `admin_be` MySQL 두 개로 분리된다.
2. `infra-mysql`은 NodePort 배정 기록과 Kerberos 정리 재시도 기록을 관리한다.
3. `admin_be` MySQL은 사용자와 신청 정보를 관리한다.

4. 두 DB는 직접 연결하지 않고 API를 통해 통신한다.
5. infra-mysql의 데이터베이스와 계정은 MySQL 공식 이미지가 최초 기동 시 생성한다.
6. `nodeport_allocations`, `krb5_cleanup_pending` 테이블은 자동으로 생성되지 않으므로 수동 DDL이 필요하다.
7. NodePort 배정은 `SELECT ... FOR UPDATE` 와 트랜잭션으로 처리한다.
8. 포트 해제는 `pod_name` 을 기준으로 DB 행을 삭제한다.
9. Kubernetes 실제 Service와 DB 기록의 차이는 reconcile 작업으로 정리한다.
10. 자동 백업은 없으며 `reclaimPolicy: Retain` 은 삭제 방지 장치일 뿐 백업은 아니다.
11. 두 데이터베이스 사이에는 외래 키가 없으며 `username` 과 `pod_name` 으로 연결해 비교한다.
12. 장애 조사 시 infra-mysql, admin_be MySQL, Kubernetes Service 상태를 각각 조회해 비교한다.

처음 작업할 때

이 문서에서 다루는 내용

- `develop`, `main`, `feature/*`, `hotfix/*` 브랜치의 역할
- 로컬에서 config-server 이미지를 띄워 Swagger와 `/health` 를 확인하는 방법
- 클러스터에서 배포 결과를 확인할 위치와 PR 규칙

빠른 이동

필요할 때	문서
운영 배포와 장애 대응을 바로 본다	운영 매뉴얼
Helm 값과 차트 구조를 본다	Helm 차트 레퍼런스
config-server가 실제로 무슨 일을 하는지 먼저 본다	개요

1. 브랜치와 배포

코드는 `develop` 에서 모으고, `main` 에 병합되면 운영 서버에 배포된다.

브랜치	역할	배포 여부	비고
<code>main</code>	운영 서버에 배포되는 브랜치	O	PR을 병합하면 배포한다.
<code>develop</code>	기능을 합쳐 테스트하는 브랜치	X	기능 개발이 끝난 뒤 이 브랜치로 PR을 만든다.
<code>feature/*</code>	기능별 작업 브랜치	X	<code>develop</code> 에서 만든다.
<code>hotfix/*</code>	긴급 수정 브랜치	X	<code>main</code> 에서 만들고, 배포하려면 다시 <code>main</code> 에 병합한다.

2. 로컬에서 API 확인하기

config-server는 Python 3.10 slim과 gunicorn으로 만든 컨테이너로 실행한다. 로컬에서는 Docker로 띄워 Swagger와 `/health` 응답을 확인할 수 있다.

```
cd admin_infra/config-server

# 이미지 빌드
docker build -t config-server:local .

# 실행 (컨테이너 포트 8000)
docker run --rm -p 8000:8000 config-server:local
```

- Swagger: `http://localhost:8000/apidocs/`
- 상태 확인: `http://localhost:8000/health`

계정 생성과 Pod 생성은 Kubernetes API, infra-mysql, `/kube_share` NFS, NAS SSH가 있어야 동작한다. 로컬 컨테이너에서는 Swagger와 코드 변경을 확인하고, 실제 생성은 클러스터에서 시험 사용자 한 명으로 확인한다.

3. 클러스터에서 확인할 것

- 운영 config-server는 FARM 클러스터의 `ailab-infra` namespace에서 실행한다.
- `kubectl` 은 control-plane에서 실행한다. CI/CD는 운영 서버 `farm8` 에 SSH로 접속해 `helm upgrade` 를 실행한다.
- 배포 뒤 확인할 명령은 운영 매뉴얼에 있다.

```
# 배포된 config-server 확인
kubectl get pods -n ailab-infra | grep config-server
```

4. 커밋과 PR

1. `develop` 에서 `feature/#이슈번호-작업명` 브랜치를 만든다. 예: `feature/#155-scheduler`
2. 커밋 메시지는 `[분류] #이슈번호 설명` 으로 쓴다. 예: `[feat] #4 메인 기능 만들기`
3. 작업이 끝나면 `feature` 에서 `develop` 으로 PR을 만든다.
4. 정기 배포는 `develop` 에서 `main` 으로 PR을 만든다. 제목은 `[deploy] develop -> main (설명)` 으로 쓴다.
5. 한 명 이상 승인한 뒤 병합한다. `main` 에 병합하면 운영 서버에 배포된다.

배포 전에 다른 사람이 같은 클러스터에서 테스트하거나 Helm 값을 바꾸고 있는지 확인한다. 현재 실행 중인 이미지가 어느 커밋에서 만들어졌는지도 확인한 뒤 배포한다.

운영 매뉴얼

이 문서에서 다루는 내용

- config-server 배포 상태 확인, 운영 주소, 주요 리소스 이름
- 계정과 Pod 생성·삭제 뒤에 무엇을 점검해야 하는지
- 오류 응답, 잔여 리소스, NodePort 기록, Secret을 어떤 순서로 확인해야 하는지

빠른 이동

필요할 때	문서
생성 순서와 실패 지점을 먼저 본다	시스템 아키텍처
API 입력과 오류 형식을 본다	API 레퍼런스
포트 DB와 복구 기준을 본다	데이터베이스
Helm 값과 Secret 이름을 본다	Helm 차트 레퍼런스

config-server 배포, 계정과 Pod 작업, 문제 확인·복구 방법을 정리한다. 계정·Pod·포트가 만들어지는 순서는 [시스템 아키텍처](#), 용어는 [기초 개념](#)에서 확인한다. Helm 값과 Secret 이름은 [Helm 차트 레퍼런스](#)에 있다. **keytab·개인키·비밀번호의 실제 값은 문서나 로그에 적지 않는다.**

1. 작업 전에 확인할 것

작업을 시작하기 전에 다음을 지킨다.

1. **계정과 Pod는 따로 지운다.** 계정을 지운 뒤 Pod가 남거나, Pod를 지운 뒤 계정과 홈이 남을 수 있다. 삭제할 때는 두 API를 모두 호출했는지 확인한다.
2. **작업 전후 값을 기록한다.** 사용자 이름, Pod 이름, UID/GID, 선택 노드, NodePort를 적어 둔다. 문제가 생기면 이 기록으로 어느 단계에서 멈췄는지 확인한다.
3. **API 오류 뒤에는 같은 요청을 바로 반복하지 않는다.** 계정·Pod 생성은 여러 외부 시스템을 차례로 바꾼다. 오류 응답의 `stage` 와 `rollback` 을 먼저 보고 일부만 만들어진 것이 없는지 확인한다.
4. **비밀 값은 출력하지 않는다.** keytab, SSH 개인키, DB 비밀번호, shadow 내용은 화면·로그·티켓 어디에도 남기지 않는다.

- 5. **Kubernetes 리소스를 직접 지우기 전에 기록을 확인한다.** config-server와 MySQL에 남은 포트 배정 기록을 먼저 본다. Pod나 Service 한쪽만 지우면 기록과 실제 상태가 더 달라질 수 있다.
- 6. **예전 chart·스크립트는 복구에 쓰지 않는다.** 레포에 남아 있는 레거시(ContainerSSH, 사용자 PVC chart, restart_*.sh)는 현행 구조와 무관하다(10절 금지 사항).
- 7. **config-server 제어 API는 승인된 경로에서만 접근한다.** 이 API에는 인증이 없으므로(API 레퍼런스의 공통 사항), WAS와 승인된 운영망 밖으로 노출되지 않게 유지한다.
- 8. **배포 뒤에도 실제 동작을 확인한다.** 배포가 끝나면 승인 한 건으로 계정·Pod·접속 포트가 만들어지는지 확인한다.

2. 이름과 접속 주소

2.1 주요 이름과 포트

항목	값
운영 네임스페이스	ailab-infra
Helm release	containersssh-config-server
config-server Deployment	containersssh-config-server
config-server Service	containersssh-config-service
config-server HTTP 포트	Service 80 → 컨테이너 8000
config-server 외부 NodePort	30082
사용자 Pod 접두사	ailab-
사용자 NodePort 범위	30000-32767
Pod Ready 최대 대기	300초 (배포 브랜치에 따라 다를 수 있음)
Pod 삭제 최대 대기	60초
진행 상태 보존	최종 갱신 후 1시간

config-server 관련 리소스에는 과거 이름인 containersssh 접두사가 남아 있다. 현재 ContainerSSH 프록시는 배포하지 않는다.

2.2 서비스별 namespace

namespace는 Kubernetes 리소스를 나누어 두는 단위다.

namespace	배포 대상	분리한 이유
ailab-infra	config-server, infra-mysql, redis, 사용자 Pod 전체	사용자 Pod를 만드는 서비스와 Pod를 함께 둔다.

namespace	배포 대상	분리한 이유
ailab-frontend	admin_fe (정적 서빙)	화면 파일을 제공하는 서비스만 둔다.
default (admin-prod)	admin_be (Spring Boot WAS)	신청과 승인 정보를 관리한다. config-server가 사용자 설정을 조회할 때 admin-prod.default 를 사용한다.
monitoring	kube-prometheus-stack (Prometheus·Grafana)	애플리케이션을 다시 배포해도 모니터링 데이터를 계속 볼 수 있게 따로 둔다.

레거시 namespace `containerssh`, `cssh` 가 클러스터에 남아 있을 수 있다. 현재 서비스는 이 namespace를 쓰지 않는다. 리소스를 지우려면 실제 사용 중인지 먼저 확인한다.

2.3 관리할 때 쓰는 주소와 명령

대상	방법	비고
Grafana 대시보드	<code>http://210.94.179.18:30080</code>	admin 로그인. 비밀번호는 K8s Secret에서 kubectl로 조회한다
Swagger (config-server API)	<code>http://210.94.179.18:30082/apidocs/</code>	현재 API 목록을 볼 수 있다. 인증이 없으므로 <code>Try it out</code> 은 실제 계정·Pod·파일을 바꾼다. README의 9732 포트는 잘못된 값이며 NodePort는 30082다.
infra-mysql (pod_port_db)	<code>kubectl exec -it infra-mysql-0 -n ailab-infra -- mysql -u pod_port_user -p pod_port_db</code>	비밀번호는 K8s Secret에서 kubectl로 조회한다
config-server 로그	<code>kubectl logs -f deploy/containerssh-config-server -n ailab-infra</code>	승인·삭제 실패의 1차 확인 지점

Secret 값 조회는 이름을 찾은 뒤 jsonpath로 디코딩한다.

```
kubectl get secrets -n ailab-infra # Secret 이름 확인
kubectl get secret <이름> -n ailab-infra -o jsonpath='{.data.<키>}' | base64 -d
```

3. 작업 전 상태 확인

매일 한 번, 또는 배포·계정 작업을 하기 전에 아래 항목을 확인한다.

3.1 config-server가 떠 있는지

```
kubectl -n ailab-infra get deploy,sts,pod
kubectl -n ailab-infra get svc
kubectl -n ailab-infra get events --sort-by=.lastTimestamp
kubectl -n ailab-infra rollout status deploy/containersssh-config-server
```

다음 네 가지를 확인한다.

- config-server Deployment가 1/1 Ready이다.
- infra-mysql과 Redis 구성요소가 Ready이다.
- config-server Service가 30082 NodePort를 유지한다.
- 새 Warning 이벤트와 반복 재시작이 없다.

3.2 현재 배포 설정

비밀 값을 출력하지 않고 배포 구조를 확인한다.

```
kubectl -n ailab-infra get deploy containersssh-config-server -o custom-
columns=IMAGE:.spec.template.spec.containers[0].image,SA:.spec.template.spec.serviceAccountName,NODE:.spec.tem
plate.spec.nodeSelector
kubectl -n ailab-infra describe deploy containersssh-config-server
kubectl -n ailab-infra get svc containersssh-config-service
helm -n ailab-infra get manifest containersssh-config-server
```

현재 배포에는 아래 값이 들어가야 한다.

항목	확인할 값
replica	1
프로세스	gunicorn worker 4개, timeout 700초
ServiceAccount	config-server
config-server 배치	지정 control-plane 노드의 nodeSelector와 NoSchedule toleration
컨테이너 포트	8000
readiness	GET /health, 초기 15초, 10초 주기
liveness	별도 probe 없음
리소스	request 500m/512Mi, limit 2000m/1024Mi
API Service	NodePort 30082, Service 80 → container 8000
계정 저장소	NFS kube_share → /kube_share

항목	확인할 값
비밀 mount	NAS, FARM, FARM AD SSH Secret이 read-only
Kerberos 설정	krb5-conf ConfigMap → /etc/krb5.conf
image-store	pvc-image-store → /image-store

config-server 계정의 Kubernetes 권한도 확인한다.

```
kubectl auth can-i list pods -n ailab-infra --as=system:serviceaccount:ailab-infra:config-server
kubectl auth can-i create services -n ailab-infra --as=system:serviceaccount:ailab-infra:config-server
kubectl auth can-i list nodes --as=system:serviceaccount:ailab-infra:config-server
kubectl auth can-i list services --all-namespaces --as=system:serviceaccount:ailab-infra:config-server
```

마지막 명령은 다른 namespace에서 사용 중인 NodePort도 확인하는 권한이다. **no**가 나오면 권한을 바로 바꾸지 말고, 현재 배포의 Role과 ClusterRole을 먼저 확인한다.

3.3 API 응답과 로그

```
config_server_url='운영_config-server_URL'

curl -fsS "$config_server_url/health"
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=30m
```

/health는 config-server 프로세스만 확인한다. admin_be, MySQL, Redis, Prometheus, NAS, FARM SSH, AD 상태는 확인하지 않는다. /health가 OK여도 승인이 실패하면 이 서비스들을 차례로 확인한다.

Pod 생성 로그에는 다음 태그가 차례로 나온다.

- **FETCH_USER_CONFIG** — admin_be 사용자 설정 조회
- **SELECT_NODE** — Prometheus 점수 기반 노드 선택
- **NODEPORT** — 포트 배정
- **WAIT_POD_READY** — Pod Ready 폴링
- **FARM SSH**, **FARM AD SSH** — 노드·AD 원격 작업

3.4 사용자 Pod와 접속 Service

```
kubectl -n ailab-infra get pod -l username
kubectl -n ailab-infra get svc -l app=ailab-nodeport
kubectl -n ailab-infra get pod -o wide
```


각 Service의 `pod_name` 라벨이 실제 Pod 이름과 같은지 확인한다. Error·Failed Pod나 연결 대상이 없는 Service를 발견해도 바로 지우지 않는다. 사용자 이름, 포트 배정 기록, 생성·삭제 로그를 먼저 확인한다(1절 5번).

특정 사용자 Pod의 실행 설정과 마운트를 확인할 때는 jsonpath로 필요한 필드만 뽑는다.

```
pod_name='점검할_ailab_Pod_이름'

kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{.spec.nodeName}{"\n"}{.spec.restartPolicy}{"\n"}{.spec.containers[0].image}{"\n"}{.spec.containers[0].resources}{"\n"}'
kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{range .spec.volumes[*]}{.name}{"\t"}{.hostPath.path}{"\t"}{.hostPath.type}{"\n"}{end}'
kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{range .spec.containers[0].volumeMounts[*]}{.name}{"\t"}{.mountPath}{"\t"}{.readOnly}{"\n"}{end}'
kubectl -n ailab-infra get pod "$pod_name" -o jsonpath='{range .spec.containers[0].env[*]}{.name}{"\n"}{end}'
```

사용자 Pod에서 다음 항목을 확인한다.

- FARM 홈이 `/home` 에 연결되어 있다. 노드에 마운트된 NFS 경로를 hostPath로 연결한다.
- Kerberos 활성 시 `/run/user/<uid>` 가 같은 경로로 연결되어 있다.
- GPU를 배정했다면 `/dev/nvidia<N>` 과 추가 NVIDIA 디바이스가 있다.
- **사용자 Pod에는 keytab, kube_share, image-store가 없어야 한다.** 이 셋이 보이면 잘못된 스펙이다.

현재 Pod 스펙에는 securityContext와 사용자 probe가 없다. 컨테이너 안의 UID/GID, SSH, Jupyter, noVNC 동작은 guest image의 entrypoint까지 확인해야 알 수 있다.

아래 명령은 중복된 NodePort만 출력한다. 아무것도 나오지 않아야 한다.

```
kubectl -n ailab-infra get svc -l app=ailab-nodeport -o jsonpath='{range .items[*].spec.ports[*]}{.nodePort}{"\n"}{end}' | sort -n | uniq -d
```

3.5 GPU와 NFS 상태

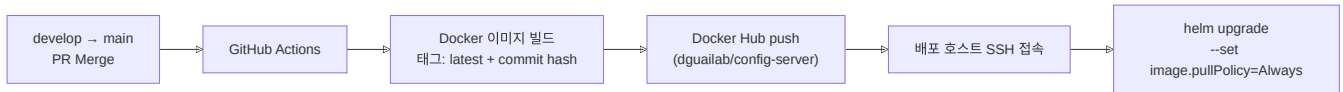
대상	방법
통합 대시보드 (Grafana)	<code>http://210.94.179.18:30080</code> admin 로그인
스토리지 응답성	Grafana 대시보드 <code>storage-latency-health</code> — NFS 장애 의심 시 가장 먼저 본다
Prometheus 직접 질의	<code>kubectl -n monitoring port-forward svc/monitoring-kube-prometheus-prometheus 9090:9090</code>

Prometheus 값은 대시보드뿐 아니라 Pod를 올릴 노드를 고를 때도 쓴다. Prometheus 조회가 모두 실패하면 `create-pod` 도 실패한다. GPU 지표가 없을 때 노드를 고르는 방식은 **시스템 아키텍처**의 GPU 노드 선택 방식에 있다.

4. 배포와 롤백

4.1 배포가 진행되는 순서

배포 자동화는 GitHub Actions를 사용하며, **main** 브랜치에 **push** 이벤트가 발생할 때만 실행된다. 즉 **develop** → **main** PR이 Merge 되는 순간 운영 배포가 시작된다.



CI는 다음을 실행한다.

1. **config-server/Dockerfile** 로 이미지를 빌드한다.
2. **latest** 와 소스 버전을 나타내는 commit hash 태그 두 가지로 레지스트리에 푸시한다.
3. **config-server/Chart** 를 배포 호스트로 전달한다.
4. CI Secret의 NFS, NAS, FARM, AD, Realm 값을 Helm에 주입한다.
5. **ailab-infra** release의 이미지 태그를 commit hash로 갱신한다.

CI/CD용 GitHub Secrets는 **DOCKER_USERNAME**, **DOCKER_PASSWORD**, **K8S_HOST**, **K8S_USERNAME**, **K8S_PRIVATE_KEY**, **K8S_PORT** 이다. 현재 username이 toni 기준으로 등록되어 있어 관리자 변경 시 인수인계가 필요하다. 비밀 값은 repository 파일, 일반 운영 절차, 로그에 복사하지 않는다.

4.2 배포 전 확인

코드를 병합하기 전에 다음을 확인한다.

1. **다른 작업 진행 여부** — 다른 사람이 같은 클러스터에서 인프라 테스트를 하거나 Helm 값을 바꾸고 있으면, 변경 시점과 범위를 먼저 맞춘다.
2. **배포 브랜치** — main과 hotfix 계열 브랜치가 분기된 이력이 있어 Ready 대기·krb5 방식·마운트 구성이 브랜치마다 다르다. "지금 클러스터에 떠 있는 이미지가 어느 커밋인가"부터 확인한다.

명령으로는 다음을 확인한다.

```
helm lint config-server/Chart
git diff -- config-server .github/workflows/deploy-config-server.yaml
kubectl -n ailab-infra get deploy/containersssh-config-server -o
jsonpath='{.spec.template.spec.containers[0].image}{"\n"}'
helm -n ailab-infra history containersssh-config-server
```

다음에 맞아야 한다.

- 변경 범위가 config-server와 chart 의도에 맞다.
- 이미지 태그가 commit hash인지 확인한다.
- chart가 참조하는 Secret과 ConfigMap 이름이 운영 네임스페이스에 존재한다(Helm 차트 레퍼런스 1.4절).
- FARM 노드 목록과 AD DC 목록의 이름이 실제 Kubernetes 노드 이름과 일치한다.

- `FARM_HOME_MOUNT_ROOT` 가 모든 후보 노드에서 동일한 FARM 홈을 가리킨다.

4.3 배포 뒤 확인

```
kubectl -n ailab-infra rollout status deploy/containersssh-config-server --timeout=5m
kubectl -n ailab-infra get pod -l app=containersssh-config-server -o wide
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=10m
kubectl -n ailab-infra get svc containersssh-config-service
```

로그에서 먼저 볼 오류는 다음과 같다.

- `ModuleNotFoundError` — requirements.txt 누락 또는 파일명 불일치이다.
- `WORKER TIMEOUT` — 초기 로딩 시간이 긴 경우이다 (Dockerfile 타임아웃 설정 확인).

이후 시험 사용자 또는 승인된 기존 사용자 한 명으로 실제 동작을 확인한다.

1. `/health` 를 호출한다.
2. 사용자 조회 API를 호출한다.
3. 시험 사용자 또는 승인된 기존 사용자로 Pod 생성 단계를 확인한다.
4. NodePort Service와 실제 접속을 확인한다.
5. 선택 노드의 Kerberos ccache와 FARM 홈 읽기·쓰기를 확인한다.
6. Pod 삭제 후 Service, 포트 예약, 노드 타이머가 정리되는지 확인한다.

4.4 롤백

배포 뒤 기존에 되던 기능이 동작하지 않으면 Helm release 이력을 보고 이전 버전으로 되돌린다.

```
helm -n ailab-infra history containersssh-config-server
helm -n ailab-infra rollback containersssh-config-server '정상_리비전'
kubectl -n ailab-infra rollout status deploy/containersssh-config-server --timeout=5m
```

롤백은 **config-server Deployment**만 이전 버전으로 되돌린다. 계정 파일, MySQL 포트 배정 기록, Kubernetes Service, FARM 노드의 keytab은 자동으로 이전 상태가 되지 않는다. 배포 중 실패한 요청이 있으면 해당 요청의 `stage`, `rollback`, 실제 리소스를 함께 확인한다.

5. 계정 관리

계정 정보는 `/kube_share` 의 passwd 파일에 기록한다(시스템 아키텍처의 계정 정보). 아래 API만 이 파일을 수정한다. passwd 파일을 직접 편집하지 않는다.

5.1 조회

```
config_server_url='운영_config-server_URL'
target_user='사용자명'

curl -fsS "$config_server_url/accounts/users"
curl -fsS "$config_server_url/accounts/users/$target_user"
```

계정 상세에서 사용자 이름, UID, GID, 홈, 셸을 확인한다. shadow와 keytab 내용은 조회하지 않는다(1절 4번).

5.2 생성

계정은 관리자 승인 뒤 admin_be가 만든다. 운영자가 직접 호출하는 것은 예외이며, 호출 전에는 같은 사용자가 없는지 확인한다.

```
curl -fsS -X PUT "$config_server_url/accounts/users" -H 'Content-Type: application/json' --data '{
  "name": "사용자명",
  "passwd_base64": "BASE64로_인코딩한_비밀번호",
  "gecos": "사용자 설명",
  "supplementary_groups": [
    {"name": "그룹명", "gid": 20050}
  ]
}'
```

성공 응답의 UID/GID를 기록하고(1절 2번) 다음 순서로 확인한다.

1. 사용자 조회 API
2. NAS 홈 디렉터리의 숫자 소유권과 0700
3. FARM AD 사용자와 <user>_gid
4. Kubernetes keytab Secret의 **존재 여부만** 확인 (내용은 보지 않는다)

```
target_user='사용자명'
kubectl -n ailab-infra get secret "krb5-keytab-$target_user" -o custom-
columns=NAME:.metadata.name,CREATED:.metadata.creationTimestamp
```

5.3 그룹

그룹 생성은 이름을 필수로 받고, GID를 생략하면 다음 사용 가능한 값을 할당한다.

```
curl -fsS -X PUT "$config_server_url/accounts/groups" -H 'Content-Type: application/json' --data '{"name":"그룹명","members":["사용자명"]}'
```

기존 그룹에 사용자를 추가한다.

```
curl -fsS -X PUT "$config_server_url/accounts/users/사용자명/groups" -H 'Content-Type: application/json' --data '{"groups":["그룹명"]}'
```

사용자가 기본 그룹으로 쓰는 그룹은 삭제할 수 없다.

5.4 삭제

계정 삭제 전에 사용자의 Pod를 먼저 삭제한다(6.2절). Pod 이름과 선택 노드를 기록하고, Service와 NodePort 정리가 끝난 뒤 계정을 삭제한다. 순서를 지키지 않으면 주인 없는 Pod가 남는다.

```
target_user='사용자명'
curl -fsS -X DELETE "$config_server_url/accounts/users/$target_user"
```

삭제 뒤에는 다음을 확인한다.

- 계정 조회가 404를 반환한다.
- 사용자 Pod와 Service가 없다.
- keytab Secret이 없다.
- 모든 FARM 노드에서 사용자 타이머와 keytab이 없다.
- NAS 홈 디렉터리가 현재 보존·삭제 규칙에 맞게 처리되었다.

NAS 홈 삭제가 실패해도 계정 삭제는 계속 진행된다. **deleted** 응답 뒤에도 홈 디렉터리가 남을 수 있으므로 로그와 NAS 경로를 확인한다.

6. 사용자 Pod 관리

6.1 만들기

계정과 WAS 사용자 설정이 있는지 확인한 뒤 호출한다.

```
target_user='사용자명'

curl -fsS "$config_server_url/accounts/users/$target_user"
curl -fsS -X POST "$config_server_url/create-pod" -H 'Content-Type: application/json' --data '{"username":"'${target_user}'"}'
```

노드 선택, 포트 배정, krb5 배포, Pod 생성, Ready 대기를 차례로 처리하므로 응답까지 시간이 걸린다. 기다리는 동안 다른 터미널에서 진행 상태를 조회할 수 있다.

```
curl -fsS "$config_server_url/pods/$target_user/status"
```

성공 응답에서 `pod_name`, `node`, `ports` 를 기록하고(1절 2번), 실제 리소스를 확인한다.

```
pod_name='응답의_Pod_이름'

kubectl -n ailab-infra get pod "$pod_name" -o wide
kubectl -n ailab-infra get svc -l "pod_name=$pod_name"
kubectl -n ailab-infra describe pod "$pod_name"
```

Pod가 Ready이고, 응답의 각 포트에 해당하는 NodePort Service가 있으며, SSH·Jupyter·선택 기능에 실제로 접속 되면 생성이 끝난 것이다.

6.2 지우기

삭제 전 사용자에게 중단 사실을 확인하고 Pod 이름을 정확히 기록한다.

```
pod_name='삭제할_ailab_Pod_이름'

curl -fsS -X POST "$config_server_url/delete-pod" -H 'Content-Type: application/json' --data
'{"pod_name":"$pod_name"}'
```

응답의 `progress` 에서 각 단계의 완료 여부를 확인한다.

필드	의미
<code>servicesDeleted</code>	연결된 NodePort Service 삭제 완료
<code>nodeportsReleased</code>	MySQL 예약 해제 완료
<code>podDeleteRequested</code>	Kubernetes 삭제 요청 접수
<code>podDeleted</code>	실제 Pod 삭제 확인

삭제 후 실제로 사라졌는지 조회한다.

```
kubectl -n ailab-infra get pod "$pod_name"
kubectl -n ailab-infra get svc -l "pod_name=$pod_name"
```

Pod가 이미 없다는 성공 응답은 노드 이름을 읽지 못해 Kerberos 노드 정리를 건너뛰었을 수 있다. 이 경우 사용자와 마지막 노드를 기준으로 `kdc-setup` 운영 문서의 keytab, ccache, timer를 확인한다.

수동으로 지울 때 **kubectl** 로 Pod만 지우지 않는다. Service와 MySQL 배정 행이 남는다. 배정 정리(reconcile)는 같은 `pod_name` 의 Service가 하나라도 남아 있으면 그 Pod의 배정 행을 지우지 않는다. Service가 남은 상태에서는 이 배정이 계속 남는다(시스템 아키텍처의 NodePort 배정 방식).

6.3 사용자 Pod에 들어가 확인하기

어떤 경로든 Pod 이름 찾기부터 시작한다. 사용자 Pod 이름은 `ailab-<username>--<랜덤>` 형식이다.

```
kubectl get pods -n ailab-infra | grep <username>
```

(a) **kubectl exec** 로 확인하기. 사용자의 SSH 설정·비밀번호와 관계없이 root로 바로 들어간다. 컨테이너 프로세스, 마운트, 홈 권한을 확인할 때 쓴다.

```
kubectl exec -it ailab-user2100-1a2b3c4d -n ailab-infra -- bash
```

(b) **사용자가 쓰는 SSH 경로로 확인하기.** 사용자가 접속하지 못한다는 문의는 이 경로로 확인한다. NodePort는 Service 목록이나 infra-mysql `nodeport_allocations` 에서 확인한다. 이 번호는 pfSense 전달 대상 확인에 쓰고, SSH 접속에는 사용자에게 안내한 외부 주소와 포트를 쓴다.

```
kubectl get svc -n ailab-infra | grep <username> # ssh 용도 Service의 NodePort 확인  
ssh -p <사용자에게_안내한_외부_포트> <username>@<사용자에게_안내한_주소>
```

교내망에서는 되는데 외부에서만 접속하지 못하면 **시스템 아키텍처**의 외부 접근 경로를 보고 Service의 NodePort와 pfSense 전달 대상이 같은지 확인한다.

(c) **컨테이너에 들어갈 수 없을 때.** Pod가 CrashLoop 등으로 `exec` 할 수 없으면 로그와 노드에서 확인한다.

```
kubectl logs ailab-user2100-1a2b3c4d -n ailab-infra # 컨테이너 stdout 로그  
kubectl describe pod ailab-user2100-1a2b3c4d -n ailab-infra # 이벤트(FailedMount 등)
```

노드에 SSH로 들어가 컨테이너 런타임을 직접 보는 방법도 있다.

```
crictl ps | grep <username> # 해당 노드의 컨테이너 목록  
crictl logs <container-id>
```

6.4 Pod가 죽었거나 노드에 문제가 있을 때

현재 config-server는 Pod가 죽은 것을 감지해 다른 노드에 자동으로 다시 만들지 않는다. 운영자가 다음 순서로 확인하고 복구한다.

1. 사용자 이름, 기존 Pod 이름, resource group, 노드, 이미지, NodePort를 기록한다.
2. Pod가 Failed인지, 노드가 NotReady인지, 애플리케이션만 응답하지 않는지 구분한다.
3. 같은 사용자의 다른 Running Pod가 없는지 확인한다.
4. FARM 홈과 AD·keytab Secret은 삭제하지 않는다. Pod를 다시 만들어도 계정 관련 파일은 그대로 써야 한다.
5. 장애 노드가 후보에서 다시 선택되지 않도록 WAS resource group 또는 노드 상태를 먼저 정리한다.
6. `/create-pod` 로 대체 Pod를 만들고 새 노드의 ccache, 홈, SSH·Jupyter를 확인한다.
7. 현재 구현은 새 NodePort를 할당하므로 **새 접속 정보를 사용자에게 전달한다.**

8. 대체 Pod가 정상인지 확인한 뒤에야 기존 Service, Pod, NodePort 배정 기록, 노드의 Kerberos 파일을 정리한다.

```
target_user='사용자명'

kubectl -n ailab-infra get pod -l "username=$target_user" -o wide
kubectl -n ailab-infra get svc -l "username=$target_user"
curl -fsS "$config_server_url/pods/$target_user/status"
```

자동으로 다시 만드는 기능이 없으므로 Failed Pod를 발견했다고 기존 Service부터 지우지 않는다.

6.5 Pod를 다른 노드로 옮길 때

`/migrate` 는 같은 resource group의 다른 노드가 충분히 여유 있을 때 새 Pod를 먼저 만드는 API다. 그러나 사용자 image-store 연결과 기존 NodePort 유지가 검증되지 않았으므로 **운영 사용자에게 일반적으로 실행하지 않는다.**

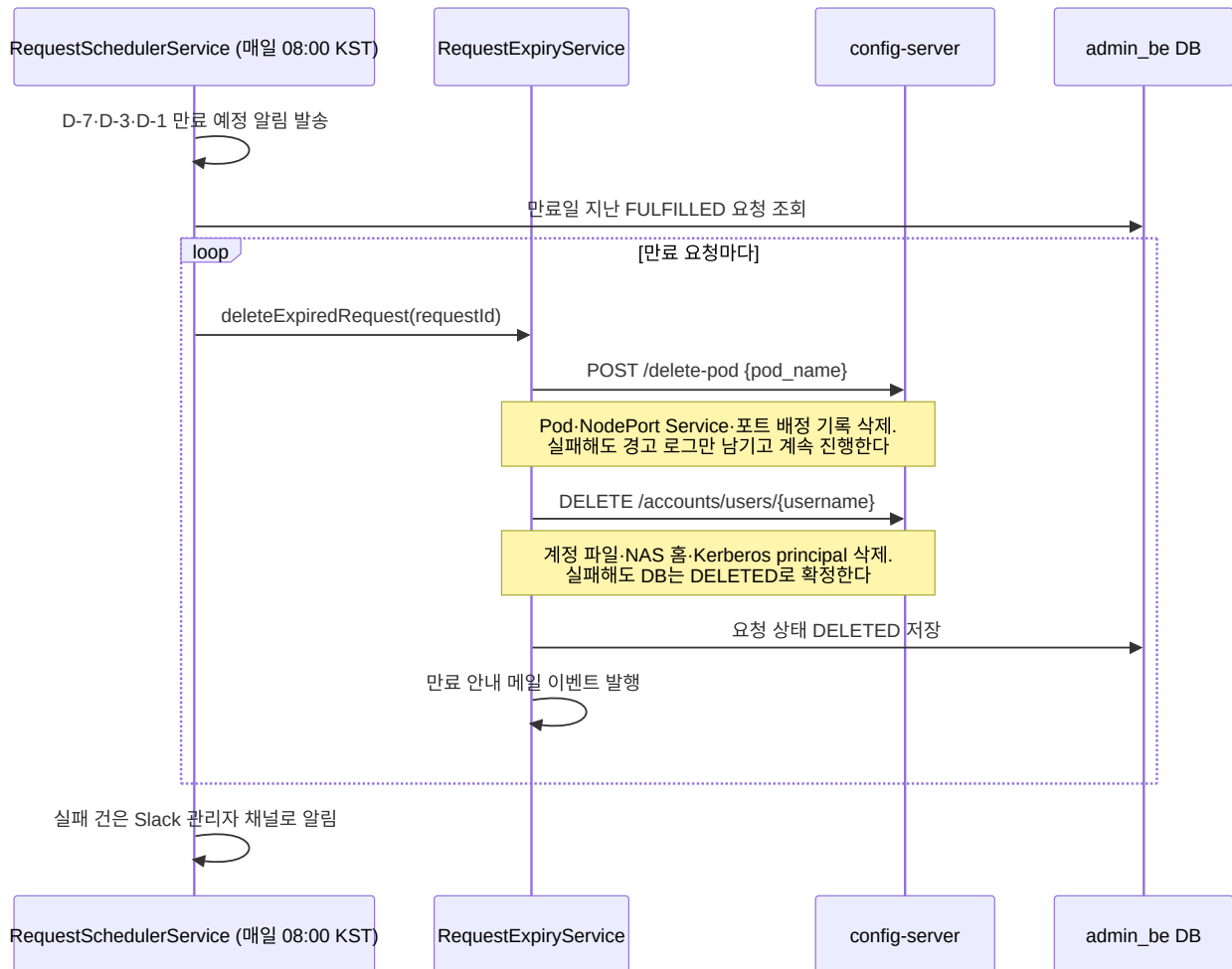
시험 환경에서 확인할 때는 다음을 모두 준비한다.

- 삭제해도 되는 시험 사용자와 재현 가능한 이미지
- 두 개 이상의 정상 GPU 후보 노드
- FARM 홈 백업 또는 시험 데이터
- 현재 Pod·Service·NodePort·선택 노드 기록
- 새 Pod 실패 시 기존 Pod를 유지하는 확인 절차
- 새·이전 노드의 Kerberos 자산 점검

새 Pod가 Ready여도 홈 데이터, 설치 패키지, SSH·Jupyter·noVNC, 포트 연속성, 이전 노드 정리까지 확인해야 한다.

6.6 만료된 사용자 정리

자원 회수는 admin_be의 일일 스케줄러가 자동 수행한다. `RequestSchedulerService` 가 매일 **08:00(KST)** 에 돌면서 만료 예정 알림을 보내고, 만료일이 지난 요청을 `RequestExpiryService` 에 넘겨 config-server API 두 개를 순서대로 호출한다. (사용자 비활성 스케줄러는 별도로 09:00에 돈다.)



만료 처리에서는 다음 두 가지를 확인한다.

1. **삭제할 때는 두 API를 모두 호출한다.** `POST /delete-pod` 는 Pod, Service, 포트만 지우고 `DELETE /accounts/users` 는 계정, 홈, principal만 지운다. 상세 명세는 [API 레퍼런스](#)를 참고한다.
2. **외부 정리가 실패해도 DB는 DELETED로 확정된다.** 인프라에 남은 자원이 있어도 admin_be 화면에는 삭제 완료로 보인다. 만료 처리가 있었던 날에는 8절에서 Pod, Service, 포트 기록을 확인한다. 과거에는 만료 처리에서 계정만 지우고 Pod·NodePort를 남기는 결함 T-KI-01이 있었고 26-07-06에 수정 배포했다. 만료 처리 경로는 아직 실제로 검증하지 않았다.

7. 문제가 생겼을 때

7.1 먼저 확인할 순서

문제가 생기면 API 응답부터 실제 리소스까지 다음 순서로 확인한다.

API HTTP 상태

- └ error stage / error code / rollback
 - └ config-server 로그
 - └ 연결된 서비스 (WAS·MySQL·Redis·Prometheus·NAS·FARM)
 - └ Kubernetes 실제 리소스
 - └ MySQL·FARM 노드 잔존 상태

같은 요청을 재시도하기 전에 부분 생성 여부를 확인한다. 특히 계정 생성과 Pod 생성은 여러 외부 저장소를 순차 갱신하므로, 중간 실패는 "일부만 만들어진" 상태를 남긴다.

7.2 증상별로 먼저 볼 곳

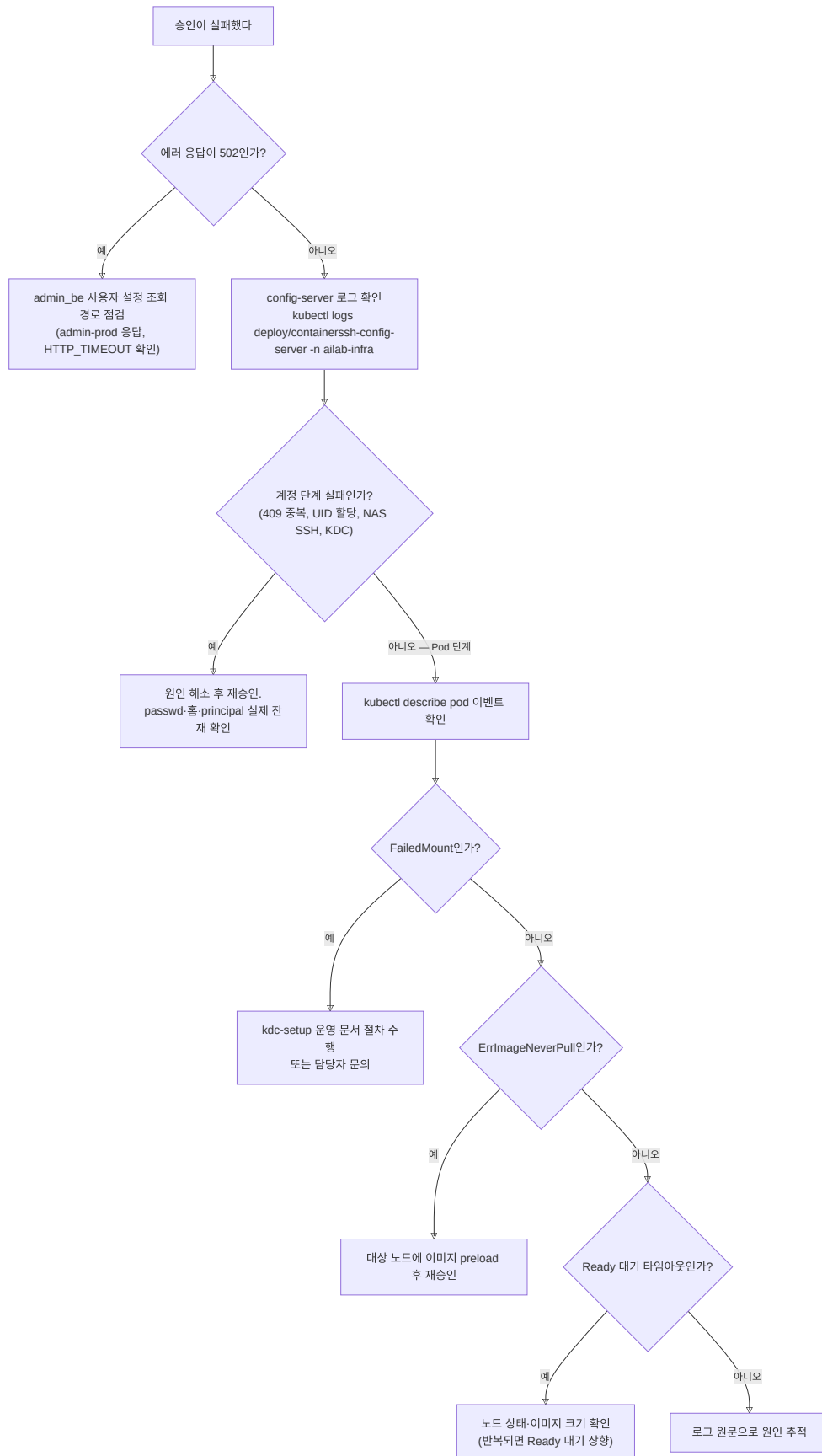
자주 생기는 문제와 먼저 볼 곳을 정리했다. 각 항목의 자세한 절차는 이어지는 절에 있다.

장애	증상	확인	해결
Pod Pending / ErrImageNeverPull	승인 후 Pod가 Pending, Ready 대기 초과로 승인 롤백	<code>kubectl -n ailab-infra describe pod <pod></code> 이벤트, 대상 노드 `crictl images \	grep decs`
FailedMount (krb5)	<code>mount.nfs: Key has expired</code> 로 신규 Pod 홈 마운트 실패	<code>kubectl -n ailab-infra describe pod <pod></code> 의 FailedMount 이벤트	FARM NAS 홈 마운트에는 Kerberos 인증이 전제된다. kdc-setup 운영 문서 절차를 따르거나 담당자에게 문의한다
계정 생성 실패	승인이 계정 단계에서 실패(409, UID 할당 실패), PENDING 복귀	<code>kubectl -n ailab-infra logs deploy/containerssh-config-server, /kube_share/passwd</code> , NAS SSH 수동 확인	원인(중복 username, NAS SSH, KDC) 해소 후 재승인. 롤백 로그를 맹신하지 말고 passwd·홈·principal의 실제 잔재를 확인한다
NodePort 외부 접속 불가	내부(교내망)는 정상인데 외부 접속만 안 됨	<code>nodeport_allocations</code> 와 Service의 NodePort, pfSense 전달 대상, 사용자에게 안내한 외부 주소를 대조	pfSense 전달 대상이나 안내한 주소를 바로잡고 다시 접속 확인

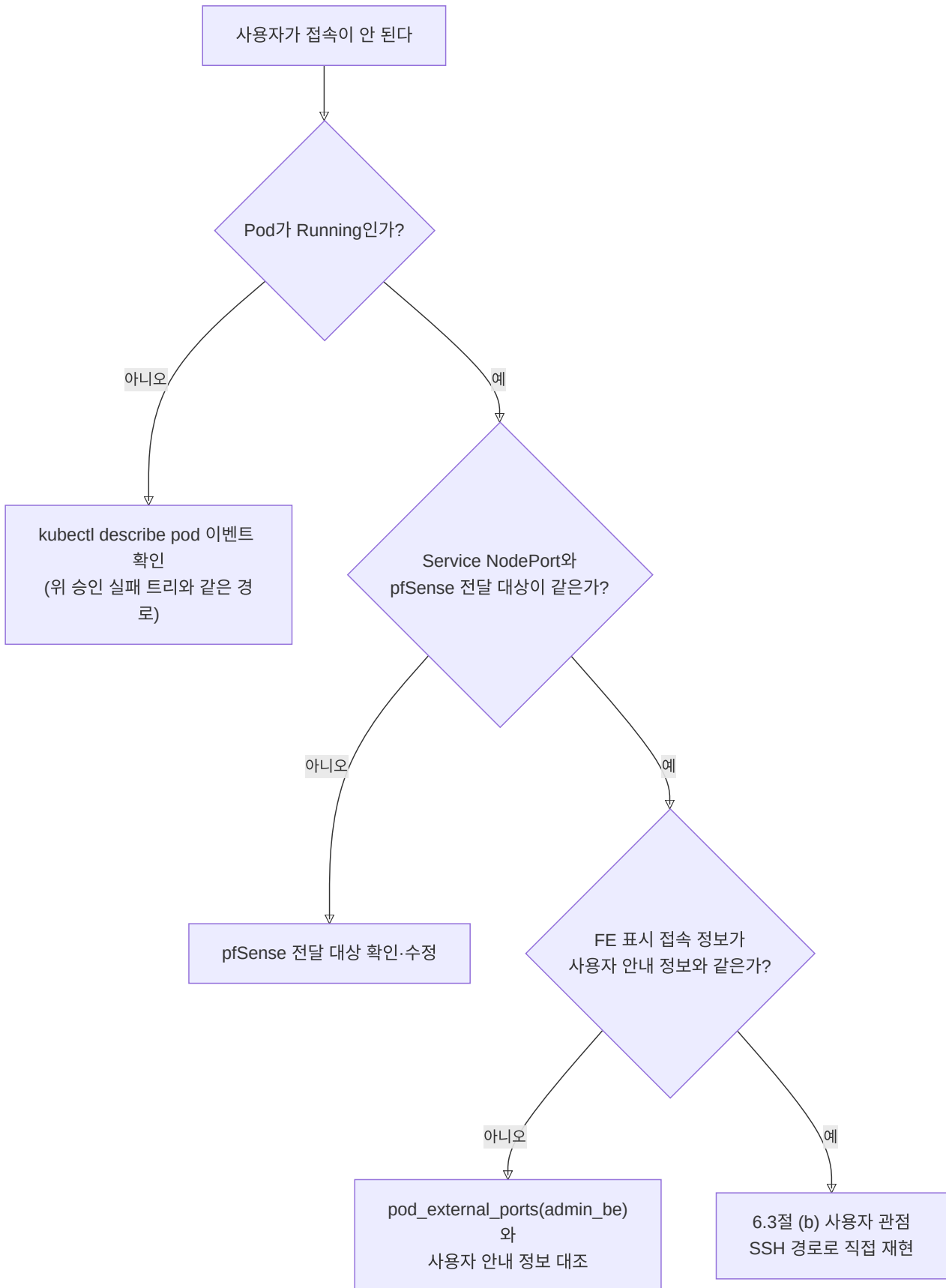
7.3 승인과 접속 문제 확인 순서

승인이 실패했을 때와 사용자가 접속하지 못할 때는 아래 순서로 확인한다.

트리 1 — 승인이 실패했다



트리 2 — 사용자가 접속이 안 된다



7.4 사용자 설정 조회 실패 (**FETCH_USER_CONFIG**)

config-server가 admin_be에서 사용자 설정을 가져오지 못한 경우다. 이때 승인은 되었지만 Pod 생성만 실패할 수 있다.

1. `admin-prod` Deployment와 Service 상태
2. 대상 사용자 승인·설정 존재 여부
3. config-server에서 admin_be Service DNS와 HTTP 응답
4. 응답 JSON의 image, gpu_nodes, additional_ports 형식
5. config-server의 3초 외부 HTTP 제한(`HTTP_TIMEOUT_SEC`) 안에 응답하는지

WAS가 HTTP 200 본문에 404 상태를 넣는 경우도 사용자 없음으로 처리된다.

7.5 노드 선택 실패 (`SELECT_NODE` , `NODE_LIST_FAILED`)

```
kubectl get node
kubectl get node -o custom-columns=NAME:.metadata.name,TAINTS:.spec.taints
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=20m
```

- WAS가 내려준 후보 이름이 실제 Kubernetes 노드와 일치하는지 확인한다.
- `gpu_nodes` 가 비어 있을 때 사용할 Ready 상태의 일반 워커 노드가 있는지 확인한다.
- Prometheus가 각 후보의 GPU 지표를 반환하는지 확인한다. exporter가 죽은 노드는 0점으로 오인 선택될 수 있다(시스템 아키텍처의 GPU 노드 선택 방식).
- 선택된 노드에 FARM 홈과 Kerberos 관리 채널이 준비되어 있는지 확인한다.

7.6 NodePort 할당 또는 Service 생성 실패 (`NODEPORT` , `CREATE_NODEPORT_SERVICE`)

1. 요청 포트가 숫자이며 내부 포트가 중복되지 않는지 확인한다.
2. 30000-32767 범위의 실제 Service 사용 포트를 조회한다.
3. MySQL `nodeport_allocations` 의 같은 Pod·포트 예약을 조회한다(데이터베이스).
4. 오류 응답의 `nodeportsReleased` 와 `servicesDeleted` 를 확인한다.
5. Pod가 남아 있으면 연결된 Service 라벨을 대조한다.

포트 할당 기록과 Service가 다르다는 이유만으로 DB 행이나 Service를 바로 삭제하지 않는다. 사용자 접속 여부와 생성·삭제 로그를 먼저 확인한다.

7.7 Pod Ready 실패

```
kubectl -n ailab-infra describe pod "$pod_name"
kubectl -n ailab-infra logs "$pod_name" --all-containers
kubectl -n ailab-infra get events --field-selector "involvedObject.name=$pod_name"
```

증상별로 볼 곳이 다르다.

증상	우선 확인
<code>ImagePullBackOff</code> , <code>ErrImagePull</code> , <code>ErrImageNeverPull</code>	이미지 이름, 태그, 레지스트리 접근, 노드 캐시 (<code>crictl images</code>)

증상	우선 확인
CreateContainerConfigError	환경 변수, hostPath, Secret, ConfigMap
CrashLoopBackOff	guest image entrypoint, UID/GID, ccache, 셀 설정
Pending	직접 지정된 nodeName, 노드 Ready, hostPath, 디바이스
Ready 시간 초과	이미지 크기, 노드 I/O, 컨테이너 readiness 상태

생성 실패 응답이 Pod와 NodePort를 실제로 정리했는지 반드시 확인한다.

7.8 Kerberos 배포 또는 홈 접근 실패

deploying_krb5 단계에서 실패하면 다음을 확인한다.

1. 사용자 keytab Secret 메타데이터
2. 선택 노드와 FARM 노드 설정의 일치
3. ai lab-krb5 forced-command SSH 채널
4. 선택 노드의 decs-krb-refresh@<user> 로그
5. ccache 소유권과 FARM TGT
6. FARM 홈 마운트와 NAS 소유권

상세 명령과 머신 ccache 안전 규칙은 kdc-setup 운영 문서를 따른다.

7.9 MySQL 또는 Redis 장애

MySQL 장애는 NodePort 예약 실패로 Pod 생성을 중단시킨다. infra-mysql Pod, Service, PVC, 연결 제한과 config-server DB 오류를 확인한다.

Redis 장애는 진행 상태 기록만 누락시킨다 — 생성 자체는 성공할 수 있다. 생성 API와 Kubernetes 리소스가 성공했는데 상태 API가 unknown 이거나 오래된 단계라면 Redis 연결과 TTL을 확인한다. Redis 복구를 위해 정상 사용자 Pod를 삭제하지 않는다.

7.10 삭제가 중간에 멈췄을 때

오류 응답의 progress 와 실제 상태를 비교해 어느 단계까지 처리됐는지 확인한다.

실제 상태	다음 조치
Service만 삭제됨	사용자 접속 중단을 알리고 포트 할당 기록과 Pod 상태를 확인한다
Service·예약 삭제, Pod 남음	같은 Pod 이름으로 삭제 API 재시도 여부를 판단한다
Pod 삭제, 노드 Kerberos 파일 남음	Kerberos 재조정 작업 상태를 확인하고 대상 노드에서 정리한다
계정 삭제, NAS 홈 남음	데이터 보존 여부를 확인한 뒤 NAS 운영 절차로 처리한다

Kerberos 파일 정리가 실패한 경우에는 infra-mysql의 `krb5_cleanup_pending`도 확인한다. 이 테이블에 행이 있으면 30분마다 실행되는 CronJob이 다시 정리한다. CronJob도 실패하면 해당 사용자와 노드의 keytab, timer, ccache를 [kdc-setup 운영 문서](#) 순서대로 확인한다.

수동으로 정리하기 전에는 만료 스케줄러나 krb5 reconcile이 같은 파일을 다시 처리하지 않는지 확인한다.

8. 기록과 실제 상태 비교

MySQL·계정 파일의 기록이 Kubernetes·NAS·AD의 실제 상태와 같은지 정기적으로 확인한다. 만료 처리가 있었던 날에는 반드시 확인한다.

8.1 Pod·Service

```
kubectl -n ailab-infra get pod -l username -o custom-  
columns=NAME:.metadata.name,USER:.metadata.labels.username,NODE:.spec.nodeName,PHASE:.status.phase  
kubectl -n ailab-infra get svc -l app=ailab-nodeport -o custom-  
columns=NAME:.metadata.name,USER:.metadata.labels.username,POD:.metadata.labels.pod_name,NODEPORT:.spec.ports[0].nodePort
```

Pod가 없는 Service, Service가 없는 Running Pod, 같은 Pod에 중복으로 열린 목적 포트를 목록으로 남긴다. **이 단계에서는 지우지 않는다.** 기록과 실제 상태를 비교한 뒤에 처리 방법을 결정한다.

만료가 있었던 날의 간이 확인 명령이다.

```
kubectl get pods -n ailab-infra # 만료 사용자 Pod 잔류 여부  
kubectl get svc -n ailab-infra | grep ailab- # NodePort Service 잔류 여부
```

잔류가 보이면 수동 정리는 `POST /delete-pod` → `DELETE /accounts/users` 순서를 지킨다(6.2절, 5.4절).

8.2 계정·AD·NAS

시험 사용자 또는 문제가 있는 사용자에 대해 다음 값을 한 줄씩 비교한다. 값이 다른 행이 문제를 찾아볼 곳이다.

확인 위치	비교 값
config-server	사용자 이름, UID, GID
FARM AD	sAMAccountName, uidNumber, gidNumber, 전용 그룹
NAS	홈 경로, 숫자 소유권, 모드
선택 노드	ccache 소유 UID, principal
Pod	<code>id</code> , 홈 소유권, ccache 경로

9. 코드나 설정을 바꾼 뒤 확인할 것

코드나 설정을 바꾼 뒤 기존 기능이 깨지지 않았는지 확인한다.

9.1 API 또는 계정 로직

- 기존 응답 코드와 `stage`, `error_code`, `rollback` 형식을 유지한다.
- `passwd·group·shadow` 쓰기 순서와 실패했을 때 되돌리는 처리를 함께 시험한다.
- NAS, AD, Secret 중간 실패를 각각 시험한다.
- 사용자 이름과 그룹 입력 검증을 우회할 경로가 없는지 확인한다.

9.2 Pod 명세 또는 guest image

- UID/GID와 추가 그룹 환경 변수를 양쪽에서 같은 이름으로 사용한다.
- `/home` 과 `/run/user/<uid>` hostPath를 함께 검증한다.
- SSH 22, Jupyter 8888, noVNC 6080, 추가 포트 회귀 시험을 수행한다.
- GPU 디바이스와 nodeName 변경 뒤 다른 Pod가 같은 GPU 디바이스에 접근하지 않는지 확인한다.

9.3 Helm과 권한

- ServiceAccount의 namespaced 권한과 Node 조회 권한을 최소 범위로 유지한다.
- Secret 값은 values 파일과 로그에 남기지 않는다.
- 배포 후 실제 이미지 태그와 source 버전을 기록한다.

9.4 문제 확인에 쓰는 설정값

문제를 확인할 때 바꿔 보며 원인을 좁힐 수 있는 값들이다. 위치는 `config-server/main.py` 상단 `app.config` (코드 상수와 env), `config-server/Chart/values.yaml` (Helm 배포값) 두 곳이다.

키	위치	역할	디버깅 때 언제 바꾸나
<code>NAMESPACE</code> (<code>ai lab-infra</code>)	main.py 상수	모든 Pod·Service를 만드는 namespace	다른 리소스와 분리한 시험 클러스터에서만
<code>POD_READY_MAX_WAIT_SEC</code> (300초)	main.py	Ready 대기 초과 시 생성 롤백. 배포 브랜치에 따라 값이 다를 수 있음	큰 이미지 첫 기동으로 타임아웃 롤백이 반복될 때 상향
<code>imagePullPolicy</code>	main.py 사용자 Pod 스펙	<code>IfNotPresent</code> — 노드에 이미지가 없을 때만 내려받는다	<code>ErrImagePull</code> 원인을 확인할 때 이미지 이름·태그와 노드의 이미지 목록을 먼저 확인한다
<code>PROM_URL</code>	main.py 상수	노드 선택용 Prometheus 주소	모니터링 스택 주소·포트가 바뀌었을 때
<code>WAS_URL_TEMPLATE</code>	main.py 상수	admin_be 사용자 설정 조회 주소	admin_be 배포 위치(namespace·Service명) 변경 시

키	위치	역할	디버깅 때 언제 바꾸나
<code>HTTP_TIMEOUT_SEC</code> (3.0)	main.py 상수	admin_be·Prometheus 호출 제한 시간	느린 응답으로 502가 날 때 원인을 확인할 때
<code>BASE_ETC_DIR</code> (<code>/kube_share</code>)	main.py 상수	계정 파일(passwd 등) 루트 경로	운영에서는 바꾸지 않는다. 시험 환경에서만 변경한다.
<code>NFS_SERVER</code> / <code>NFS_USER_SHARE_PATH</code>	env ← values.yaml <code>nfs.*</code>	사용자 홈 NFS 마운트 원천	NAS 이전·export 경로 변경 시
<code>KRB5_REALM</code>	env ← values.yaml <code>krb5.*</code>	비우면 Kerberos 설정을 만들지 않는다	KDC 설정을 관리하는 사람과 확인한 뒤 변경
<code>db.*</code> 블록	values.yaml	NodePort 배정 DB(infra-mysql) 접속점	포트 배정 실패 시 접속 대상이 맞는지 확인
<code>redis.*</code> 블록	values.yaml	사용자 이미지 저장/로드 상태 Redis	이미지 상태 조회 실패 시 접속 대상 확인
<code>namespace</code> / <code>nodeSelector</code>	values.yaml	배포 namespace·config-server 배치 노드 고정	고정 노드 점검으로 임시 이전 시 (tolerations 짝 확인)

변경 방법은 재배포 1회이다. values.yaml 값은 `helm upgrade` 로 반영하고, main.py 상수는 이미지 재빌드가 필요하므로 CI/CD(main 머지) 경로를 쓴다.

```
helm upgrade --install containerssh-config-server ./Chart -n ailab-infra
```

10. 하지 않는 작업

- shadow, keytab, SSH 개인키, DB 비밀번호를 화면·로그·티켓·일반 운영 문서에 붙이지 않는다. 관리자 전용 문서 외에는 원문을 남기지 않는다.
- 사용자 확인 없이 Running Pod와 홈을 삭제하지 않는다.
- MySQL NodePort 행만 지우거나 Service만 지우지 않는다. 기록과 실제 리소스가 더 어긋날 수 있다.
- 대체된 ContainerSSH·사용자 PVC chart를 현재 복구 경로로 재배포하지 않는다.
- 레거시 **restart 스크립트**(`restart_auth.sh`·`restart_containerssh.sh`·`restart_pvc.sh`)를 운영 배포에 사용하지 않는다. 전부 레거시 namespace(`containerssh`·`cssh`)를 향하던 것으로 레거시 차트 정리와 함께 레포에서 제거됐다. 과거 체크아웃에 남아 있어도 쓰지 않는다. 운영 서비스는 전부 `ailab-infra`에 있으며, config-server 재배포는 4.1절 CI/CD 또는 `config-server/Makefile`(`make deploy`) 경로를 쓴다.
- 이미지 저장·마이그레이션 경로를 검증 없이 운영 사용자에게 실행하지 않는다(6.5절).
- D-state(디스크 I/O에 걸려 죽지도 살지도 못하는 프로세스 상태)가 있는 FARM 노드에서 NFS 복구를 반복하지 않는다.

11. 참고 문서

- 계정·Pod·포트가 만들어지는 방식 — 시스템 아키텍처
- 배경 용어 — 기초 개념
- API 전체 명세 — API 레퍼런스
- 배포값·Secret 이름·차트 구조 — Helm 차트 레퍼런스
- NodePort 배정 테이블 — 데이터베이스
- FARM AD·Kerberos·NFS 장애 — kdc-setup 운영 문서

API 레퍼런스

이 문서에서 다루는 내용

- config-server HTTP API의 엔드포인트, 호출 주체, 입력, 응답
- 어떤 API가 실제 `admin_be` 에서 호출되고 어떤 API는 수동 확인용인지
- Pod·계정 생성과 삭제 요청에서 주의할 오류 형식

빠른 이동

필요할 때	문서
승인 뒤 내부 처리 순서를 먼저 본다	시스템 아키텍처
운영 중 오류 응답을 해석한다	운영 매뉴얼
Kerberos 준비 실패를 본다	kdc-setup 운영

config-server의 HTTP API 명세다. `main.py` 의 Flask route와 `/accounts` blueprint를 합쳐 총 **12개 엔드포인트**를 제공한다.

현재 배포된 API 목록은 Swagger UI `http://210.94.179.18:30082/apidocs/` 에서도 확인할 수 있다.

1. 공통 사항

항목	값
Base URL (운영)	<code>http://210.94.179.18:30082</code> (namespace <code>ailab-infra</code> , Service <code>containerssh-config-service</code>)

항목	값
요청/응답 형식	JSON (Content-Type: application/json)
인증	없음 — 승인된 운영망 밖으로 노출하지 않는다. 현재 Helm 차트에는 NetworkPolicy가 없으므로 클러스터의 별도 접근 제어가 있는지 확인한다.
에러 응답 형태	Pod 계열은 infra_error() 포맷(step, error, detail, progress 등), 계정 계열은 {"error": "..."} 단순 포맷

호출 주체는 admin_be(Spring Boot WAS)의 서비스 클래스이다. admin_be 실코드 기준으로 확인한 매핑이다.

admin_be 서비스	호출 엔드포인트	시점
AdminRequestCommandService	PUT /accounts/users	관리자 승인 시
PodService	POST /create-pod, POST /delete-pod	승인 시 / 만료 또는 실패를 되돌리는 삭제 시
UbuntuAccountService	DELETE /accounts/users/{username}	만료 또는 실패를 되돌리는 삭제 시
GroupService	PUT /accounts/groups	그룹 생성 시
(호출처 없음 — 미확인)	GET /pods/{username}/status, POST /migrate, GET /accounts/*, DELETE /accounts/groups, PUT /accounts/users/{u}/groups	admin_be 코드에서 호출처가 발견되지 않았다. 운영자 수동 호출·실험용으로 보인다

2. GET /health

항목	값
메서드·경로	GET /health
호출 주체	헬스체크(서버 생존 확인). admin_be 서비스 호출은 없다

입력. 없다.

처리 순서. 즉시 응답한다. 외부 의존성(DB, Kubernetes)을 검사하지 않으므로, 200이 와도 DB 연결 실패 상태일 수 있다.

성공 응답. 200 본문 "OK" (문자열).

대표 실패. 없다(무조건 200).

3. POST /create-pod

항목	값
메서드·경로	POST /create-pod
호출 주체	admin_be PodService (승인 트랜잭션의 마지막 단계)

입력.

```
{"username": "user2100"}
```

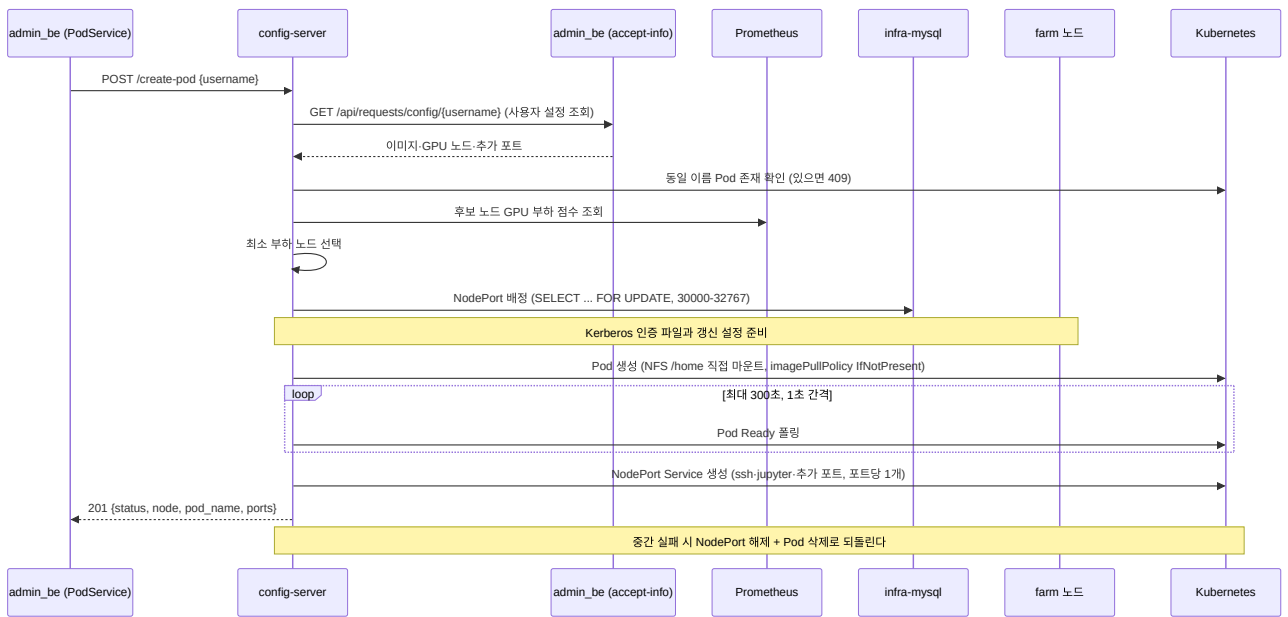
`username` 하나만 받는다. 이미지, GPU 노드, 추가 포트는 config-server가 admin_be에 다시 요청해 가져온다.

처리 순서.

1. `username` 없으면 400을 반환한다.
2. admin_be에 GET `http://admin-prod.default/api/requests/config/{username}` 을 요청해 사용자 설정(이미지, GPU 노드 목록, 추가 포트)을 받아온다. 통신 실패나 JSON 파싱 실패는 502, 사용자가 없으면 404이다.
3. Pod 이름을 `ai lab-<username>-<랜덤>` 형식으로 생성한다.
4. 같은 이름의 Pod가 이미 있으면 409를 반환한다. 같은 username이라도 이름이 다른 Pod는 막지 않으므로, 사용자 단위 중복 호출은 admin_be가 막아야 한다.
5. 후보 노드 목록을 만든다. admin_be가 준 `gpu_nodes` 를 사용하고, 비어 있으면 Ready 상태의 워커 노드 전체를 사용한다.
6. Prometheus(서버 지표를 수집하는 모니터링 시스템)에 각 후보 노드의 GPU 부하를 물어 **가장 한가한 노드**를 고른다.
7. Pod 스펙을 조립한다(`build_pod_spec`). 이 단계 안에서 아래 일이 함께 일어난다.
 - `/kube_share` 계정 파일 확인 — **passwd에 사용자가 없으면 실패**한다(계정 생성이 선행 조건).
 - 저장된 사용자 이미지(`/image-store/images/user-<username>.tar`)가 있으면 로드하고, 없으면 WAS가 준 base 이미지를 쓴다.
 - 기본 포트 22(ssh), 8888(jupyter), 추가 포트에 NodePort를 배정한다. DB에서 30000~32767 중 빈 번호를 고르고 `SELECT ... FOR UPDATE` 로 동시에 같은 번호를 고르지 못하게 한다. 클러스터 전체 Service 조회에 실패하면 DB 기록만으로 배정하므로, 이미 다른 Service가 쓰는 포트를 고르면 뒤의 Service 생성이 실패할 수 있다.
 - 추가 포트에 내부 포트 6080 이 있거나 용도가 `novnc` , `vnc` 이면 Pod 환경 변수에 `ENABLE_VNC=true` 를 넣는다.
 - Kerberos 설정이 켜져 있으면 선택된 FARM 노드에 인증 파일과 갱신 설정을 준비한다. 실패하면 여기서 중단된다. 점검과 복구는 [kdc-setup 운영 문서](#)를 따른다.
 - NFS user-share를 `/home` 에 직접 마운트하도록 볼륨을 구성한다. 사용자 Pod의 `imagePullPolicy` 는 `IfNotPresent` 이므로 노드에 이미지가 없을 때만 레지스트리에서 가져온다.

1. Kubernetes에 Pod를 생성한다.
2. 최대 300초 동안 1초 간격으로 Pod가 Ready 상태가 되기를 기다린다.
3. Ready가 되면 포트마다 NodePort Service(`ai lab-<username>-<용도>-<외부포트>`)를 생성한다.

4. 성공 응답을 반환한다. 8~10단계 어디서든 실패하면 배정한 NodePort 해제 + 만든 Pod 삭제(+Service 삭제)로, 그때까지 만든 것을 되돌린다.



성공 응답. 201

```

{
  "status": "created",
  "node": "farm5",
  "pod_name": "ailab-user2100-1a2b3c4d",
  "ports": [
    {"internal_port": 22, "external_port": 30000, "usage_purpose": "ssh"},
    {"internal_port": 8888, "external_port": 30001, "usage_purpose": "jupyter"}
  ]
}
  
```

대표 실패.

상 태	의미
400	<code>username</code> 누락, 또는 스펙 조립 단계의 검증 실패(알 수 없는 노드, passwd에 사용자 없음 등)
404	WAS에 해당 사용자가 없음 (<code>USER_CONFIG_NOT_FOUND</code>)
409	동일 이름 Pod가 이미 존재 (<code>POD_ALREADY_EXISTS</code>)
502	admin_be 사용자 설정 조회 실패 — 통신 오류·비정상 응답 (<code>USER_CONFIG_FETCH_FAILED</code>)
500	노드 선택·NodePort 배정·Kerberos 준비·Pod 생성·Ready 대기·Service 생성 실패. 응답의 <code>rollback</code> 필드로 어디까지 되돌렸는지 알 수 있다

4. GET /pods/<username>/status

항목	값
메서드·경로	GET /pods/<username>/status
호출 주체	생성 진행 상황을 조회하는 운영자·클라이언트용 API. admin_be 호출처는 미확인

POST /create-pod 는 Pod 준비가 끝날 때까지 최대 300초 동안 기다리는 동기 API이다. 이 API는 생성 요청이 끝나기 전에도 사용자별 진행 상황을 가볍게 조회한다.

입력. 경로의 `username`.

성공 응답. 200

```
{
  "username": "user2100",
  "stage": "waiting_ready",
  "message": "이미지 pull / 컨테이너 기동 대기 중",
  "updated_at": "2026-07-22T01:23:45+00:00"
}
```

stage 는 `unknown`, `started`, `selecting_node`, `building_pod_spec`, `allocating_nodeport`, `deploying_krb5`, `creating_pod`, `waiting_ready`, `creating_services`, `ready`, `failed` 중 하나이다. 생성 이력이 없으면 stage 는 `unknown` 이고 `updated_at` 은 없다. `failed` 의 `message` 에는 보안을 위해 상세 예외나 Kubernetes 오류 원문을 넣지 않으므로, 원인은 config-server 로그에서 확인한다.

대표 실패. 상태 저장소 조회에 실패하면 500 (`POD_STATUS_LOOKUP_FAILED`)을 반환한다.

5. POST /delete-pod

항목	값
메서드·경로	POST /delete-pod
호출 주체	admin_be PodService (만료 스케줄러, 실패를 되돌리는 삭제)

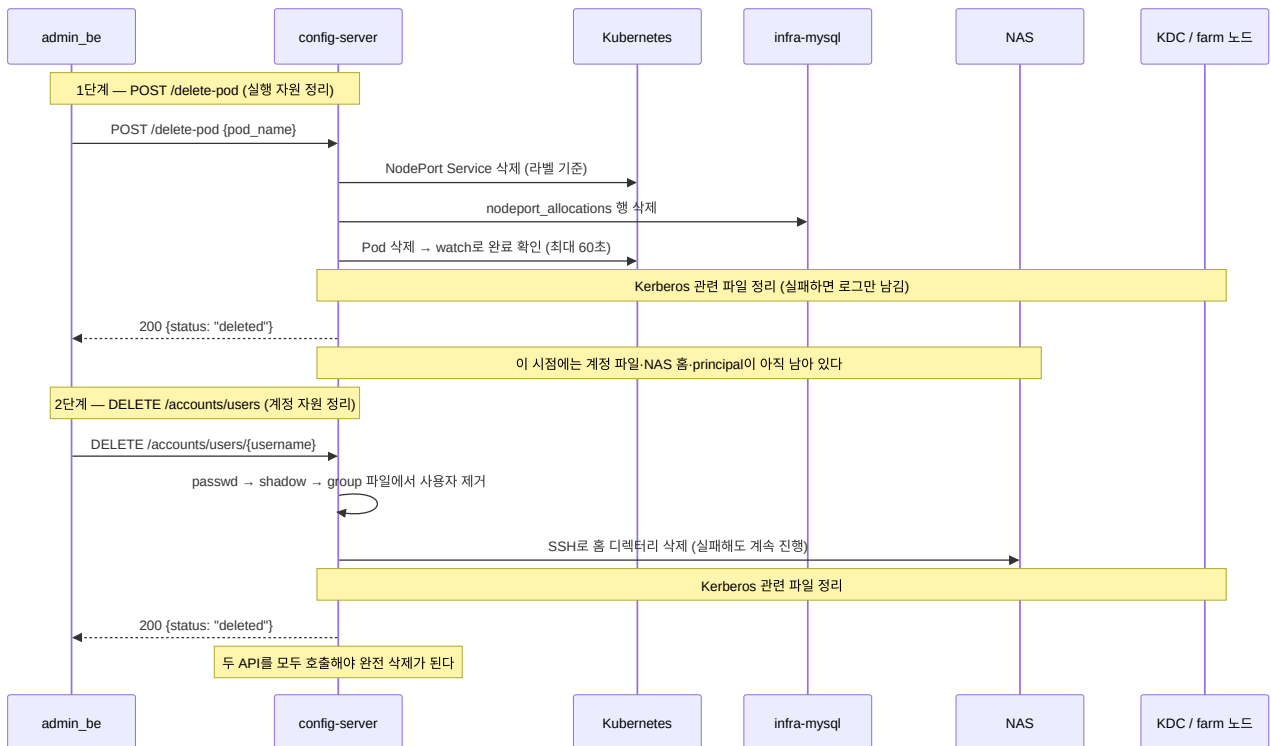
입력.

```
{"pod_name": "ai lab-user2100-1a2b3c4d"}
```

처리 순서.

1. `pod_name` 없으면 400, `ai lab-` prefix가 아니면 400(`INVALID_POD_NAME`)을 반환한다. 이름에서 username을 파싱한다.
2. 해당 Pod의 NodePort Service들을 라벨(`app=ai lab-nodeport, pod_name=<pod>`) 기준으로 삭제한다.
3. `nodeport_allocations` 테이블에서 해당 Pod의 포트 배정 기록 행을 삭제한다.
4. Kubernetes Pod를 삭제하고, watch(쿠버네티스 이벤트 스트림 구독)로 **실제 삭제 완료**를 최대 60초 기다린다. Pod가 원래 없었으면(404) `already_absent: true` 로 200을 반환한다.
5. Kerberos 관련 파일을 정리한다. 이 정리가 실패해도 Pod 삭제 결과는 실패로 바꾸지 않고 로그만 남긴다.

이 API는 Pod, Service, NodePort DB 행만 정리한다. 계정 파일(passwd 등), NAS 홈, Kerberos principal은 그대로 남고, 아래 10절의 `DELETE /accounts/users` 가 정리한다. 두 API 중 하나만 호출하면 Pod 관련 자원이나 계정 관련 파일이 남는다.



성공 응답. 200

```

{
  "status": "deleted",
  "pod_name": "ai lab-user2100-1a2b3c4d",
  "progress": {
    "servicesDeleted": true,
    "nodeportsReleased": true,
    "podDeleteRequested": true,
    "podDeleted": true
  }
}
  
```

`progress` 는 롤백 결과가 아니라 **각 단계의 완료 여부**이다. Pod가 원래 없던 경우 `already_absent: true` 가 추가된다.

대표 실패.

상태	의미
400	<code>pod_name</code> 누락, 또는 <code>ai lab-</code> prefix가 아닌 이름
500	Service 삭제·NodePort 해제·Pod 삭제 실패, 또는 60초 내 삭제 미완료(<code>POD_DELETE_TIMEOUT</code>)

6. POST /migrate

항목	값
메서드·경로	<code>POST /migrate</code>
호출 주체	admin_be 호출처 없음 — 운영자 수동·실험용 (미확인)

입력.

```
{
  "username": "user2100",
  "nodes": ["farm5", "farm6"],
  "min_improvement_ratio": 0.2
}
```

`username` 과 `nodes` (후보 노드 목록)가 필수이고, `min_improvement_ratio` (이만큼은 좋아져야 옮긴다는 개선 문턱, 기본 0.2)는 선택이다.

처리 순서.

- 사용자별 락 파일(`/tmp/migrate-<username>.lock`)을 잡아 같은 사용자의 동시 마이그레이션을 막는다.
- 후보 노드 이름을 실제 클러스터 노드 이름으로 정규화한다. 모르는 노드가 있으면 400이다.
- 실행 중인 사용자 Pod를 찾는다. 없으면 404이다. 현재 노드가 후보 목록에 없으면 400이다.
- 현재 노드를 뺀 후보가 없으면 `skipped` (`no_candidate_node`)로 200을 반환한다.
- Prometheus GPU 점수를 현재 노드와 후보들에 대해 계산한다. 최고 후보가 `min_improvement_ratio` 만큼 충분히 좋지 않으면 `skipped` (`no_significant_improvement`)로 200을 반환한다.
- WAS에서 사용자 설정을 다시 조회한다.
- 기존 Pod 안에서 이미지 commit/save를 실행해 사용자 상태를 image-store에 저장한다. 실패하면 500이다.
- 새 Pod 이름을 만들고, 새 노드에 Pod를 생성한 뒤 Ready를 최대 60초 기다리고 NodePort Service를 만든다. 실패하면 새 Pod와 새로 배정한 포트를 정리하고 500이다.
- 새 Pod가 완전히 성공한 뒤에야 기존 Pod의 Service·포트 배정·Pod를 삭제한다.

성공 응답. `200`


```
{
  "status": "migrated",
  "from": "farm5",
  "to": "farm6",
  "new_pod": "ailab-user2100-9z8y7x6w",
  "ports": [{"internal_port": 22, "external_port": 30005, "usage_purpose": "ssh"}]
}
```

건너뛴 경우에도 200이며 `{"status": "skipped", "reason": ...}` 형태이다.

대표 실패.

상태	의미
400	<code>username</code> / <code>nodes</code> 누락, 알 수 없는 노드, 현재 노드가 후보 목록에 없음
404	실행 중인 Pod 없음
500	이미지 commit 실패(<code>image_commit_failed</code>), 새 Pod 기동 실패, Service 생성 실패

7. GET /accounts/users

항목	값
메서드 · 경로	<code>GET /accounts/users</code>
호출 주체	admin_be 호출처 없음 — 운영자 점검용 (미확인)

입력. 없다.

처리 순서. NFS 계정 파일 `/kube_share/passwd` (리눅스 계정 목록 파일)를 읽어 전체 사용자를 반환한다.

성공 응답. 200

```
{
  "users": [
    {
      "name": "user2100",
      "uid": 20001,
      "gid": 20001,
      "gecos": "GPU User",
      "home": "/home/user2100",
      "shell": "/bin/bash"
    }
  ]
}
```

대표 실패. 파일 읽기 예외 시 500(`{"error": ...}`).

8. GET /accounts/users/<username>

항목	값
메서드·경로	GET /accounts/users/<username>
호출 주체	admin_be 호출처 없음 — 운영자 점검용 (미확인)

입력. 경로의 `username`.

처리 순서.

- passwd에서 사용자를 찾는다. 없으면 404이다.
- group 파일을 읽어 기본 그룹과 추가 그룹을 함께 반환한다.

성공 응답. 200

```
{
  "user": {"name": "user2100", "uid": 20001, "gid": 20001, "gecos": "", "home": "/home/user2100", "shell":
"/bin/bash"},
  "groups": [
    {"name": "user2100", "gid": 20001, "type": "primary"},
    {"name": "developers", "gid": 20005, "type": "supplementary"}
  ]
}
```

대표 실패.

상태	의미
404	사용자 없음
500	파일 읽기 예외

9. PUT /accounts/users

항목	값
메서드·경로	PUT /accounts/users
호출 주체	admin_be AdminRequestCommandService (승인 시)

입력.

```
{
  "name": "user2100",
  "passwd_base64": "cGFzc3dvcmQ=",
  "gecos": "GPU User",
  "primary_group_name": "user2100",
  "supplementary_groups": [{"name": "ailab", "gid": 20005}]
}
```

필수는 `name`, `passwd_base64` (Base64로 감싼 평문 비밀번호)이다. 나머지는 선택이다.

이전 문서에는 `uid`, `gid`, `passwd_sha512` 가 필수라고 적혀 있지만 현재 코드와 다르다. UID/GID는 서버가 빈 번호를 골라 정하고, 비밀번호 해시(SHA-512 crypt)도 서버가 만든다.

처리 순서.

1. 필수 필드와 `supplementary_groups` 형식(`{name, gid}` 목록)을 검증한다. `passwd_base64` 디코딩에 실패하면 400이다.
2. `/kube_share` 계정 파일 구조가 비어 있으면 `base_etc/` 템플릿으로 초기화한다.
3. `passwd` 파일을 잠가 다른 요청이 동시에 같은 UID를 고르지 못하게 한다. NFS 파일에는 잠금을 걸지 않고 로컬 `/tmp` 잠금 파일을 쓴다. 사용자가 이미 있으면 409를 반환한다. 없으면 20000 이상이고 `/home/` 을 쓰는 사용자 중 가장 큰 UID 다음 번호를 찾아 `passwd`에 추가한다. GID는 UID와 같은 값이다.
4. `group` 파일에 기본 그룹을 만들고, 요청에 든 추가 그룹에는 사용자를 구성원으로 넣는다. 실패하면 `passwd`에 쓴 내용도 되돌리고 500을 반환한다.
5. `shadow`(암호 해시 파일)에 SHA-512 crypt 해시를 기록한다. 실패하면 롤백 후 500이다.
6. `SUDO_ALLOWED_COMMANDS` 설정이 있으면 사용자별 `sudoers`(sudo 사용 권한을 정의하는 파일)를 0440 권한으로 만든다.
7. NAS(`192.168.2.30:6954`)에 SSH로 접속해 홈 디렉터리를 만든다(`mkdir`, `chown`, `chmod 700`). NFS의 `root_squash` 설정 때문에 Pod는 홈 디렉터리를 직접 만들 수 없다. 실패하면 계정 파일을 되돌리고 500을 반환한다.
8. Kerberos 설정이 켜져 있으면 사용자 `principal`과 `keytab Secret`을 만든다. 실패하면 홈 디렉터리를 지우고 계정 파일을 되돌린 뒤 500을 반환한다. 점검과 복구는 [kdc-setup 운영 문서](#)를 따른다.

성공 응답. 201

```
{
  "status": "created",
  "user": {"name": "user2100", "uid": 20001, "gid": 20001, "home": "/home/user2100", "shell": "/bin/bash"},
  "group": {"name": "user2100", "gid": 20001},
  "supplementary_groups": [{"name": "ailab", "gid": 20005}],
  "sudoers": "/kube_share/sudoers.d/user2100"
}
```

대표 실패.

상 태	의미
400	필수 필드 누락, <code>passwd_base64</code> 디코딩 실패, <code>supplementary_groups</code> 형식 오류
409	사용자 이미 존재
500	<code>group/shadow/sudoers</code> 기록 실패, NAS SSH 실패(<code>NAS_SSH_FAILED</code>), KDC 실패(<code>KDC_FAILED</code>) — 모두 계정 파일을 되돌린 뒤 반환한다

10. DELETE /accounts/users/<username>

항목	값
메서드 · 경로	<code>DELETE /accounts/users/<username></code>
호출 주체	<code>admin_be UbuntuAccountService</code> (만료, 실패를 되돌리는 삭제)

입력. 경로의 `username`.

처리 순서.

1. `passwd`에서 사용자를 제거한다. 없으면 404이다.
2. `shadow`에서 해당 행을 제거한다.
3. `group` 파일을 정리한다 — 모든 그룹의 멤버 목록에서 사용자를 빼고, 이 사용자가 쓰던 그룹(명시적 멤버였거나 primary GID 그룹)이 비게 되면 그룹 자체를 삭제한다.
4. NAS 홈 디렉터리를 SSH로 삭제한다. **실패해도 경고 로그만 남기고 계속 진행한다**(계정 파일은 이미 지워진 상태).
5. Kerberos 관련 파일을 정리한다. 이 단계가 실패하면 오류를 기록하고 다음 작업을 계속한다.

`POST /delete-pod`도 함께 호출해야 Pod와 계정이 모두 정리된다(5절의 시퀀스 다이어그램 참조). 이 API만 호출하면 Pod와 NodePort가 남고, `delete-pod`만 호출하면 계정, 홈, principal이 남는다.

성공 응답. `200 — {"status": "deleted", "user": "user2100"}`

대표 실패.

상태	의미
404	사용자 없음
500	Kerberos 정리 등 처리 중 예외 (NAS 홈 삭제 실패는 500이 아니라 경고 후 진행)

11. PUT /accounts/groups

항목	값
메서드·경로	PUT /accounts/groups
호출 주체	admin_be GroupService

입력.

```
{"name": "developers", "gid": 20005, "members": ["user2100", "user2101"]}
```

필수는 **name** 이다. **gid** 는 생략하면 group 파일 기준으로 빈 번호를 골라 자동 배정(20000 이상)하고, **members** 는 생략 가능하다.

처리 순서.

1. **gid** 타입을 검증한다(정수 또는 정수 문자열만 허용, 아니면 400).
2. **members** 에 적힌 사용자가 passwd에 전부 존재하는지 확인한다. 없는 사용자가 있으면 400이다.
3. group 파일에 락을 걸고 — 같은 이름이 있으면 409, 지정한 gid가 이미 쓰이면 409, 아니면 새 그룹 행을 추가한다.

성공 응답. 201 — {"group": {"name": "developers", "gid": 20005}}

대표 실패.

상태	의미
400	name 누락, gid 타입 오류, 존재하지 않는 멤버
409	그룹 이름 또는 gid 중복

12. DELETE /accounts/groups/<groupname>

항목	값
메서드·경로	DELETE /accounts/groups/<groupname>
호출 주체	admin_be 호출처 없음 (미확인)

입력. 경로의 **groupname** .

처리 순서.

1. group 파일에서 그룹을 찾는다. 없으면 404이다.

- 이 그룹을 기본 그룹으로 쓰는 사용자가 passwd에 있으면 삭제를 거부하고 400을 반환한다. 오류 응답에는 해당 사용자 이름이 들어간다.
- 그룹 행을 삭제한다.

성공 응답. 200 — {"status": "deleted", "group": "developers", "gid": 20005}

대표 실패.

상태	의미
400	기본 그룹으로 사용 중
404	그룹 없음

13. PUT /accounts/users/<username>/groups

항목	값
메서드·경로	PUT /accounts/users/<username>/groups
호출 주체	admin_be 호출처 없음 (미확인)

입력.

```
{"groups": ["developers", "ai-lab"]}
```

처리 순서.

- groups 목록이 비어 있으면 400이다.
- 사용자가 passwd에 없으면 404이다.
- 지정한 그룹들의 멤버 목록에 사용자를 추가한다(이미 있으면 그대로 둔다).
- 요청한 그룹 중 존재하지 않는 것이 있으면 404를 반환한다.

성공 응답. 200 — {"status": "updated", "user": "user2100", "groups": ["ai-lab", "developers"]}

대표 실패.

상태	의미
400	groups 누락
404	사용자 없음, 또는 존재하지 않는 그룹 포함

14. 호출할 때 확인할 점

- 위 12개 API는 요청자를 확인하지 않는다. 승인된 운영망 밖에서 호출할 수 없게 해야 한다. 현재 Helm 차트는 NetworkPolicy를 만들지 않으므로, 네트워크 접근 제어는 클러스터 설정에서 따로 확인한다.
- `create-pod`를 같은 사용자에게 여러 번 호출하면 Pod와 Service가 여러 개 만들어질 수 있다. 사용자 단위 중복 호출은 admin_be가 막아야 한다.
- `POST /delete-pod` (Pod, Service, 포트)와 `DELETE /accounts/users` (계정, 홈, principal)를 모두 호출해야 관련 자원이 남지 않는다.
- 8888 포트도 다른 포트처럼 NodePort Service로 열린다. 게스트 이미지의 Jupyter에 인증이 없으면 외부에서 접속할 수 있으므로 내부망에서만 접근하게 해야 한다.
- 만료 스케줄러가 이 API들을 어떤 순서로 부르는지는 [운영 가이드](#)의 만료 흐름 절을 참고한다.

Helm 차트 레퍼런스

이 문서에서 다루는 내용

- config-server Helm 차트가 어떤 리소스를 만드는지
- `values.yaml`과 템플릿이 Deployment, Service, RBAC, CronJob에 어떻게 반영되는지
- 배포 값을 바꾸기 전에 어디를 확인해야 하는지

빠른 이동

필요할 때	문서
Kubernetes 기본 용어를 먼저 본다	기초 개념
배포와 장애 대응 순서를 본다	운영 매뉴얼
Kerberos 설정 값의 위치를 본다	kdc-setup 설정

쿠버네티스 기본 용어(Pod, Service, PVC, namespace 등)는 [기초 개념](#)을 참고한다.

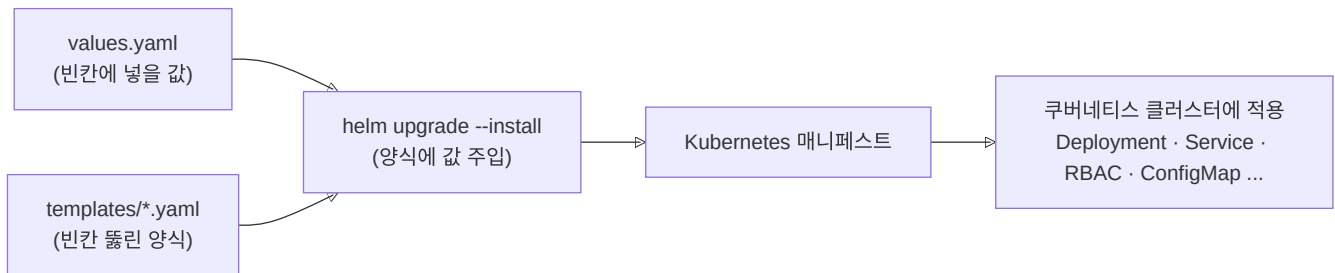
0. Helm 기본 개념

0.1 왜 Helm을 쓰는가

쿠버네티스에 무언가를 배포하려면 YAML 파일(매니페스트)을 여러 장 써야 한다. config-server 하나만 해도 Deployment, Service, 권한(RBAC), 설정(ConfigMap) 등 파일이 많다. 문제는 이 파일들 곳곳에 **배포 환경마다 달라지는 값**(이미지 태그, NFS 서버 주소, 포트 번호)이 박혀 있다는 점이다. 값이 바뀔 때마다 여러 파일을 뒤져 고치는 것은

실수하기 좋다.

- **템플릿(templates/)** — 매니페스트의 구조를 적어 둔 양식이다. 달라지는 자리는 `{{ .Values.image.tag }}` 처럼 빈칸으로 뚫어 둔다.
- **values.yaml** — 그 빈칸에 채워 넣을 값들의 모음이다. "이번 배포의 설정 전체"가 이 파일 한 장에 모인다.
- **차트(Chart)** — 템플릿 묶음 + values.yaml + 메타데이터(Chart.yaml)를 합친 패키지이다.
- **릴리스(release)** — 차트를 클러스터에 설치한 결과물의 이름이다. 같은 차트를 이름만 다르게 여러 번 설치할 수도 있다. 이 시스템의 릴리스 이름은 `containerssh-config-server` 이다.



배포 명령(`helm upgrade`)을 실행하면 Helm이 값을 양식에 넣어 Kubernetes 매니페스트를 만들고 클러스터에 적용한다. 예를 들어 NFS 서버 주소가 바뀌면 템플릿을 고치지 않고 values.yaml의 값을 바꾸면 된다.

0.2 자주 쓰는 명령

```
# 배포(설치+업데이트 겸용) — ./Chart 디렉터리의 차트를 릴리스에 반영한다
helm upgrade --install containerssh-config-server ./Chart -n ailab-infra

# 새 Pod가 정상적으로 떴는지 확인한다
kubectl rollout status deploy/containerssh-config-server -n ailab-infra

# 잘못됐으면 직전 배포 상태로 되돌린다 (Helm이 릴리스 이력을 기억한다)
helm rollback containerssh-config-server -n ailab-infra
```

`upgrade --install` 은 "릴리스가 없으면 설치하고, 있으면 갱신하라"는 겸용 명령이라 설치와 업데이트를 구분해 외울 필요가 없다. `rollback` 이 가능한 이유는 Helm이 릴리스마다 배포 이력을 버전으로 쌓아 두기 때문이다.

0.3 --set 으로 바꾼 값

`helm upgrade ... --set image.tag=abc123` 처럼 명령어 뒤에 값을 직접 붙이면 values.yaml을 고치지 않고 그 배포 한 번에만 값을 바꿀 수 있다. 다음 배포 때 values.yaml의 값으로 돌아가므로, 계속 유지할 변경은 values.yaml이나 별도 운영 values 파일에 기록한다.

1. Chart/ — config-server 차트 (현행)

config-server 배포용 Helm 차트이다. 위치는 저장소의 `config-server/Chart/` 이다.

1.1 이 차트가 클러스터에 만드는 것

리소스	이름	왜 필요한가
Deployment	containerssh-config-server	config-server 컨테이너 1개(replicas: 1)를 실행한다. 15초 뒤부터 10초마다 /health 를 확인해 Ready 상태를 판단한다.
Service (NodePort)	containerssh-config-service	config-server의 고정 접속 주소이다. 내부 80 → 컨테이너 8000(gunicorn), 외부 NodePort 30082
ServiceAccount	config-server	config-server가 Kubernetes API를 호출할 때 사용하는 계정이다.
Role + RoleBinding	config-server-role / config-server-rolebinding	ailab-infra namespace 안에서 Pod(exec·log 하위 리소스 포함)·Service·PVC·Secret을 만들고 지을 권한을 위 ServiceAccount에 붙인다. 사용자 Pod 생성·삭제뿐 아니라 Pod 안에서 명령 실행(exec)·로그 조회까지 이 권한으로 이뤄진다
ClusterRole + Binding	...-node-reader	노드 목록 조회 권한이다. 노드는 namespace에 속하지 않는(cluster-scoped) 리소스라서 Role로는 못 주고 ClusterRole이 따로 필요하다
ConfigMap	krb5-conf	Kerberos 클라이언트 설정(krb5.conf)을 담아 config-server Pod에 주입한다
CronJob	...-krb5-reconcile	30분마다 reconcile_krb5.py 를 실행한다. krb5_cleanup_pending 에 남은 실패 건을 다시 정리하고, FARM 노드의 keytab 목록과 nodeport_allocations 를 비교해 Pod가 없는 사용자의 keytab도 지운다.

1.2 파일 구성

파일	역할
Chart.yaml	차트 이름(containerssh-config-server)·버전 메타데이터이다
values.yaml	배포마다 달라지는 설정값 모음이다(1.3절)
templates/deployment.yaml	Deployment 양식이다 — 이미지·환경변수·마운트가 여기서 조립된다. values의 값 대부분이 컨테이너 환경변수로 주입된다
templates/service.yaml	NodePort Service 양식이다
templates/rbac.yaml	Role/RoleBinding + ClusterRole/ClusterRoleBinding 양식이다
templates/serviceaccount.yaml	ServiceAccount 양식이다
templates/configmap.yaml	krb5.conf ConfigMap 양식이다
templates/cronjob-krb5-reconcile.yaml	30분마다 Kerberos 정리 재시도와 남은 keytab 확인을 실행하는 CronJob 설정이다.
templates/_helpers.tpl	리소스 이름 조립 헬퍼이다 — 릴리스 이름을 그대로 리소스 이름으로 쓴다

값은 `values.yaml` 에서 `deployment.yaml` 템플릿의 `env` 를 거쳐 컨테이너 환경 변수로 들어간다. `main.py`, `utils.py` 는 이 환경 변수를 읽는다. 예를 들어 `nas.ssh.host` 는 `NAS_SSH_HOST` 가 되어 NAS SSH 접속에 사용된다.

1.3 values.yaml에 넣는 값

아래 기본값은 저장소 `values.yaml` 을 기준으로 적었다. `""` (빈 값)인 키는 실제 배포 때 별도 values 파일이나 `--set` 으로 넣는다. 저장소에는 주소·계정 같은 민감값을 두지 않는다.

image — 무엇을 배포하나

CI/CD가 새 이미지를 push한 뒤 이 값 기준으로 배포된다.

키	의미	기본값
<code>image.repository</code>	운영 이미지 저장소	<code>dguai lab/config-server</code>
<code>image.tag</code>	이미지 태그 (CI가 latest + commit hash 이중 태깅)	<code>latest</code>
<code>image.pullPolicy</code>	이미지를 매번 새로 받을지 정하는 정책	<code>Always</code>

service — 어떻게 접속하나

`admin_be`와 관리자가 이 포트로 API를 호출한다.

키	의미	기본값
<code>service.type</code>	Service 타입	<code>NodePort</code>
<code>service.port</code>	클러스터 내부 포트	<code>80</code>
<code>service.targetPort</code>	컨테이너 포트 (gunicorn)	<code>8000</code>
<code>service.nodePort</code>	외부 노출 NodePort	<code>30082</code> (README의 9732는 오기, kubectl 실측)

resources — config-server 자신의 몫

사용자 Pod의 리소스가 아니라 **관리 서버 본인**이 쓸 CPU/메모리이다. `requests`는 스케줄링 시 보장받는 최소치, `limits`는 상한이다.

키	의미	기본값
<code>resources.requests</code>	CPU/메모리 요청	<code>500m</code> / <code>512Mi</code>
<code>resources.limits</code>	CPU/메모리 상한	<code>2000m</code> / <code>1024Mi</code>

nfs / nas / farm — 연결할 서버 설정

사용자 홈 NFS 주소, NAS SSH 접속 정보, `keytab`을 배포할 FARM 노드 목록을 정한다.

키	의미	기본값
<code>nfs.server</code>	NFS 서버 주소	<code>""</code> (배포 시 주입)

키	의미	기본값
<code>nfs.userSharePath</code>	사용자 홈 루트 export 경로	""
<code>nfs.kubeSharePath</code>	계정 파일(<code>/kube_share</code>) export 경로 — config-server Pod 자신이 이 NFS를 마운트한다	""
<code>nas.ssh.host</code> / <code>port</code> / <code>user</code> / <code>keyPath</code>	NAS 홈 생성용 SSH 접속 정보 (키 실체는 Secret, 1.4절)	""
<code>farm.ssh.user</code> / <code>keyPath</code> / <code>nodes</code>	farm 노드 keytab 배포용 SSH 정보와 대상 노드 목록	"" / []
<code>farm.adSsh.user</code> / <code>keyPath</code> / <code>nodes</code>	AD/DC(krb5 principal 발급) 노드용 SSH 정보	"" / []
<code>farm.homeMountRoot</code>	노드 호스트에 마운트된 홈 루트 — 사용자 Pod <code>/home</code> 의 hostPath 원천이다 (시스템 아키텍처의 사용자 Pod 그림)	""

security / krb5 — 권한과 인증

키	의미	기본값
<code>security.sudoAllowedCommands</code>	사용자별 sudoers whitelist 명령 목록. 비어 있으면 sudoers 파일 자체를 만들지 않는다	""
<code>krb5.realm</code>	Kerberos realm(인증 도메인 이름) — 운영은 <code>AILAB.DGU</code> 이다	""
<code>krb5.kdcHost</code>	KDC(티켓 발급 서버) 호스트. 기본 <code>values.yaml</code>과 현재 CI 배포 명령에는 이 값이 없다. ConfigMap 템플릿은 이 값을 읽으므로, 현재 배포 결과의 <code>krb5.conf</code> 에는 <code>kdc</code> 와 <code>admin_server</code> 값이 비어 있다. 이 ConfigMap을 실제 Kerberos 클라이언트 설정으로 쓸 계획이면 <code>values</code> 와 CI에 주입 경로를 추가해야 한다.	(키 없음)

`krb5.realm`이 비어 있으면 config-server는 사용자 keytab과 ccache를 준비하지 않는다. NAS의 Kerberos NFS 설정에 이 값이 필요하면 새 Pod가 홈에 접근하지 못할 수 있다. 실제 NFS mount 옵션은 FARM 노드에서 확인한다.

db / redis — 상태 저장소

NodePort 배정 기록은 MySQL에, 이미지 저장·로드 상태는 Redis에 둔다(시스템 아키텍처의 NodePort 배정 방식).

키	의미	기본값
<code>db.host</code>	infra-mysql 호스트	<code>infra-mysql</code>
<code>db.name</code>	데이터베이스명	<code>pod_port_db</code>
<code>db.user</code>	DB 사용자	<code>pod_port_user</code>
<code>redis.host</code>	이미지 상태용 Redis	<code>redis-bg-master.ailab-infra.svc.cluster.local</code>
<code>redis.port</code>	Redis 포트	<code>6379</code>

DB 비밀번호는 values에 없다 — Secret `config-server-db-secret` 에서 주입된다(1.4절).

배치 관련 — config-server가 어디에 뜨나

키	의미	기본값
<code>namespace / config.namespace</code>	배포 대상 namespace. ConfigMap 템플릿은 <code>.Values.namespace</code> 를 읽고 나머지 템플릿은 <code>.Values.config.namespace</code> 를 읽는다. 두 값은 항상 같게 둔다.	<code>ailab-infra</code>
<code>nodeSelector</code>	config-server 배포 노드 고정	<code>kubernetes.io/hostname: csid-dgu-desktop</code>
<code>tolerations</code>	control-plane의 NoSchedule taint 허용. taint는 노드에 걸린 "여기 배치 금지" 표식이고, toleration은 그 금지를 통과하는 허가증이다. 고정 대상 노드가 control-plane이라 nodeSelector와 반드시 짝으로 있어야 Pod가 뜬다	<code>node-role.kubernetes.io/control-plane</code>
<code>cleanup.image</code>	현재 템플릿에서는 읽지 않는 값이다. 지워도 배포 결과는 달라지지 않는다.	<code>config-server:latest</code>

1.4 미리 만들어 둘 Secret

민감값(비밀번호, SSH 개인키)은 values.yaml에 넣지 않고 쿠버네티스 Secret으로 관리한다. Secret은 차트가 만들어 주지 않으므로 **클러스터에 미리 존재해야** 하며, 없으면 Pod가 뜨지 못한다.

Secret 이름	내용	소비처
<code>config-server-db-secret</code>	DB 비밀번호 (key: <code>password</code>)	Deployment-CronJob의 <code>DB_PASSWORD</code> 환경변수
<code>nas-ssh-key</code>	NAS 접속용 SSH 개인키	<code>/etc/nas-ssh</code> 에 read-only 마운트
<code>farm-ssh-key</code>	farm 노드 접속용 SSH 개인키	<code>/etc/farm-ssh</code> 에 read-only 마운트
<code>farm-ad-ssh-key</code>	AD/DC 노드 접속용 SSH 개인키	<code>/etc/farm-ad-ssh</code> 에 read-only 마운트

이 외에 config-server는 사용자별 `krb5-keytab-<user>` Secret을 만든다. `rbac.yaml` 의 Secret 생성 권한은 이 작업에 필요하다.

1.5 값을 바꿀 때

먼저 위치부터 잡는다. `config-server/` 는 GitHub 저장소(CSID-DGU/admin_infra) 안의 폴더이다. 저장소를 내려받아(`git clone https://github.com/CSID-DGU/admin_infra.git`) 그 안의 `config-server/` 로 이동한 상태에서 아래를 실행한다. 단, ②~④는 클러스터에 연결된 관리 서버(kubectl과 helm이 설정된 컴퓨터)에서만 동작한다. 개인 노트북에서 clone만 받아서는 ①(파일 수정)까지만 가능하다.

기본 `values.yaml` 에는 NFS·NAS·FARM·Kerberos의 실제 값이 비어 있다. 운영에서 수동으로 `helm upgrade` 를 실행할 때는 CI가 넣는 것과 같은 별도 values 파일 또는 `--set` 값을 함께 사용한다. 빈 기본값만으로 아래 명령을 실행하지 않는다.

```
# ① values.yaml에서 값을 수정한다
vi Chart/values.yaml

# ② 릴리스에 반영한다 (재배포)
helm upgrade --install containerssh-config-server ./Chart -n ailab-infra

# ③ 새 Pod가 정상적으로 떴는지 확인한다
kubectl rollout status deploy/containerssh-config-server -n ailab-infra

# ④ 잘못됐으면 직전 배포 상태로 되돌린다
helm rollback containerssh-config-server -n ailab-infra
```

반영 뒤에는 [운영 매뉴얼](#)의 배포 확인 절차를 따른다. 일회성 실험에는 `--set` 을 쓸 수 있지만, 계속 유지할 값은 0.3절처럼 파일에 남긴다.

2. pvc-image-chart/ — 이미지 저장소 (현행)

사용자 컨테이너 이미지를 tar로 보관하는 공용 PVC `pvc-image-store` 를 배포하는 작은 차트이다. **현행 구조에서 이 PVC를 마운트하는 것은 config-server뿐이다**(`/image-store` — 사용자별 이미지 저장/로드 경로는 `/image-store/images/user-<username>.tar`). 과거에는 게스트 Pod에도 붙였지만 마운트 실패 문제로 사용자 Pod 스펙에서는 제거됐다. 사용자별 홈 PVC가 폐기된 뒤에도 **image-store만 PVC로 남았다**([시스템 아키텍처](#)의 홈 디렉터리와 이미지 저장소).

키/장치	값	의미
<code>imageStore.size</code>	<code>500Gi</code>	PVC 요청 용량
<code>storageClass</code>	<code>nfs-nas-v3-expandable</code>	NFS CSI 기반 StorageClass — 어떤 방식으로 저장소를 만들지 정한 템플릿
<code>accessModes</code>	<code>ReadWriteMany</code>	여러 Pod가 동시에 마운트할 수 있는 모드. 현행 소비자는 config-server이지만, 이미지 저장소를 공유 스토리지로 유지하는 계약을 나타낸다
<code>helm.sh/resource-policy: keep</code>	<code>annotation</code>	릴리스를 지워도 PVC는 남긴다. 차트를 실수로 삭제해도 이미지 저장소 PVC가 함께 지워지지 않게 한다.

3. pvc-chart/ (폐기)

사용자/그룹별 홈 PVC를 동적 생성하던 Helm 차트(`containerssh-pvc-chart`)의 잔재이다. 개별 사용자 PVC 방식은 26-07-02에 폐기되어 현행 흐름에서는 사용하지 않는다(현행은 NFS 직접 마운트 — [시스템 아키텍처](#)의 홈 디렉터리와 이미지 저장소 참조).

이 디렉터리는 저장소에 남아 있지만 현재 배포에는 쓰지 않는다. `Chart.yaml`, `values.yaml`, `README.md`, StorageClass 매니페스트 두 장(`sc-nfs-nas-v3-resizable.yaml`, `old-sc.yaml`)은 이전 클러스터 상태를 확인할 때만 본다. 새로 배포하거나 복구할 때는 이 차트를 쓰지 않고 NFS 직접 마운트와 `pvc-image-chart/`를 사용한다.

4. infra-sql/ (현행)

config-server의 NodePort 배정 기록 등 인프라 상태를 저장하는 MySQL을 배포하는 **일반 매니페스트**다. Helm 차트가 아니라 `kubectl apply`로 YAML을 적용한다.

파일	역할
<code>nfs-mysql.yaml</code>	MySQL용 NFS CSI StorageClass <code>sc-mysql</code> 을 만든다 (NFS server <code>192.168.2.30</code> , share <code>/volume1/share</code> , reclaimPolicy <code>Retain</code> — PVC를 지워도 데이터 볼륨을 삭제하지 않고 남긴다)
<code>infra-mysql.yaml</code>	<code>ailab-infra</code> namespace에 <code>infra-mysql</code> StatefulSet(고정 이름과 저장소를 유지하는 Pod 관리 객체, 저장소 10Gi) + ClusterIP Service(클러스터 내부 전용 접속 주소)를 생성한다

DB에는 StatefulSet을 쓴다. Deployment와 달리 Pod가 다시 만들어져도 같은 이름과 저장소(PVC)를 다시 사용하므로 데이터가 이어진다. Service는 ClusterIP(클러스터 내부 전용)다. NodePort 배정 기록 DB를 클러스터 밖에 열지 않는다.

infra-mysql은 `nodeport_allocations`와 `krb5_cleanup_pending` 테이블을 자동으로 만들지 않는다. DB를 다시 만들면 두 테이블의 생성 SQL을 직접 실행해야 한다. 스키마는 [데이터베이스](#)를 참고한다.

kdc-setup

원본: md/infra/kdc-setup/index.md, md/infra/kdc-setup/design.md, md/infra/kdc-setup/operations.md, md/infra/kdc-setup/config.md

이 문서에서 다루는 내용

- config-server가 FARM AD 계정, 사용자 keytab, 노드 ccache, Kerberos NFS 홈에 쓰는 흐름 전체
- 이 묶음 안에서 설계, 운영, 설정 문서를 어떤 순서로 읽으면 되는지
- `system/kerberos-nfs` 문서와 여기 문서의 경계

빠른 이동

필요할 때	문서
파일과 단계의 설계를 본다	설계
점검과 장애 대응 순서를 본다	운영
Helm, CI, Secret 값의 위치를 본다	설정
NAS와 NFS 쪽 Kerberos 설정을 본다	System Kerberos/NFS

`kdc-setup` 은 config-server가 사용자 Pod를 만들 때 쓰는 Kerberos 설정 문서다. NAS service keytab, NAS KVNO, 다른 컨테이너 환경의 Kerberos 설정은 [System Kerberos/NFS](#)에서 다룬다.

문서 안내

문서	읽어야 하는 경우	핵심 내용
현재 페이지	문서의 역할을 먼저 볼 때	AD, Secret, 노드, Pod가 이어지는 순서
설계	계정과 Pod를 만들 때 어떤 파일이 필요한지 볼 때	AD, Secret, 선택 노드, ccache, NFS
운영	점검, 장애 대응, 노드 추가를 할 때	확인 순서와 명령
설정	Helm, CI, Secret 설정을 바꿀 때	값이 들어가는 곳과 확인 방법

계정부터 Pod까지의 순서

승인 시스템

- > config-server
- > FARM AD DC: 사용자·전용 그룹·RFC2307·keytab
- > Kubernetes Secret: 사용자 keytab 보관
- > 선택된 FARM 노드: root-only keytab·refresh timer·ccache
- > 사용자 Pod: keytab 없이 ccache와 FARM 홈을 사용

누가 무엇을 하나

- config-server는 계정과 Pod 생성·삭제를 요청하고 AD와 노드 관리 명령을 실행한다.
- AD는 사용자, 전용 그룹, RFC2307 UID/GID, 사용자 keytab을 만든다.
- 선택된 노드는 keytab을 root만 읽을 수 있는 경로에 두고 사용자 ccache를 갱신한다.
- Pod는 keytab을 받지 않고 ccache만 사용한다.
- NAS 홈 디렉터리 생성·삭제는 NAS 관리 경로에서 처리한다.
- 비밀 값과 변경 이력은 관리자 전용 시크릿과 배포 설정 문서에만 둔다.

이전 Pod 처리

새 계정과 새 Pod는 FARM AD 설정을 사용한다. 전환 전에 만든 Pod는 다시 만들기 전까지 이전 Realm이나 홈 마운트를 쓸 수 있다. 이전 Pod에 새 keytab이나 timer 파일을 덮어쓰지 말고, 현재 Pod의 환경 변수와 hostPath를 확인한 뒤 다시 만든다.

관련 문서

- [System Kerberos/NFS](#): NAS와 NFS의 Kerberos 설정
- [System container-images](#): 이미지가 ccache를 쓰는 방식
- [설정](#): config-server 배포 설정과 확인 방법

kdc-setup 설계

이 문서에서 다루는 내용

- AD 사용자, Secret, 노드 keytab, ccache가 어디에 놓이고 누가 쓰는지
- 계정 생성, Pod 시작, Pod·계정 삭제 때 Kerberos 관련 파일이 어떻게 바뀌는지
- 바꾸면 안 되는 보안·운영 규칙

빠른 이동

필요할 때	문서
전체 흐름을 먼저 본다	개요
점검과 장애 대응을 본다	운영
실제 설정값의 위치를 본다	설정

1. 이 설정에서 지킬 것

- FARM AD의 RFC2307 UID/GID와 NAS의 숫자 소유권을 같은 값으로 유지한다.
- 사용자 keytab은 Pod에 전달하지 않고, 선택 노드의 root 전용 경로에만 둔다.
- Pod를 실행하기 전에 해당 노드에서 사용자 TGT와 ccache 소유권을 검증한다.
- AD를 관리하는 접속과 노드를 관리하는 접속에 같은 키를 쓰지 않는다.
- NFS에서 D-state가 보이면 mount나 서비스를 반복해서 다시 시작하지 않는다.

2. 파일과 서비스가 있는 곳

```
config-server
├── AD 관리 채널 -> AD DC
│   └── 사용자 + <user>_gid + RFC2307 UID/GID + 사용자 keytab
├── Kubernetes API -> Secret krb5-keytab-<user>
└── 노드 관리 채널 -> Pod가 배치될 FARM 노드 한 대
    ├── /etc/decs-krb/keytabs/<user>.keytab    root-only
    ├── decs-krb-refresh@<user>.timer
    ├── /run/user/<uid>/krb5cc_aialab          사용자 소유
    └── Pod는 ccache와 FARM 홈만 공유
```

AD 사용자와 Secret은 계정이 있는 동안 유지한다. 노드의 keytab, 환경 파일, timer, ccache는 Pod가 올라간 노드에 서만 쓴다. Pod를 지우면 노드의 파일은 지우지만 AD 사용자와 Secret은 다음 Pod를 위해 남긴다.

3. 계정과 인증 파일

항목	위치	사용하는 곳	권한
사용자·전용 그룹	FARM AD	config-server, winbind, NAS	RFC2307 UID/GID 일치
사용자 keytab	Kubernetes Secret	config-server	관리자만 관리
노드 keytab	선택된 FARM 노드	root, refresh service	root 소유 0400

항목	위치	사용하는 곳	권한
갱신 환경 파일	선택된 FARM 노드	refresh service	root 소유 0400
사용자 ccache	/run/user/<uid>/krb5cc_<ai lab>	Pod, 호스트 NFS 경로	사용자 UID/GID, 0600
머신 ccache	/tmp/krb5cc_0	rpc.gssd	현재 노드 머신 principal 전용

/tmp/krb5cc_0에는 현재 노드의 머신 ticket만 둔다. 여기에 사용자 ticket이나 다른 Realm ticket을 넣으면 NFS 인증이 달라질 수 있다. 사용자 확인과 복구에는 별도 ccache를 사용한다.

4. 계정 생성부터 삭제까지

계정 생성

1. config-server가 UID/GID를 할당하고 계정 파일을 갱신한다.
2. NAS 관리 경로가 같은 숫자 소유권으로 홈 디렉터리를 준비한다.
3. AD 관리 채널이 사용자와 <user>_gid 그룹을 만들고 RFC2307 속성을 설정한다.
4. AD가 사용자 keytab을 발급하고 config-server가 Kubernetes Secret에 저장한다.

마지막 단계가 실패하면 계정 생성도 실패한다. NAS 홈과 계정 파일은 되돌리려고 하지만, AD 사용자·전용 그룹·Secret 일부가 남을 수 있다. 같은 이름으로 다시 만들기 전에는 이 세 곳을 확인한다.

Pod 시작

1. config-server가 대상 FARM 노드를 선택한다.
2. Secret의 keytab을 해당 노드의 제한된 관리 채널로 전달한다.
3. 노드는 root 전용 keytab·환경 파일·decs-krb-refresh@<user>.timer를 준비한다.
4. refresh service가 사용자 TGT를 발급하고 ccache 소유권을 검증한다.
5. 검증에 성공할 때만 Pod가 ccache와 FARM 홈을 hostPath로 공유하며 시작한다.

Pod·계정 삭제

Pod 삭제는 해당 노드의 timer, keytab, 환경 파일과 ccache를 정리한다. 계정 삭제는 AD 사용자와 Secret을 지우고 모든 FARM 노드에 사용자 keytab, timer, ccache 정리를 요청한다. 전용 그룹은 자동으로 지우지 않는다. 지울 필요가 있으면 운영 규칙을 확인한 뒤 따로 처리한다.

5. 접속 경로

접속 경로	대상	할 수 있는 일
AD 관리 채널	AD DC	사용자·전용 그룹 생성/삭제, keytab 발급
노드 관리 채널	FARM 실행 노드	keytab 배포·삭제, timer 제어, ccache 검증

접속 경로	대상	할 수 있는 일
NAS 관리 채널	NAS	홈 생성·소유권 설정·삭제

각 접속 경로는 다른 키와 forced-command를 사용한다. AD를 수정하는 키로 모든 노드에 일반 셸 접속을 할 수 없게 한다.

6. 바꾸면 안 되는 규칙

1. 사용자 keytab을 Pod에 복사하거나 mount하지 않는다.
2. AD의 UID/GID, config-server 계정 파일, NAS 소유권을 따로 바꾸지 않는다.
3. Pod 시작 전에 대상 노드의 TGT와 ccache 소유권을 확인한다.
4. 머신 ccache에는 현재 노드의 머신 principal만 둔다.
5. D-state가 있는 노드에서 강제 unmount·반복 mount·반복 service restart를 하지 않는다.

kdc-setup 운영

이 문서에서 다루는 내용

- AD, Secret, 노드 keytab, ccache, NFS 상태를 어떤 순서로 확인할지
- 계정과 Pod 생성·삭제 뒤에 어떤 흔적이 남아야 하고 무엇이 지워져야 하는지
- 사용자 단위 장애와 노드 전체 장애를 어떻게 구분할지

빠른 이동

필요할 때	문서
전체 구조와 역할 분담을 먼저 본다	개요
파일 위치와 보안 규칙을 본다	설계
Helm, CI, Secret 값의 위치를 본다	설정

1. 작업할 때 지킬 것

1. 계정과 Pod 생성·삭제는 config-server API로 수행한다.
2. keytab, 개인키, Secret **data** 는 일반 운영 문서·로그·티켓에 출력하지 않는다.
3. 사용자 장애는 해당 사용자와 Pod가 배치된 노드 범위에서 먼저 확인한다.
4. `/tmp/krb5cc_0` 에는 현재 노드의 머신 principal 이외의 ticket을 발급하지 않는다.

5. NFS 관련 D-state가 있으면 반복 restart, 강제 unmount, 연속 mount를 중단한다.

2. 정기적으로 확인할 것

Kubernetes

```
kubectl -n ailab-infra get deploy,pod,cronjob
kubectl -n ailab-infra get events --sort-by=.lastTimestamp
kubectl -n ailab-infra logs deploy/containersssh-config-server --since=30m
```

config-server가 Ready인지, AD 계정 생성과 FARM 노드 설정이 반복해서 실패하지 않는지 확인한다. 사용자 keytab Secret은 값이 아니라 메타데이터만 확인한다.

```
target_user='<user>'
kubectl -n ailab-infra get secret "krb5-keytab-$target_user" \
-o custom-columns=NAME:.metadata.name,CREATED:.metadata.creationTimestamp
```

AD와 노드 인증 파일

AD DC에서 사용자·전용 그룹·RFC2307 번호를 확인하고, 선택 노드에서는 timer와 ccache의 권한·유효성을 확인한다.

```
target_user='<user>'
target_uid='<uid>'

sudo samba-tool user show "$target_user"
sudo samba-tool group show "${target_user}_gid"
sudo systemctl is-enabled "decs-krb-refresh@$target_user.timer"
sudo systemctl is-active "decs-krb-refresh@$target_user.timer"
sudo stat -c '%U %G %a %n' \
  "/etc/decs-krb/keytabs/$target_user.keytab" \
  "/etc/decs-krb/refresh.d/$target_user.env" \
  "/run/user/$target_uid/krb5cc_ailab"
sudo env KRB5CCNAME="FILE:/run/user/$target_uid/krb5cc_ailab" klist
```

keytab·환경 파일은 root 소유 0400, ccache는 대상 UID/GID 소유 0600 이어야 한다. klist -k 로 keytab 내용을 출력하지 않는다.

NFS와 노드 Kerberos ticket

```
systemctl is-active rpc-gssd
findmnt -T /home/tako2/share/user -o TARGET,SOURCE,FSTYPE,OPTIONS
sudo klist -c FILE:/tmp/krb5cc_0
ps -eo pid,stat,comm,args | awk '$2 ~ /^D/'
```

홈 경로 mount, rpc.gssd, 머신 principal, D-state를 함께 확인한다. Pod에서만 문제가 나면 Pod UID/GID와 ccache 공유 경로를 먼저 본다. 노드 전체에서 문제가 나면 머신 ticket, rpc.gssd, NFS mount를 먼저 본다.

3. 계정과 Pod를 만든 뒤 확인할 것

계정 생성 후

1. config-server 계정 파일과 AD RFC2307 UID/GID를 대조한다.
2. Secret 메타데이터, NAS 홈의 숫자 소유권·모드를 확인한다.
3. Pod 생성 뒤 대상 노드의 timer·ccache와 Pod 내부 홈 읽기·쓰기를 확인한다.

Pod 삭제 후

대상 노드에서 timer, keytab, 환경 파일이 제거됐는지 확인한다. AD 사용자와 Secret은 다음 Pod 실행을 위해 남아야 한다.

```
target_user='<user>'
sudo systemctl status "decs-krb-refresh@$target_user.timer" --no-pager
sudo test ! -e "/etc/decs-krb/keytabs/$target_user.keytab"
sudo test ! -e "/etc/decs-krb/refresh.d/$target_user.env"
```

계정 삭제 후

AD 사용자와 사용자 keytab Secret이 사라졌는지, 모든 FARM 노드의 사용자 timer·keytab·ccache가 정리됐는지 확인한다. 전용 AD 그룹은 자동 삭제 대상이 아니므로 임의로 지우지 않는다.

4. 문제별 확인 순서

증상	먼저 확인할 것	금지할 것
AD 생성·keytab 발급 실패	config-server stage, 대상 DC, 동일 이름의 사용자·그룹	DC wrapper의 반복 직접 실행
대상 노드 배포 실패	노드 목록, 제한된 SSH 채널, keytab 파일 모드, service journal	keytab을 복사해 우회
ccache 갱신 실패	DNS·시간·Realm·principal·keytab 모드	keytab 내용을 출력

증상	먼저 확인할 것	금지할 것
Pod 홈 접근 실패	Pod UID/GID, ccache, rpc.gssd, 머신 ticket, mount	NAS를 바로 재시작
NFS D-state	stack·kernel log·NFS 응답·네트워크	반복 mount/unmount 또는 service restart

머신 ccache의 principal이 잘못됐거나 ticket이 만료된 경우에만, 정확한 현재 노드 머신 principal을 확인한 후 별도 승인 절차에 따라 복구한다. 사용자 principal이나 다른 Realm을 지정하지 않는다.

5. 노드 추가·변경

신규 AD DC와 FARM 실행 노드는 먼저 제한된 관리 채널, 시간 동기화, rpc.gssd와 FARM 홈 mount를 준비한다. 시험 노드 한 대에서 시험 사용자로 keytab 권한, timer, ccache, TGT, NFS 읽기·쓰기를 검증한 뒤 노드 목록에 추가한다.

공통 refresh 스크립트·systemd 템플릿의 변경은 사용자 배포 중에도 다시 설치될 수 있다. 시험 노드에서 확인한 뒤 기존 노드의 버전·소유권을 비교하고 확대한다.

kdc-setup 설정

이 문서에서 다루는 내용

- FARM AD 관련 설정이 Helm values, CI secret, Kubernetes Secret, 노드 파일 중 어디에 들어가는지
- 값을 직접 노출하지 않고 배포 상태를 확인하는 방법
- 설정을 바꾸기 전에 범위를 어떻게 나눠 확인해야 하는지

빠른 이동

필요할 때	문서
전체 흐름과 문서 경계를 먼저 본다	개요
파일·서비스 위치와 보안 규칙을 본다	설계
운영 중 점검 순서를 본다	운영

설정이 들어가는 곳

config-server의 FARM AD 설정은 Helm values, CI secret, Kubernetes Secret, 노드의 root 전용 파일에 나뉘어 있다. 실제 비밀 값과 변경 이력은 관리자 전용 시크릿과 배포 설정 문서에만 두며, 이 문서에는 값, 개인키, keytab을 적지 않는다.

CI Secret / 관리자 전용 시크릿 문서

- > Helm values 주입
- > config-server Deployment 환경 변수·Secret mount
- > 제한된 AD/노드 관리 채널
- > 노드 root-only keytab과 사용자 ccache

설정별 사용처

구분	대표 항목	사용하는 곳	확인할 것
Realm	KRB5_REALM	config-server, 사용자 Pod	새 설정은 FARM AD Realm을 쓴다.
노드 관리	FARM_SSH_*, 노드 목록	config-server	AD 관리용 키와 계정을 섞지 않는다.
AD 관리	FARM_AD_SSH_*, DC 목록	config-server	AD DC에서 허용한 명령만 실행한다.
홈 경로	FARM_HOME_MOUNT_ROOT	사용자 Pod	FARM 노드에 이미 마운트된 NFS 홈 경로를 가리킨다.
NAS 홈 관리	NAS SSH 설정	config-server	홈 디렉터리 생성과 삭제에만 쓴다.
사용자 keytab	krb5-keytab-<user> Secret	config-server	Pod에는 마운트하지 않는다.

Kubernetes 배포 확인

값을 출력하지 않고 Deployment가 필요한 환경 변수와 Secret mount를 참조하는지 확인한다.

```
kubectl -n ailab-infra get deploy containersssh-config-server \
-o custom-columns=IMAGE:.spec.template.spec.containers[0].image,SA:.spec.template.spec.serviceAccountName
kubectl -n ailab-infra describe deploy containersssh-config-server
kubectl -n ailab-infra get secret farm-ssh-key farm-ad-ssh-key
```

사용자 keytab Secret은 이름·생성 시각만 확인한다. `kubectl get secret -o yaml`, `jsonpath`로 `data`를 출력하거나 terminal history에 남기는 명령은 사용하지 않는다.

노드에서 확인할 파일

노드는 사용자별 keytab을 `/etc/decs-krb/keytabs/` 아래 root 전용으로 보관하고, `decs-krb-refresh@<user>.timer`가 사용자 ccache를 갱신하도록 준비한다. 노드 머신 ccache와 사용자 ccache는 서로 다른 파일·principal로 분리한다.

대상	설정 기준
사용자 keytab·환경 파일	root 소유, 0400

대상	설정 기준
사용자 ccache	대상 UID/GID 소유, 0600
머신 ccache	현재 노드 머신 principal만 사용
rpc.gssd	노드 Kerberos NFS 경로와 함께 점검
forced-command 계정	일반 셀·포괄적인 sudo 없이 허용 동작만 실행

아직 쓰지 않는 ConfigMap

config-server에 마운트되는 `krb5-conf` ConfigMap은 현재 계정과 Pod 생성에 직접 쓰는 Kerberos 설정 파일이 아닙니다. config-server가 KDC 명령을 직접 쓰게 되면 KDC와 admin server 주소를 어디에서 넣을지 먼저 정한다. 그 전에는 비어 있는 endpoint를 정상 설정으로 보거나 복구에 사용하지 않는다.

설정을 바꾸기 전에 확인할 것

1. AD 관리, 노드 관리, NAS 홈 관리 중 무엇을 바꾸는지 먼저 정한다.
2. 실제 비밀 값은 관리자 전용 시크릿 문서에서만 확인하고 일반 문서에 복사하지 않는다.
3. 시험 노드 한 대와 시험 사용자로 ccache·NFS 접근을 확인한다.
4. 기존 Pod가 이전 Realm·홈 경로를 쓰는지 확인하고, 설정값을 덮어쓰지 않는다.
5. 검증 뒤에만 CI/Helm 설정을 확대 적용한다.